KingbaseES 数据库 SQL 调优指南

金仓数据库管理系统 KingbaseES

文档版本：V9（V009R001C010）

发布日期：2025 年 12 月 3 日

中电科金仓（北京）科技股份有限公司

目　录

第 1 章　SQL 调优概述 1
1.1　SQL 调优基础 1
1.1.1　关于 SQL 调优 1
1.1.2　SQL 调优的目的 1
1.2　SQL 调优步骤 2
1.3　SQL 调优工具 3
1.3.1　诊断技能 4
1.3.2　优化手段 4

第 2 章　查询优化器基础 6
2.1　概述 6
2.2　查询优化器简介 7
2.2.1　逻辑优化 7
2.2.1.1　子查询优化 7
2.2.1.1.1　子查询提升 7
2.2.1.1.2　EXISTS 子连接提升 8
2.2.1.1.3　ANY 子连接提升 8
2.2.1.1.4　合并子查询公共表达式 9
2.2.1.1.5　合并 EXISTS 子连接 10
2.2.1.2　等价谓词重写 11
2.2.1.2.1　扩展 EXISTS 子连接 11
2.2.1.2.2　不等式变换 12
2.2.1.3　连接优化 12
2.2.1.3.1　外连接消除 12
2.2.1.3.2　消除无用表项 13
2.2.1.3.3　半连接转换 14
2.2.1.4　算子移动 14
2.2.1.4.1　谓词下推 14
2.2.1.4.2　UNION 外层条件下推 14
2.2.1.5　语义查询优化 15
2.2.1.5.1　COUNT(DISTINCT) 查询优化 15

2.2.1.5.2 distinct 消除 16
2.2.1.5.3 消除无用的目标列 16
2.2.1.5.4 分组键值消除 16
2.2.2 物理优化 17
2.2.2.1 物理优化概述 17
2.2.2.2 优化器统计信息 17
2.2.2.2.1 统计信息类型 17
2.2.2.2.2 统计信息的收集 19
2.2.2.2.2.1 自动收集 19
2.2.2.2.2.2 主动收集 21
2.2.2.2.3 查看统计信息 22
2.2.2.2.3.1 查看单列统计信息 22
2.2.2.2.3.2 查看扩展统计信息 23
2.2.2.3 优化器基数估计 26
2.2.2.4 代价估算模型 28
2.2.2.4.1 单表扫描路径 28
2.2.2.4.2 连接算法 29
2.2.2.4.2.1 嵌套循环连接（NestLoop） 29
2.2.2.4.2.2 归并连接（MergeJoin） 30
2.2.2.4.2.3 Hash 连接（HashJoin） 31
2.2.2.4.3 连接类型 32

第 3 章 执行计划 34
3.1 查看执行计划 34
3.1.1 使用 EXPLAIN 查看 34
3.1.2 使用对象管理工具查看 35
3.1.3 在日志记录中查看 36
3.2 阅读执行计划 37
3.2.1 主要内容 37
3.2.2 执行顺序 39
3.2.3 计划节点 40
3.2.3.1 扫描节点 41
3.2.3.1.1 顺序扫描（Seq scan） 42
3.2.3.1.2 索引扫描（Index scan） 42
3.2.3.1.3 位图扫描（Bitmap scan） 43
3.2.3.1.4 TID 扫描（TID scan） 43
3.2.3.1.5 覆盖索引扫描（Index only scan） 44
3.2.3.1.6 其他扫描过滤节点 44
3.2.3.2 连接节点 45
3.2.3.2.1 嵌套循环连接（NestLoop join） 45
3.2.3.2.2 哈希连接（Hash join） 45
3.2.3.2.3 归并连接（Merge join） 46

3.2.3.3 物化节点 46
3.2.3.3.1 物化节点（Material） 46
3.2.3.3.2 分组节点（Group） 46
3.2.3.3.3 排序节点（Sort） 47
3.2.3.3.4 去重节点（Unique） 47
3.2.3.3.5 其他物化节点 48
3.2.3.4 控制节点 48
3.2.3.4.1 BitmapAnd/BitmapOr 节点 48
3.2.3.4.2 Result 节点 48
3.2.3.4.3 Append 节点 49
3.2.3.4.4 RecursiveUnion 节点 49
3.3 分析执行计划 49
3.3.1 分析步骤 50
3.3.2 执行计划的改变 51
3.4 控制执行计划（使用 HINT） 52
3.4.1 概述 52
3.4.2 HINT 的使用 52
3.4.3 HINT 的功能 53
3.4.4 配置参数 57
3.4.5 示例 57
3.4.5.1 常用 Scan 类型 HINT 示例 58
3.4.5.2 Parallel 类型 HINT 示例 60
3.4.5.3 ROWS 类型 HINT 示例 61
3.4.5.4 常用 Join 类型 HINT 示例 62
3.4.5.5 Leading 连接顺序 HINT 示例 64
3.4.5.6 SET 类型 HINT 示例 65
3.4.5.7 Blockname 类型 HINT 示例 66
3.4.5.8 Materialize 类型 HINT 示例 67
3.4.5.9 Use_nl_with_index 类型 HINT 示例 68
3.4.5.10 Aggregate 类型 HINT 示例 68
3.4.5.11 Ordered 类型 HINT 示例 70
3.4.5.12 HINT 混合使用示例 71
3.4.6 注意 71

第 4 章 手动 SQL 优化示例 72
4.1 概述 72
4.2 使用索引 72
4.2.1 概述 72
4.2.2 索引使用问题示例 73
4.2.2.1 缺少索引 74
4.2.2.2 使用表达式索引 76
4.2.2.3 使用联合索引 77

4.2.2.4 使用索引提升 Like 模式匹配性能 79
4.2.2.4.1 前匹配或者精确匹配使用 Btree 索引 79
4.2.2.4.2 后匹配 reverse() 表达式 80
4.2.2.4.3 中间匹配使用 TRGM 索引 81
4.3 更新统计信息 81
4.3.1 统计信息未及时更新 82
4.3.2 基数估计不准 83
4.4 增大 work_mem 84
4.5 使用分区 85
4.6 使用物化视图 90
4.7 缓存执行计划 91
4.7.1 PBE 扩展协议下执行计划的缓存 92
4.7.2 Q 报文简单协议下执行计划的缓存 92
4.8 调整性能参数 94
4.8.1 成本参数 94
4.8.2 节点开关控制参数 95
4.8.3 多表连接参数 95
4.8.4 内存设置相关参数 96
4.8.5 其他相关参数 97
4.9 使用并行 98
4.9.1 并行查询原理 98
4.9.2 并行查询相关参数 99
4.9.3 并行查询支持的范围 101
4.9.3.1 扫描节点 101
4.9.3.2 连接节点 102
4.9.3.3 物化节点 104
4.9.3.4 控制节点 104
4.9.4 Worker 进程数与查询性能关系 105
4.10 改写 SQL 107
4.10.1 Query Mapping 的功能 107
4.10.2 Query Mapping 的使用 108
4.10.3 示例 109

第 5 章 自动 SQL 优化组件 115
5.1 使用 SQL 优化建议器 115
5.1.1 概述 115
5.1.2 调优报告说明 116
5.1.3 调优接口说明 117
5.1.4 调优建议报告生成 118
5.1.4.1 基本配置 118
5.1.4.2 生成统计信息更新建议 119
5.1.4.3 生成索引建议 120

5.1.4.4 生成 SQL 改写建议 120
5.1.4.5 使用 queryId 生成建议 123
5.2 SQL 监控 125
5.2.1 SQL 监控的功能 126
5.2.2 SQL 监控的目的 126
5.2.2.1 SQL 监控的示例 126
5.2.3 SQL 监控的外部接口 131
5.2.3.1 SQL 监控参数 131
5.2.3.2 DBMS_SQL_MONITOR 子程序 132
5.2.3.3 SQL 监控视图 133

第 6 章 使用 KDB_HINT_PLAN（已废弃） 134
6.1 概述 134
6.2 KDB_HINT_PLAN 的功能 134
6.2.1 Scan method 135
6.2.2 Join method 135
6.2.3 Join order 135
6.2.4 Row number correction 136
6.2.5 Parallel query configuration 136
6.2.6 GUC setting 136
6.3 KDB_HINT_PLAN 的使用 136

版权声明 143

服务周期承诺 144

第1章　SQL 调优概述

1.1　SQL 调优基础

随着互联网技术在商业应用中的广泛应用，基于数据库构建的应用系统规模日益庞大且结构复杂，这使得数据库性能的重要性愈发显著。在众多数据库性能问题中，SQL 语句的执行效率占据了较大比重，因此 SQL 性能调优成为数据库管理员进阶过程中必须掌握的关键技能。凭借多年积累的性能优化经验，KingbaseES 总结并提供了一套系统化的 SQL 调优方法论供读者参考使用。

本章节包含以下内容：

- 关于 SQL 调优
- SQL 调优的目的

1.1.1　关于 SQL 调优

SQL 调优是针对未达标性能指标的 SQL 语句进行识别与优化的过程，并通过持续迭代调整以确保达成既定的可量化优化目标。

1.1.2　SQL 调优的目的

当 SQL 语句未达预期性能指标时，即构成待解决的性能问题。

在确定问题所在后，典型的性能调优至少包含一个以下目标：

- 减少用户响应时间，这意味着减少用户发出请求到接收请求的时间。
- 提高吞吐量，这意味着使用最少的资源来处理请求。

考虑这样一个场景，对于一条执行时间为 30 秒的 SQL 语句，在硬件资源充足的情况下，可采用并行处理方式来缩短用户响应时间；而当硬件资源紧张时，为提升吞吐量，则需借助索引、HINT 等技术减少资源占用。不同的问题是基于不同的优化目标设定。

1.2　SQL 调优步骤

在确定调优目标后（例如将用户响应时间从 60 秒降至 5 秒），问题转变为如何实现这一目标。

KingbaseES 提供自动调优和手动 SQL 调优两种方式：自动调优功能可协助 DBA 识别 SQL 问题并提供建议；而手动调优则要求 DBA 依据 SQL 执行计划执行优化。

SQL 调优通常是一个迭代的过程，为了实现目标，典型的 SQL 调优需要不断重复以下一个或多个步骤：

1. 识别高负载语句

通过查看过去的执行历史记录，识别消耗较多系统资源的 SQL 语句。KingbaseES 提供的工具包括：

- sys_stat_statement
- kbbadger

以上两种工具使用方法请参见《KingbaseES 数据库性能调优指南》" kbbadger 日志分析工具" 章节。

- SYS_KWR

使用方法请参见《KingbaseES 数据库性能调优指南》" 生成 KWR 快照" 章节。

- SQL_MONITOR

使用方法请参见本手册 "SQL 监控 " 章节。

2. 收集性能相关的数据

优化器统计信息是 SQL 调优的关键因素；如果统计信息不完整或不及时、不准确，则优化器将无法生成最优执行计划。与 SQL 性能相关的数据包括该语句访问的表和视图的结构，以及该语句可用索引的定义，这些信息均便于后续进行性能问题分析。

如何收集统计信息请参考本手册章节统计信息的收集 和扩展统计信息

3. 确定性能问题产生的原因

对于特定 SQL 语句，确定性能问题产生的原因需要分析执行计划和使用 SQL 优化建议器。SQL 优化建议辅助分析给出 SQL 调优的建议，使用方法参考本手册使用 SQL 优化建议器章节。

分析方法参见本手册章节分析执行计划

通常来说，SQL 性能问题的原因包括：

- 缺少 SQL 访问结构

缺乏索引、物化视图、分区等 SQL 访问结构是造成 SQL 性能不佳的常见原因。选用最佳的访问结构可显著提升 SQL 性能，有时甚至能将其效能提升至几个数量级。

索引部分请参考本手册章节使用索引

物化视图请参考本手册章节使用物化视图

分区表优化请参考本手册章节使用分区

- 次优的执行计划

优化器的主要作用是生成最优（或较优）的执行计划。由于优化器自身局限性，有时会选择不理想的访问路径。例如，对低选择率条件的大表采用全表扫描而非使用索引；多表查询时未能让具有更强筛选能力的两个表优先连接。

- 陈旧的优化统计数据

当统计信息维护操作（无论是自动还是手动执行）未能及时跟踪表数据因 DML 操作而发生的变化时，所收集的统计信息可能会过时。由于表上存在陈旧的统计信息，这些信息无法准确反映当前表的数据情况，导致优化器基于错误的信息做出决策并生成次优的执行计划。

更多信息，参见本手册章节优化器统计信息

- 低效的 SQL 语句设计

  如果 SQL 语句被设计成执行不必要的操作，则优化器将无法进行多项性能提升工作。低效设计的例子包括：

  – 忽略添加连接条件，导致笛卡尔积连接

  – 指定 UNION 而不是 UNION ALL

  – 使子查询针对外部查询中的每一行执行

- 硬件问题

由于当前硬件环境（包括 CPU、内存、IO 和网络等）的限制，所有优化手段均无法实现预期性能目标，则需要通过硬件扩展来提升性能。

- 架构问题

单机硬件扩展无法解决的情况下，则需要考虑读写分离集群等方案。

4. 定义问题的范围

解决方案的范围应当与问题的范围相匹配。在数据库级别考虑的问题与在语句级别解决问题的方法之间存在差异。

例如，在单个 SQL 语句未使用有用的索引的情况下，更改整个数据库的优化器参数（enable_seqscan = off）可能会损害整体性能。如果单个 SQL 语句有问题，则可以考虑通过使用 QueryMapping 的方式将影响范围限制在语句级。

5. 实施优化手段

在步骤 3 中，可以得知由于 SQL 性能问题产生的原因多种多样，因此优化手段的选择需要根据具体情况实施。例如，可通过将 UNION 改写为 UNION ALL 来提升性能，也可通过创建索引来提高执行效率。

6. 评估优化效果

在实施优化手段后，通过实际执行来评估优化效果是否达到了预期。

1.3　SQL 调优工具

为了方便用户调优，在这里提供两类 SQL 调优工具：

- 诊断技能

用来诊断 SQL 性能问题的理论。

- 优化手段

用来解决 SQL 性能问题的优化手段。

1.3.1　诊断技能

在实施 SQL 性能优化之前，掌握必要的数据库基础知识至关重要，本指南推荐用户具备下表所列知识与技能。

- SQL 和 PLSQL

由于 GUI 的工具存在，所以很多数据库用户即使不具备 SQL 知识，用户仍可进行数据库应用开发与管理，这在很大程度上降低了数据库开发门槛。这虽降低了数据库开发门槛，却也意味着若无 SQL 基础，用户将无法实施应用程序及数据库的性能调优。

更多信息，请参考 SQL 语言和 PLSQL 过程语言。

- 数据库查询优化器基础

若想实现更高效的 SQL 性能调优，建议深入理解数据库查询优化器的基本原理，并熟悉其中的逻辑优化与物理优化机制。

更多信息，请参见本手册“查询优化器基础 ”章节。

- 执行计划

执行计划是 SQL 性能调优的关键工具；通过对其中的详细信息进行分析，可以找到影响查询效率的根本原因，并据此制定有效的优化策略。

更多信息，请参见本手册“执行计划 ”章节。

- SQL 监控

DBA 可通过 SQL 监控功能实时获取 SQL 语句运行期间的资源占用情况、执行时长及执行计划等相关信息。

更多信息，请参考本手册“SQL 监控”章节。

1.3.2　优化手段

KingbaseES 提供了多种手段来供用户做 SQL 调优，详见手动 SQL 优化示例 ，常见的调优手段包括：

- 使用索引
- 更新统计信息
- 增大 work_mem
- 使用分区
- 使用物化视图
- 缓存执行计划

- 调整性能参数
- 使用并行
- 改写 SQL

第2章　查询优化器基础

2.1　概述

查询优化是 SQL 性能调优的核心内容。在数据库系统中，SQL 语句仅明确查询目标而未指定具体执行方法，其实现细节完全由数据库系统内部处理机制完成。当查询处理过程存在效率瓶颈时，将导致系统性能下降问题。

虽然数据库开发人员在编写 SQL 语句时会基于自身知识和经验尽可能优化查询性能，但由于应用程序通常需要编写大量复杂的查询语句，且这些语句的逻辑结构可能非常复杂，因此很难保证每个 SQL 语句都能达到理想的执行效率。

在 KingbaseES 中，SQL 语句的处理通常都要经历以下 5 个阶段。

1. **词法语法分析阶段**（parser **子系统**）

   检查 SQL 语句是否符合词法语法规则，然后转化为一颗解析树。

2. **语义检查阶段**（analyzer **子系统**）

   检查解析树是否符合语义规则，然后转化为一颗查询树。

3. **查询重写阶段**（rewritter **子系统**）

   如果查询树中涉及视图或者规则，重写为查询树。

4. **查询优化阶段**（planner **子系统**）

   经过逻辑优化和物理优化，生成最优的执行计划。

5. **查询执行阶段**（executor **子系统**）

   按照执行计划进行执行。

在数据库查询处理的五个阶段中，查询优化阶段决定了 SQL 语句将以何种方式执行。因此，查询优化器是提升查询效率的重要工具，并且其优化过程对数据库开发人员来说是透明化的。

查询优化器的核心功能是生成最优的执行计划（Execution Plan）。对于给定的一条 SQL 语句，通常存在一个或多个可供选择的执行方案，每个方案都能准确返回预期结果。优化器将基于预估成本和性能分析，选择预计运行效率最高的执行路径。

为了实现最优化的执行规划，KingbaseES 数据库系统集成了以下两种关键优化技术：

- 基于规则的逻辑优化(RBO)
- 基于代价估算的物理优化(CBO)

这两种优化技术均以自动化的方式实现，能够显著提升数据库系统的运行效率，并最大程度地降低开发人员的手动干预需求。同时，深入理解这些优化机制也有助于开发者在编写更高效的 SQL 语句，从而进一步提升查询性能。

2.2　查询优化器简介

2.2.1　逻辑优化

本章节介绍 KingbaseES 优化器支持的逻辑优化规则及其使用方法。KingbaseES 数据库优化器在生成具体的执行计划之前，会基于关系代数理论，对输入的 SQL 语句进行一系列等价变换处理，将原始查询转换为更高效的执行方式。

KingbaseES 优化器支持的逻辑优化规则可分为两大类：其中，能够确保变换后性能一定有所提升的规则被称为启发式规则；而另一类规则则需要优化器根据代价估算自动决定是否采用，这类规则称为基于代价的规则。参数 kdb_rbo.rbo_rule 用于控制基于代价的逻辑优化规则的启停状态，默认设置为 off 表示该功能处于关闭状态。用户可根据实际需求选择：强制使用时将参数值设为 on；需要优化器根据代价估算自动选择最优策略时则设为 cost。

本章后续的示例内容基于以下表结构定义：

```
CREATE TABLE t1(a INT, b INT);

CREATE TABLE t2(a INT, b INT);

CREATE TABLE t3(a INT PRIMARY KEY, b INT);
```

本章将重点介绍以下几类关键逻辑优化规则。

2.2.1.1　子查询优化

2.2.1.1.1　子查询提升

优化器支持将 FROM 子句中的子查询上移至父查询，使其与父查询直接进行连接操作。这种子查询提升特性为启发式规则，通过提供更多的连接顺序和方式选择，使优化器能够生成更优的执行计划。

下面的例子将表 t1 与子查询 v 的连接转换为表 t1 与表 t2 的连接。

```
EXPLAIN (costs off) SELECT * FROM t1, (SELECT t2.b FROM t2) v WHERE t1.a = v.b AND t1.a < 10;
           QUERY PLAN
--------------------------------
 Merge Join
   Merge Cond: (t1.a = t2.b)
```

```
   ->  Sort
         Sort Key: t1.a
         ->  Seq Scan on t1
               Filter: (a < 10)
   ->  Sort
         Sort Key: t2.b
         ->  Seq Scan on t2
(9 行记录)
```

2.2.1.1.2　EXISTS 子连接提升

若 EXISTS 子连接中包含父查询表的列，则该子连接被称为相关子连接。KingbaseES 优化器针对此类结构提供特定优化：将相关 EXISTS 子连接提升为半连接操作，将相关 NOT EXISTS 子连接优化为反连接操作；而对于不相关的 EXISTS 子连接则保持原有结构不变。此外，当连接条件为等值匹配时，优化器会为子连接增加去重操作，并将其转换为内连接操作。

```
EXPLAIN (costs off) SELECT * FROM t1 WHERE EXISTS(SELECT 1 FROM t2 WHERE t1.a > t2.a);
          QUERY PLAN
------------------------------
 Nested Loop Semi Join
   Join Filter: (t1.a > t2.a)
   ->  Seq Scan on t1
   ->  Materialize
         ->  Seq Scan on t2
```

2.2.1.1.3　ANY 子连接提升

使用 IN、SOME 或 ANY 关键字的子连接统称为 ANY 子连接。当这些子连接中不涉及父查询变量时，优化器会自动将基础的 ANY 子连接转换为半连接操作，将 NOT ANY 子连接转换为反连接操作。这种转换策略属于启发式规则，通过将子连接转换为特定的连接类型，使得子连接中的表只需进行一次扫描。这种方式不仅提高了执行效率，还为优化器提供了更多的优化选择空间。

示例 1：将 ANY 子连接变成 t1,t2 表的半连接。

```
EXPLAIN (costs off) SELECT * FROM t1 WHERE a >= ANY(SELECT a FROM t2);
          QUERY PLAN
-------------------------------
 Nested Loop Semi Join
   Join Filter: (t1.a >= t2.a)
   ->  Seq Scan on t1
   ->  Materialize
         ->  Seq Scan on t2
```

示例 2：将 NOT ANY 子连接变成 t1,t2 表的反连接。

```
EXPLAIN (costs off) SELECT * FROM t1 WHERE a NOT IN(SELECT a FROM t2);
         QUERY PLAN
----------------------------
 Hash Anti NA Join
   Hash Cond: (t1.a = t2.a)
   ->  Seq Scan on t1
   ->  Hash
         ->  Seq Scan on t2
```

2.2.1.1.4　合并子查询公共表达式

对查询语句中各子查询内的公共表达式进行识别并实施合并策略，以减少重复的表访问和连接操作次数，从而提高整体查询性能。

该特性缺省为关闭状态，使用时需要设置以下参数：

```
-- 方法 1: 在数据库中配置
SET kdb_rbo.rbo_rule = on;
SET kdb_rbo.enable_merge_comm_expr = on;
-- 方法 2: 在 kingbase.conf 文件中配置
kdb_rbo.rbo_rule = on
kdb_rbo.enable_merge_comm_expr = on
```

该变换为基于代价的逻辑优化规则，支持两种使用场景：

- 场景一：SQL 中存在重复的子查询以及多个子查询中存在同样的表达式，可以将这几个子查询合并。例如下面的例子，SELECT a FROM t2 WHERE t2.b > 10 和 SELECT b FROM t2 WHERE t2.b > 10 在执行计划中被合并成了一个公共的查询 SELECT a,b FROM t2 WHERE t2.b > 10, 从而对表 t2 的扫描次数减少为一次。相似的子查询在 SQL 中出现的次数越多并且合并后查询出的结果集越小，则该规则对 SQL 的性能提升越好。

```
EXPLAIN (costs off) SELECT * FROM (SELECT a FROM t2 WHERE t2.b > 10) v1,  (SELECT b FROM t2 WHERE t2.b > 10)
v2 WHERE v1.a = v2.b;
                        QUERY PLAN
-------------------------------------------------------------
 Hash Join
   Hash Cond: (generated_cte_1.a = generated_cte_1_1.b)
   CTE generated_cte_1
     ->  Seq Scan on t2
           Filter: (b > 10)
   ->  CTE Scan on generated_cte_1
   ->  Hash
         ->  CTE Scan on generated_cte_1 generated_cte_1_1
```

- 场景二：多个 UNION ALL 分支中存在共同访问的表，且公共表涉及到的连接字段和过滤条件相同，则可以提取这些公共表和原来的 UNION ALL 进行连接。

```
EXPLAIN (costs off)
SELECT t1.a,t2.b FROM t1,t2 WHERE t1.a = t2.b AND t2.b = 12
union all
SELECT t1.a,t2.a FROM t1,t2 WHERE t1.a = t2.b;
           QUERY PLAN
----------------------------------------
 Merge Join
   Merge Cond: (t1.a = t2_1.b)
   ->  Sort
         Sort Key: t1.a
         ->  Seq Scan on t1
   ->  Sort
         Sort Key: t2_1.b
         ->  Append
               ->  Seq Scan on t2 t2_1
                     Filter: (b = 12)
               ->  Seq Scan on t2
(11 行记录)
```

2.2.1.1.5　合并 EXISTS 子连接

当 SQL 语句中包含多个（NOT）EXISTS 子连接且这些子连接之间存在”包含”或”被包含”的关系时，优化器将通过转换生成新的单一（NOT）EXISTS 子连接或布尔表达式，从而减少重复执行子连接的次数。、

这一转换策略属于基于代价的逻辑优化规则，默认处于关闭状态，需要配置以下参数以启用该功能：

```
SET kdb_rbo.rbo_rule = on;
SET kdb_rbo.enable_coalesce_exists_sublink = on;
```

例如，

- 子连接 S1，S2 相同。

  `WHERE EXISTS(S1) AND/OR EXISTS(S2)` 可以转换为 `WHERE EXISTS(S1)`。

  `WHERE EXISTS(S1) AND NOT EXISTS(S2)` 可以转换为 `WHERE FALSE`。

  `WHERE EXISTS(S1) OR NOT EXISTS(S2)` 可以转换为 WHERE TRUE。

- 子连接 S1 包含 S2。

  `WHERE EXISTS(S1) AND EXISTS(S2)` 可以转换为 `WHERE EXISTS(S2)`。

  `WHERE EXISTS(S1) OR EXISTS(S2)` 可以转换为 `WHERE EXISTS(S1)`。

- 子连接 S1，S2 处于不同析取合取层的合并。例如下面的例子 S1,S2 相同时，

  `WHERE (A AND EXISTS(S1)) OR EXISTS(S2)` 可以转换为 `WHERE EXISTS(S2)`

```
EXPLAIN (costs off) SELECT * FROM t1 WHERE
(t1.a > 10 AND EXISTS(SELECT 1 FROM t2 WHERE a < 10 AND t2.a < t1.a))
OR
EXISTS(SELECT 1 FROM t2 WHERE a < 10 AND t2.a < t1.a);
           QUERY PLAN
--------------------------------
 Nested Loop Semi Join
   Join Filter: (t1.a > t2.a)
   ->  Seq Scan on t1
   ->  Materialize
         ->  Seq Scan on t2
               Filter: (a < 10)
```

2.2.1.2　等价谓词重写

2.2.1.2.1　扩展 EXISTS 子连接

针对 SELECT 子句中的 EXISTS 子连接，若其 WHERE Clause 的顶层判断条件为 OR 关系，并且其中部分 OR 条件未涉及子连接所访问表的列，则可以将这部分条件扩展为新的独立子连接，并与原子连接形成逻辑或的关系。通过这种优化方式，能够有效利用索引机制，缩短 SQL 语句的执行时间。

该转换规则属于基于代价的逻辑优化策略，默认处于关闭状态。在实际应用中需要配置相关参数以启用此功能：

```
SET kdb_rbo.rbo_rule = on;
SET kdb_rbo.enable_exists_expand = on;
```

例如，EXISTS 子连接中存在外层查询的变量 t1.b <10，子连接可以等价转换为 exists (SELECT 1 FROM t2 WHERE t2.a = 1) OR EXISTS (SELECT 1 FROM t2 WHERE t1.b <10)。

```
EXPLAIN (costs false) SELECT * FROM t1 WHERE EXISTS (SELECT 1 FROM t2 WHERE t2.a = 1 OR t1.b <10);
               QUERY PLAN
----------------------------------------
 Seq Scan on t1
   Filter: ($0 OR (SubPlan 2))
   InitPlan 1 (returns $0)
     ->  Seq Scan on t2
           Filter: (a = 1)
   SubPlan 2
     ->  Result
           One-Time Filter: (t1.b < 10)
           ->  Seq Scan on t2 t2_1
```

2.2.1.2.2　不等式变换

对于满足特定转换规则的多个不等式约束条件，优化器将实施等价变换处理。这种转换能够有效消除冗余过滤条件，并更好地利用索引机制，从而显著提升查询性能。

对于多个不等式约束条件进行析取和合取操作，优化器可以进行等价变换处理。经过转换后的新 SQL 语句能够有效消除冗余过滤条件，并更好地利用索引机制，从而实现性能提升的目标。

示例 1：and 条件保留取值范围小的条件

```
EXPLAIN (costs off) SELECT * FROM t1 WHERE t1.a < 156 AND t1.a < 10;
     QUERY PLAN
--------------------
 Seq Scan on t1
   Filter: (a < 10)
```

示例 2：or 条件保留取值范围大的条件

```
EXPLAIN (costs off) SELECT * FROM t1 WHERE t1.a < 156 OR t1.a < 10;
     QUERY PLAN
---------------------
 Seq Scan on t1
   Filter: (a < 156)
```

示例 3：条件冲突时生成 false

```
EXPLAIN (costs off) SELECT * FROM t1 WHERE t1.a < 10 AND a > 10;
        QUERY PLAN
--------------------------
 Result
   One-Time Filter: false
```

2.2.1.3　连接优化

2.2.1.3.1　外连接消除

在满足特定条件的情况下，KingbaseES 优化器能够自动将外连接转换为一种执行效率更高的连接形式。

其基本原理如下：由于外连接会对不满足连接条件的记录补全 NULL 值，因此当查询结果中可以确保完全不存在需要补全 NULL 的情况时，外连接（包括左连接、右连接和全连接）均可转换为内连接以提高性能。若仅需保证某一侧记录无需补全，则可将全连接转换为左连接。

示例 1：条件 t2.b is not null 保证左连接的右侧不存在值为 NULL 的记录，则左连接可以转换内连接。

```
EXPLAIN (costs off) SELECT * FROM t1 LEFT JOIN t2 ON t1.a = t2.a WHERE t2.b IS NOT NULL;
              QUERY PLAN
---------------------------------------
```

```
 Merge Join
   Merge Cond: (t2.a = t1.a)
   ->  Sort
         Sort Key: t2.a
         ->  Seq Scan on t2
               Filter: (b IS NOT NULL)
   ->  Sort
         Sort Key: t1.a
         ->  Seq Scan on t1
(9 行记录)
```

示例 2：条件 t2.b is not null 只能保证全连接的右侧不存在值为 NULL 的记录，则全连接转换右连接。

```
EXPLAIN (costs off) SELECT * FROM t1 FULL JOIN t2 ON t1.a = t2.a WHERE t2.b IS NOT NULL;
              QUERY PLAN
---------------------------------------
 Merge Left Join
   Merge Cond: (t2.a = t1.a)
   ->  Sort
         Sort Key: t2.a
         ->  Seq Scan on t2
               Filter: (b IS NOT NULL)
   ->  Sort
         Sort Key: t1.a
         ->  Seq Scan on t1
(9 行记录)
```

示例 3：条件 t2.a is null 保证连接的右侧全部为补 NULL 的记录，与反连接的要求相似，所以左连接可以转换为反连接。

```
EXPLAIN (costs off) SELECT * FROM t1 LEFT JOIN t2 ON t1.a = t2.a WHERE t2.a IS NULL;
         QUERY PLAN
----------------------------
 Hash Anti Join
   Hash Cond: (t1.a = t2.a)
   ->  Seq Scan on t1
   ->  Hash
         ->  Seq Scan on t2
```

2.2.1.3.2　消除无用表项

在等值连接条件下，若参与连接的表具有唯一约束，则可以考虑消除部分表而不会影响最终的连接结果。这一优化策略属于启发式规则。

KingbaseES 优化器对于左连接和右连接，在满足以下条件时可消除内表：(1) 连接条件中内标的列有唯一性质，

如：唯一索引、distinct 的列等 (2) 内表的相关内容仅出现在连接条件后

```
EXPLAIN (costs off) SELECT t1.* FROM t1 LEFT JOIN (SELECT DISTINCT a FROM t2) v ON t1.a  = v.a;
   QUERY PLAN
----------------
 Seq Scan on t1
```

2.2.1.3.3　半连接转换

当连接条件满足唯一性质时，优化器可以将半连接成功转换为内连接操作。例如，若 IN 子查询中包含 DISTINCT 关键字，则能够保证结果的唯一性，此时最终的执行计划将被优化为内连接方式。

```
EXPLAIN (costs off) SELECT * FROM t1 WHERE a IN (SELECT DISTINCT a FROM t2);
            QUERY PLAN
----------------------------------
 Hash Join
   Hash Cond: (t1.a = t2.a)
   ->  Seq Scan on t1
   ->  Hash
         ->  HashAggregate
               Group Key: t2.a
               ->  Seq Scan on t2
```

2.2.1.4　算子移动

2.2.1.4.1　谓词下推

KingbaseES 优化器会尽可能地将 WHERE 和 ON 子句中的查询条件以及 having 子句中的约束条件下推至基表，从而减少扫描返回的记录数量，提升性能。

```
EXPLAIN (costs off) SELECT a,SUM(b) FROM t1 GROUP BY a HAVING (SUM(b) < 10 AND a=1);
       QUERY PLAN
-------------------------
 GroupAggregate
   Group Key: a
   Filter: (sum(b) < 10)
   ->  Seq Scan on t1
         Filter: (a = 1)
```

2.2.1.4.2　UNION 外层条件下推

在处理涉及 UNION 子查询与其他表的连接操作时，该优化规则能够将连接条件深入下推至 UNION 子查询内部。这一转换使得 UNION 子查询中的表可以利用索引机制，从而显著提高系统性能。

该转换规则属于基于代价的逻辑优化策略，默认处于关闭状态。在实际应用中需要配置相关参数以启用此功能：

```
SET kdb_rbo.rbo_rule = on; -- 或者 kdb_rbo.rbo_rule = cost;
SET kdb_rbo.enable_push_joininfo_to_union = on;
```

例如，在连接条件无法下推的情况下，系统需对 t2 和 t3 表执行全表扫描。通过应用该规则，我们成功将 t3 表的扫描方式从全表扫描优化为索引扫描。

```
EXPLAIN (costs off) SELECT * FROM t1, (SELECT * FROM t2 union SELECT * FROM t3) v WHERE t1.a=v.a;
                    QUERY PLAN
--------------------------------------------------
 Nested Loop
   ->  Seq Scan on t1
   ->  HashAggregate
         Group Key: t2.a, t2.b
         ->  Append
               ->  Seq Scan on t2
                     Filter: (t1.a = a)
               ->  Index Scan using t3_pkey on t3
                           Index Cond: (a = t1.a)
```

2.2.1.5　语义查询优化

2.2.1.5.1　COUNT(DISTINCT) 查询优化

KingbaseES 优化器当前在处理 SUM(DISTINCT)、COUNT(DISTINCT) 和 MAX(DISTINCT) 等去重聚合操作时，无法实施并行运算，仅能采用排序方式实现去重功能。通过开启相应配置参数后，系统将支持并行计算，并提供排序与哈希两种不同的聚合算法供用户选择。

该优化规则属于基于代价的逻辑优化策略，默认处于关闭状态。在实际应用中需要配置相关参数以启用此功能

(1) 参数 `set kdb_rbo.rbo_rule = on;`

(2) 参数 `kdb_rbo.attribute_distinct_value_threshold` 控制重复度的门限值。该参数范围是 [0.0, 1.0]。默认值是 0.1 表示对于去重列，当 100 条记录去重后的个数小于 10 条，则使用该规则。

例如，当 a 列的重复度很低时，强制使用该规则可能导致性能下降。当 a 列的重复度很高时，该规则可以提升查询的性能。

```
EXPLAIN (costs off) SELECT COUNT(DISTINCT a) FROM t1;
            QUERY PLAN
----------------------------------
 Aggregate
   ->  Group
         Group Key: t1.a
         ->  Sort
```

```
                Sort Key: t1.a
                ->  Seq Scan on t1
```

2.2.1.5.2　distinct 消除

当需要去重的列为非空且满足唯一性约束条件时，系统将能够省略去重操作。

这一优化策略属于启发式规则，需确保参数 kdb_rbo.rbo_rule 设为 ON 以启用该特性。例如，在表 t3 中，若列 a 为主键，则对 a 和 b 两列进行去重操作时，系统会自动消除冗余的去重步骤。

```
EXPLAIN (verbose,costs off) SELECT DISTINCT a,b FROM t3;
      QUERY PLAN
------------------------
 Seq Scan on public.t3
   Output: a, b
```

2.2.1.5.3　消除无用的目标列

当子查询的目标列未出现在最终的输出结果中且未被上层查询引用时，KingbaseES 优化器会将该目标列设置为空值（NULL）。这一特性特别适用于目标列包含高耗时操作（如函数调用或复杂表达式）的情况，能够有效提高 SQL 语句的执行效率。

需要注意的是，此功能仅对目标列为 immutable 或者 stable 属性的函数有效。例如，在下面的例子中，sum() 函数具有不变态属性，因此可以成功转换为 NULL；而 random() 函数由于是 volatile 属性，则不会进行任何转换操作。

```
EXPLAIN (verbose,costs off) SELECT COUNT(1) FROM (SELECT SUM(b),RANDOM() FROM t2);
                QUERY PLAN
----------------------------------------
 Aggregate
   Output: count(1)
   ->  Result
         Output: NULL::bigint, random()
```

将函数的属性设为易变（volatile）时，会导致无法使用并行处理、不能创建基于该函数的索引，并且无法消除无用的目标列。如果用户确认特定函数对相同的输入值总能返回相同的结果，则可以手动将其属性修改为” 稳定（stable、immutable）” 以获得更好的优化效果。例如，执行命令”ALTER FUNCTION test1(INT) IMMUTABLE;” 即可完成属性修改。

2.2.1.5.4　分组键值消除

对于包含主键的表，若分组字段包含主键，则该表中出现在分组字段中的其他列均可被移除。例如，在下面的例子中，执行计划仅保留主键 a 作为分组键。

```
EXPLAIN (verbose,costs off) SELECT a, b, COUNT(1) FROM t3 GROUP BY a,b;
         QUERY PLAN
----------------------------
 HashAggregate
   Output: a, b, count(1)
   Group Key: t3.a
   ->  Seq Scan on public.t3
         Output: a, b
```

2.2.2　物理优化

2.2.2.1　物理优化概述

物理优化是基于代价的查询优化（Cost-based Optimizer，简称 CBO），其基本工作原理是利用统计信息进行基数（行数）估计，然后通过代价模型根据估算的行数对每个物理算子进行代价计算，最终在所有候选物理执行方案中选取最优路径。

本章以 SQL 优化为视角，系统阐述 KingbaseES 的统计信息、基数估计及代价模型的工作机制。

- 优化器统计信息
- 优化器基数估计
- 优化器代价模型

2.2.2.2　优化器统计信息

KingbaseES 优化器通过存储表和索引的数据分布统计信息，使得优化器无需实际执行 SQL 语句即可预估表和索引的行数。该优化器的统计信息体系包含单列统计信息和多列扩展统计信息两类：其中单列统计信息以普通数据形式存储于系统表 sys_statistic 中；而多列扩展统计信息则分别存储在系统表 sys_statistic_ext 和 sys_statistic_ext_data 中。

2.2.2.2.1　统计信息类型

KingbaseES 统计信息主要分为四大类：表级统计信息、列统计信息、索引统计信息和多列扩展统计信息。

下表列出了常见的统计信息类型。

统计信息分类	基本统计信息
表级统计信息	• 表的行数 • 表的页面数

-续表-

统计信息分类	基本统计信息
列级统计信息	基本信息： • NULL 值率 • 列的平均宽度 • 列的不同值个数或比例（number of distinct values，NDV）
	数据分布信息： • 高频值（most common values，MCV） • 直方图 • 相关系数
索引统计信息	• 索引的行数 • 索引的页面数
多列扩展统计信息	• 函数依赖 • 多元 N-Distinct 计数 • 多元 MCV 列表

表和索引是数据库中最基本的对象，其统计信息主要存储于 sys_class 系统表中。优化器在执行数据操作时，会依据表或索引的记录总数、磁盘页面数等参数，估算扫描成本，并以此作为选择访问路径的决策依据。

列统计信息是指对每个表的每一个列（也叫属性）的统计信息，它主要存储在 sys_statistic 系统表中，体现数据的分布情况，优化器通过列的数据分布情况来估算过滤条件和连接条件选择率，以及估算分组数量等。可以通过系统表视图 sys_stats 来查看列的统计信息。

系统视图列名	说明
tablename	表名
attname	列名
inherited	是否为继承列，f 为否，即为表列本身，t 为是，表该列继承自其他表列
null_frac	NULL 值的比例，如果没有 NULL 值，为 0。
avg_width	非空项的平均存储宽度，以字节计。
n_distinct	不同值数量或者比例，其取值有 3 种情况: • = 0，代表未知或者未计算的情况。 • > 0，代表消除重复值后的个数，一般不使用这种情况。 • < 0，其绝对值是消重之后的个数占总个数的比例。
most_common_vals	高频值数据
most_common_freqs	高频值数据对应的频率

-续表-

系统视图列名	说明
histogram_bounds	对于不在高频值中的数据生成直方图
correlation	记录当前列未排序的数据分布和排序后的数据分布的相关性

多列扩展统计信息主要存储在 sys_statistic_ext 和 sys_statistic_ext_data 中，可以通过系统表视图 sys_stats_ext 来查看。

系统视图列名	说明
tablename	表名
statistics_name	多列扩展统计信息名
attnames	扩展统计信息中多列的列名列表
kinds	扩展统计信息类型：函数依赖、NDISTINCT、多列 MCV
n_distinct	多列不同值数量或者比例
dependencies	函数依赖统计信息
most_common_vals	多列高频值数据
most_common_freqs	多列高频值数据对应的频率
most_common_base_freqs	基于单列统计信息计算的多列频率

2.2.2.2.2　统计信息的收集

良好查询性能最重要的先决条件是正确的统计信息，陈旧的统计信息可能会导致优化器生成低效的执行计划。KingbaseES 的统计进程并非采用全表扫描的方式收集数据，而是通过随机采样一定数量的元组作为样本，以此生成统计信息。参数 default_statistics_target 的设置直接影响了样本容量：默认情况下该参数值为 100，在计算实际采样数量时，采用 `300 * defualt_statistics_target=30000` 作为采样的样本默认容量。

采样的方式包括：自动收集和主动收集。

2.2.2.2.2.1　自动收集

autovacuum 是 KingbaseES 提供的一个 deamon 进程。每当事务提交或回滚时，系统会向 sysstat 进程发送通知消息。sysstat 接收到这些消息后，会对统计信息进行汇总，并将结果记录到文件中。当发现某个表的修改量超过

预设阈值时，就会触发一次新的统计信息更新操作。具体来说，autovacuum 通过执行 ANALYZE 命令来完成这一任务。与传统的扫描方式不同，ANALYZE 会对表进行随机采样，而非遍历每一行数据。这些预设的阈值以及其他相关配置参数，均包含在数据库配置文件 kingbase.conf 中。主要涉及以下几个方面：

1、**总控** autovacuum **参数：**

- autovacuum：

  是否允许自动 vacuum，默认值是 on。

- log_autovacuum_min_duration：

  默认值为-1，不记录 vacuum 日志。0 记录所有 vacuumlog。

  正整数，单位为微秒，表示 vacuum 执行时间小于该值的不记录，大于该值的的记录 log。

- autovacuum_max_workers：

  最大的 autovacuum 进程的数量，默认值为 3。

- autovacuum_naptime：

  检查数据库的时间间隔。默认为 1 分钟。

2、**自动** vacuum **参数：**

- autovacuum_vacuum_threshold：

  激活自动 vaccum 的最小 DML 操作的行数，默认值 50。

- autovacuum_vacuum_scale_factor：

  该参数采用百分比的方式设定阀值，默认值为 20%。

备注:

表上 update 和 delete 记录总数 >= autovacuum_vacuum_scale_factor * 表上记录数（reltuples）+ autovacuum_vacuum_threshold 后，触发 vacuum。

3、**自动** analyze **参数**

- autovacuum_analyze_threshold：

  激活自动 analyze 操作的最小行数，默认值 50。

- autovacuum_analyze_scale_factor：

  到达阀值时自动激活 analyze 操作，默认值为 10%。

备注:

表上 insert、update 和 delete 记录总数 >= autovacuum_ ANALYZE _scale_factor * 表上记录数（reltuples）+ autovacuum_ ANALYZE _threshold 后，触发自动 analyze。

4、**事务 ID 相关参数**

- autovacuum_freeze_max_age：

  为防止事务 ID 的重置，在启用 vacuum 操作之前，表的 sys_class.relfrozenxid 字段的最大值，默认为 2 亿。

备注:

表上事务的最大年龄配置参数 autovacuum_freeze_max_age，达到这个阀值将触发 autovacuum 进程。

- autovacuum_vacuum_by_snapshotcsn：

  当数据库管理系统执行 VACUUM 或 AUTOVACUUM 清理操作时，该机制能够绕过长期未提交事务（long transactions）的阻塞问题，从而保证垃圾数据的正常清除。对于那些频繁出现长事务的应用场景，当数据库表出现显著膨胀且影响系统响应性能时，建议启用此参数。默认情况下，该参数值为-1，表示功能处于关闭状态。用户可根据实际需求，在 0 到 2,147,483,647 之间设置参数值（最大值为 2^31 - 1）。将参数设置为 0 时，系统会持续开启此功能。

注意:

该功能的使用会对闪回查询机制产生影响。因此，在配置时需要结合具体业务需求进行权衡，并且不建议在同一个表上同时启用这两项功能。

2.2.2.2.2.2　主动收集

主动收集是指用户通过预先设置的自动化脚本或手动执行 ANALYZE 命令来实现。该命令会触发数据库执行统计信息的采集和更新操作。

无参数方式执行 ANALYZE 命令会触发对数据库中所有表的统计信息进行全面更新，由于操作范围较大，所需执行时间通常较长，因此通常不推荐使用这种方式。建议用户在数据发生变化时，有针对性地选择需要更新的表进行分析。对于大表来说，该操作可能会耗费较长时间。如果面对非常大的表，且仅需为部分列生成统计信息，可以使用 `ANALYZE table(column, ...)` 命令。特别需要注意的是，应确保与连接条件、WHERE 子句、SORT 子句、GROUP BY 子句或 HAVING 子句相关的列都被包含在内。

主动收集统计信息的时机一般为：

- 装载大量数据后
- CREATE INDEX 操作后
- 在大量更改底层数据的 INSERT、UPDATE 以及 DELETE 操作之后
- 执行计划的代价估算不准确时

注意:

ANALYZE 命令在执行过程中仅需对目标表施加读锁，因此它可以与其他数据库活动并行运行。然而，建议避免在执行数据装载、INSERT、UPDATE、DELETE 或 CREATE INDEX 操作的过程中运行 ANALYZE。因为这些操作会修改表中的数据分布状态，在完成上述操作后通常需要重新执行 ANALYZE 以更新统计信息。

2.2.2.2.3　查看统计信息

2.2.2.2.3.1　查看单列统计信息

举例说明：（student 表在本章节中作为所有用例使用）

```
CREATE TABLE student(sno INT PRIMARY KEY, sname TEXT, ssex INT, id1 INT, id2 INT);
INSERT into student SELECT i, 'abc_'||MOD(i,100), MOD(i,2),MOD(i,10),MOD(i,10) FROM generate_series(1, 500)
as s(i);
INSERT into student SELECT i, NULL, MOD(i,2),MOD(i,10),MOD(i,10) FROM generate_series(501, 550) as s(i);
INSERT into student SELECT i, NULL, NULL,MOD(i,10),MOD(i,10) FROM generate_series(551,600) as s(i);
CREATE index student_sno_ssex_idx ON student(sno,ssex);
ANALYZE student;
```

通过查询 sys_class 系统表，可以获取表 student 及其索引 student_sno_ssex_idx 的相关信息，包括存储的总页数和行数。

```
SELECT relname,relpages,reltuples FROM sys_class WHERE relname IN ('student','student_sno_ssex_idx');
```

```
test=# select relname,relpages,reltuples from sys_class where relname in ('student','student_sno_ssex_idx');
       relname        | relpages | reltuples
----------------------+----------+-----------
 student              |        4 |       600
 student_sno_ssex_idx |        4 |       600
(2 行记录)
```

列级统计信息可通过系统视图 sys_stats 进行查询。例如，要查看 student 表中学号和性别两列的统计信息：

```
SELECT * FROM sys_stats WHERE tablename='student' AND attname IN ('sno', 'ssex');
```

```
-[ RECORD 1 ]-----------------+-----------------------------------------------------------------------------------------
schemaname                    | public
tablename                     | student
attname                       | sno
inherited                     | f
null_frac                     | 0
avg_width                     | 4
n_distinct                    | -1
most_common_vals              |
most_common_freqs             |
histogram_bounds              | {1,6,12,18,24,30,36,42,48,54,60,66,72,78,84,90,96,102,108,114,120,126,132,138,144,150,156,
162,168,174,180,186,192,198,204,210,216,222,228,234,240,246,252,258,264,270,276,282,288,294,300,306,312,318,324,330,336,342,
348,354,360,366,372,378,384,390,396,402,408,414,420,426,432,438,444,450,456,462,468,474,480,486,492,498,504,510,516,522,528,
534,540,546,552,558,564,570,576,582,588,594,600}
correlation                   | 0.99505407
most_common_elems             |
most_common_elem_freqs        |
elem_count_histogram          |
-[ RECORD 2 ]-----------------+-----------------------------------------------------------------------------------------
schemaname                    | public
tablename                     | student
attname                       | ssex
inherited                     | f
null_frac                     | 0.083333336
avg_width                     | 4
n_distinct                    | 2
most_common_vals              | {0,1}
most_common_freqs             | {0.45833334,0.45833334}
histogram_bounds              |
correlation                   | 0.49726775
most_common_elems             |
most_common_elem_freqs        |
elem_count_histogram          |
```

由于学号列设为主键，其统计信息显示为空值比例为 0%，唯一值数量标记为-1（表示无重复值），因此该列不存在高频值。数据分布情况主要通过直方图来描述。

性别列有两个取值 0 和 1 分别代表不同性别，由于存在 NULL 值，null_frac 是比例，即 null 值数为 600*0.083333336=50 行。不同值数量为 2，即只有两个数 {0, 1}，存在高频值：0:0.45833334，1:0.45833334。

2.2.2.2.3.2　查看扩展统计信息

KingbaseES 系统支持多元统计信息的计算功能。然而，由于不同列之间的组合数量庞大，自动计算所有可能的多元统计信息在实际应用中并不可行。因此，建议用户根据具体需求手动创建扩展统计对象。

1. 函数依赖

下面以 student 表为例展示函数依赖关系统计信息的应用：

```
EXPLAIN ANALYZE SELECT * FROM student WHERE id1=1 AND id2=1;

QUERY PLAN
-----------------------------------------------------------------------------------------------
Seq Scan on student (cost=0.00..13.00 rows=6 width=22) (actual time=0.009..0.091 rows=60 loops=1)
Filter: ((id1 = 1) AND (id2 = 1))
Rows Removed by Filter: 540
Planning Time: 0.236 ms
Execution Time: 0.106 ms

(5 行记录)
```

在实际情况下，属性 id1 和 id2 之间存在函数依赖关系，每一行中 id1 = id2，因此在表 student 上创建依赖关系的多列统计信息：

```
CREATE STATISTICS stts(dependencies) ON id1,id2 FROM student;
ANALYZE student;
```

然后再执行查询时，结果估算正确：

```
EXPLAIN ANALYZE SELECT * FROM student WHERE id1=1 AND id2=1;
               QUERY PLAN
-----------------------------------------------------------------------------------------------------
 Seq Scan on student  (cost=0.00..13.00 rows=60 width=22) (actual time=0.024..0.134 rows=60 loops=1)
   Filter: ((id1 = 1) AND (id2 = 1))
   Rows Removed by Filter: 540
 Planning Time: 0.220 ms
 Execution Time: 0.148 ms
(5 行记录)
```

可以通过 sys_stats_ext 视图查看扩展统计信息：

```
SELECT * FROM sys_stats_ext WHERE tablename='student' AND statistics_name='stts';
```

```
-[ RECORD 1 ]----------+----------------------------------------
schemaname             | public
tablename              | student
statistics_schemaname  | public
statistics_name        | stts
statistics_owner       | coder
attnames               | {id1,id2}
kinds                  | {f}
n_distinct             |
dependencies           | {"4 => 5": 1.000000, "5 => 4": 1.000000}
most_common_vals       |
most_common_val_nulls  |
most_common_freqs      |
most_common_base_freqs |
```

2. 多元 N-Distinct 计数

在 GROUP BY a, b 的情况下，由于统计信息中仅包含单列的统计数据，在组合多于一列时，这种基于单列统计信息的选择会导致执行计划错误，估算结果通常不准确。

为了改善这一估计问题，可以为列组收集多个不同的统计信息。与函数依赖类似，对每个可能的列分组逐一进行统计操作在实际应用中是不可行的。因此，建议仅针对 N-Distinct 选项定义的统计对象中共同出现的列组收集数据。系统将从指定的列集中，为两个或更多列的所有可能组合分别收集数据。

示例如下：

```
EXPLAIN ANALYZE SELECT COUNT(*) FROM student GROUP BY id1,id2;
                QUERY PLAN
-------------------------------------------------------------------------------------------------------
 HashAggregate  (cost=14.50..15.10 rows=60 width=16) (actual time=0.275..0.278 rows=10 loops=1)
   Group Key: id1, id2
```

```
   ->  Seq Scan on student  (cost=0.00..10.00 rows=600 width=8) (actual time=0.007..0.058 rows=600 loops=1)
 Planning Time: 0.076 ms
 Execution Time: 0.306 ms
(5 行记录)
```

通过执行计划可以看到 groupby 的 HashAggregate 节点基数估计偏差很大。当对列 a,b 进行分组时，预估值为 1000，而实际上因为 a,b 都是从 0 到 99 不断循环且值相同，所以应该有 100 个分组。

因此对 t 表建立多列统计信息后，基数估算准确。

```
CREATE STATISTICS stts1(ndistinct) ON id1,id2 FROM student;
ANALYZE student;
EXPLAIN ANALYZE SELECT COUNT(*) FROM student GROUP BY id1,id2;
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------
 HashAggregate  (cost=14.50..14.60 rows=10 width=16) (actual time=0.260..0.263 rows=10 loops=1)
   Group Key: id1, id2
   ->  Seq Scan on student  (cost=0.00..10.00 rows=600 width=8) (actual time=0.007..0.057 rows=600 loops=1)
 Planning Time: 0.119 ms
 Execution Time: 0.291 ms
(5 行记录)
```

可以通过 sys_stats_ext 视图查看扩展统计信息：

```
SELECT * FROM sys_stats_ext WHERE tablename='student' AND statistics_name='stts1';
```

```
-[ RECORD 1 ]----------+-------------
schemaname             | public
tablename              | student
statistics_schemaname  | public
statistics_name        | stts1
statistics_owner       | coder
attnames               | {id1,id2}
kinds                  | {d}
n_distinct             | {"4, 5": 10}
dependencies           |
most_common_vals       |
most_common_val_nulls  |
most_common_freqs      |
most_common_base_freqs |
```

3. 多元 MCV 列表

类似于对 N-DISTINCT 和函数依赖项的操作，对每个可能的列分组逐一执行统计信息收集操作在实际应用中是不切实际的。在此场景下，MCV 列表不同于对 N-DISTINCT 和函数依赖项的操作，专门用于存储相关联列的值。因此，仅针对在 MCV 选项定义的统计对象中同时出现的这些列组合进行数据收集。

示例如下：

添加多元 MCV 列信息校正基数估计：

```
CREATE STATISTICS stts2(mcv) ON id1,id2 FROM student;
ANALYZE student;
EXPLAIN ANALYZE SELECT * FROM student WHERE id1=1 AND id2=1;
```

```
                                        QUERY PLAN
-----------------------------------------------------------------------------------------------
 Seq Scan on student  (cost=0.00..13.00 rows=60 width=22) (actual time=0.011..0.091 rows=60 loops=1)
   Filter: ((id1 = 1) AND (id2 = 1))
   Rows Removed by Filter: 540
 Planning Time: 0.100 ms
 Execution Time: 0.106 ms
(5 行记录)
```

可以通过 sys_stats_ext 视图查看扩展统计信息：

```
SELECT * FROM sys_stats_ext WHERE tablename='student' AND statistics_name='stts2';
```

```
-[ RECORD 1 ]----------+-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
schemaname             | public
tablename              | student
statistics_schemaname  | public
statistics_name        | stts2
statistics_owner       | coder
attnames               | {id1,id2}
kinds                  | {m}
n_distinct             |
dependencies           |
most_common_vals       | {{0,0},{1,1},{2,2},{3,3},{4,4},{5,5},{6,6},{7,7},{8,8},{9,9}}
most_common_val_nulls  | {{f,f},{f,f},{f,f},{f,f},{f,f},{f,f},{f,f},{f,f},{f,f},{f,f}}
most_common_freqs      | {0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1}
most_common_base_freqs | {0.010000000000000002,0.010000000000000002,0.010000000000000002,0.010000000000000002,0.010000000000000002,0.010000000000000002,0
.010000000000000002,0.010000000000000002,0.010000000000000002,0.010000000000000002,0.010000000000000002,0.010000000000000002}
```

2.2.2.3　优化器基数估计

数据库查询优化器的功能实现依赖于统计信息的收集与分析。统计信息的主要作用是为查询执行计划提供准确的基础数据预测依据。KingbaseES 通过统计信息计算表中各列的选择率 (Selectivity)，选择率用于衡量” 过滤条件” 或” 连接条件” 在实际操作中的筛选效率，其取值范围为 [0, 1]，表示经过该条件筛选后的剩余元组数与原数据总行数的比率。

以前面章节中student表为例：

Student 表中有 600 行数据，查看 sno 和 id1 列的单列统计信息：

```
SELECT * FROM sys_stats WHERE tablename='student' AND attname IN ('sno','id1');
```

```
-[ RECORD 1 ]-----------------+-----------------------------------------------------------------
schemaname                    | public
tablename                     | student
attname                       | id1
inherited                     | f
null_frac                     | 0
avg_width                     | 4
n_distinct                    | 10
most_common_vals              | {0,1,2,3,4,5,6,7,8,9}
most_common_freqs             | {0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1}
histogram_bounds              |
correlation                   | 0.107472524
most_common_elems             |
most_common_elem_freqs        |
elem_count_histogram          |
-[ RECORD 2 ]-----------------+-----------------------------------------------------------------
schemaname                    | public
tablename                     | student
attname                       | sno
inherited                     | f
null_frac                     | 0
avg_width                     | 4
n_distinct                    | -1
most_common_vals              |
most_common_freqs             |
histogram_bounds              | {1,6,12,18,24,30,36,42,48,54,60,66,72,78,84,90,96,102,108,114,120,
126,132,138,144,150,156,162,168,174,180,186,192,198,204,210,216,222,228,234,240,246,252,258,264,270,
276,282,288,294,300,306,312,318,324,330,336,342,348,354,360,366,372,378,384,390,396,402,408,414,420,
426,432,438,444,450,456,462,468,474,480,486,492,498,504,510,516,522,528,534,540,546,552,558,564,570,
576,582,588,594,600}
correlation                   | 1
most_common_elems             |
most_common_elem_freqs        |
elem_count_histogram          |
```

```
EXPLAIN ANALYZE SELECT * FROM student WHERE id1 = 9;

QUERY PLAN
-----------------------------------------------------------------------------------------------------
Seq Scan on student (cost=0.00..11.50 rows=60 width=22) (actual time=0.014..0.091 rows=60 loops=1)
Filter: (id1 = 9)
Rows Removed by Filter: 540
Planning Time: 0.077 ms
Execution Time: 0.106 ms

(5 行记录)
```

针对上面的查询，通过 id1 列统计信息的 MCV 值，`id1=9` 的频率是 0.1，计算得到谓词条件 `id1=9` 的选择率为 0.1，基数估计行数 rows=600*0.1=60。

```
EXPLAIN ANALYZE SELECT * FROM student WHERE sno < 100 AND id1 = 9;

QUERY PLAN
-----------------------------------------------------------------------------------------------------------------
-----------
```

```
Index Scan using student_pkey on student (cost=0.28..10.26 rows=10 width=22) (actual time=0.020..0.038
rows=10 loops=1)
Index Cond: (sno < 100)
Filter: (id1 = 9)
Rows Removed by Filter: 89
Planning Time: 0.104 ms
Execution Time: 0.055 ms

(6 行记录)
```

根据上面的查询，通过 id1 列统计信息的 MCV 值，`id1=9` 的频率是 0.1；sno 列统计信息的直方图计算 `sno<100` 的频率为 1/6。基于这两个独立谓词条件的选择率计算，可以得出联合条件 `sno <100 AND id1=9` 的选择率为 `0.1*1/6`，最终得到基数估计行数 `rows=600*0.1*1/6=10`。

KingbaseES 主要采用基于选择率的基数估计方法。在生成执行计划时，优化器会对查询中的每个谓词条件进行选择率计算，并根据整体的选择率进行基数估计。

2.2.2.4　代价估算模型

为了合理高效的评估诸多查询路径的效率，KingbaseES 构建了一个基于资源消耗量化的模型——代价估算模型。该模型能够对不同执行路径的成本进行量化比较，从而确定最优的查询执行方案。

KingbaseES 查询优化代价估算模型主要是基于 CPU 开销和 I/O 开销，即：

总代价 =I/O **代价** + CPU **代价** + **通信代价（并行）**

大体上来说，I/O 和 CPU 的代价估算和需要进行 IO 和 CPU 运算的数据规模相关，而数据规模的估算和基数估算关系密切。

2.2.2.4.1　单表扫描路径

在对数据做任何处理之前，通常都需要获取单表的数据，KingbaseES 中常见的单表扫描方式包括：**顺序扫描**（SeqScan）**和索引扫描**（IndexScan）。

1、顺序扫描/全表扫描

基本算法原理是根据实际的数据的存取顺序，连续扫描表中的所有数据。

主要适用于：数据选择率高的情况。

例如：

```
EXPLAIN ANALYZE SELECT * FROM student WHERE sno > 1;
              QUERY PLAN
-----------------------------------------------------------------------------------------------------
 Seq Scan on student  (cost=0.00..11.50 rows=599 width=22) (actual time=0.008..0.088 rows=599 loops=1)
   Filter: (sno > 1)
```

```
   Rows Removed by Filter: 1
 Planning Time: 0.129 ms
 Execution Time: 0.117 ms

(5 行记录)
```

2、索引扫描

索引扫描的基本原理是利用预构建的索引结构，根据查询条件直接定位符合条件值的位置信息，并在此基础上高效检索所有满足条件的数据记录。

主要适用于：选择率低的情况。在选择率低的情况下，通过索引扫描所需访问的索引页面和数据页面数量将显著少于顺序扫描方式。由于减少了不必要的页面访问操作，相应的 I/O 和 CPU 资源消耗也会降低。

例如：

```
EXPLAIN ANALYZE SELECT * FROM student WHERE sno=10;
                QUERY PLAN
-----------------------------------------------------------------------------------------------------------
-----------------
Index Scan using student_sno_ssex_idx on student  (cost=0.28..8.29 rows=1 width=22) (actual time=0.014..0.
015 rows=1 loops=1)
   Index Cond: (sno = 10)
 Planning Time: 0.086 ms
 Execution Time: 0.031 ms

(4 行记录)
```

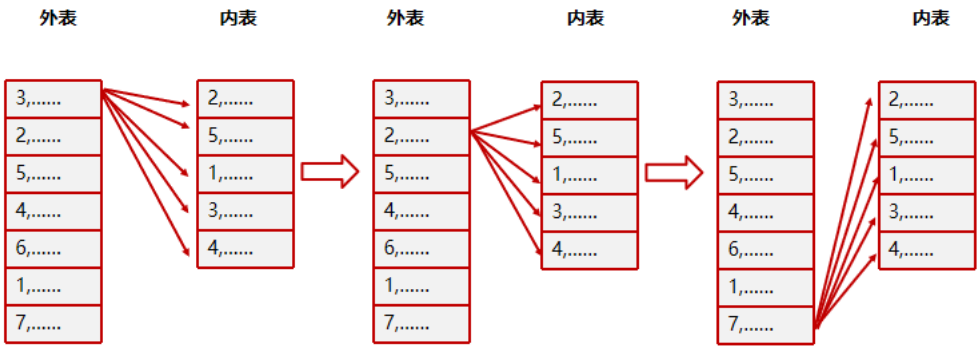
2.2.2.4.2　连接算法

KingbaseES 提供了 3 种类型的连接算法：**嵌套循环连接、归并连接和散列连接**。

2.2.2.4.2.1　嵌套循环连接（NestLoop）

嵌套循环连接是实现两表连接的基础算法，其核心机制基于双重嵌套循环结构。外层循环依次取出外层表中的每一行记录，与内层表的所有记录进行匹配比较，筛选出满足连接条件的元组。

主要适用于：1）内外表数据量不大的情况 2）内表数据量很小，外表数据量大的情况。



例如：创建表 score，插入 10 万条记录，然后跟 student 做循环嵌套连接，student 作为循环外层的表存在的元组条数为 n，score 存在的元组条数为 m。

```
CREATE TABLE score(sid INT, score INT);
INSERT INTO score SELECT sid, (random()*100)::INTEGER AS score FROM generate_series(1, 100000) AS sid;
CREATE INDEX idx_sc on score(sid);
ANALYZE score;
SET max_parallel_workers_per_gather = 0; -- 禁用并行
```

循环嵌套连接时，表 student 将每次拿出一条元组，与 score 的 m 条元组逐一比较。此时在连接中 student 表循环了 1 次、score 表循环了 n 次，共比较了 n*m 次。

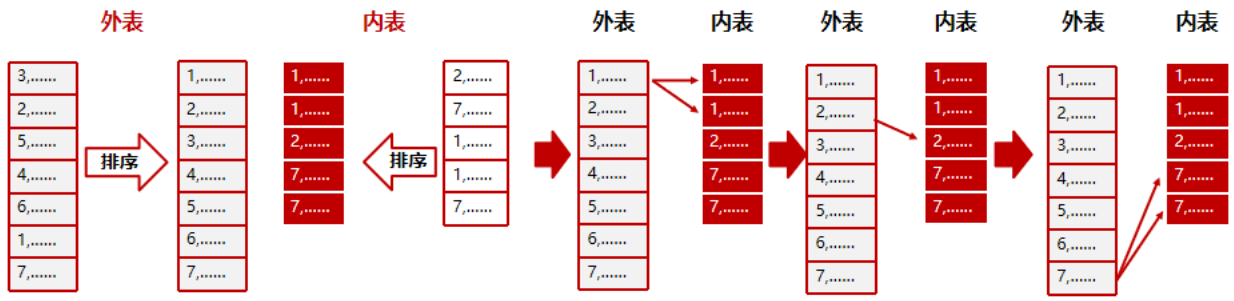
```
EXPLAIN ANALYZE SELECT * FROM student t1 JOIN score t2 ON t1.sno=t2.sid WHERE t1.sno<10 AND t2.sid=1;
             QUERY PLAN
-----------------------------------------------------------------------------------------------------------
--------------------------
 Nested Loop  (cost=0.57..16.62 rows=1 width=30) (actual time=0.024..0.025 rows=1 loops=1)
   ->  Index Scan using student_sno_ssex_idx on student t1  (cost=0.28..8.29 rows=1 width=22) (actual
time=0.008..0.008 rows=1 loops=1)
         Index Cond: ((sno < 10) AND (sno = 1))
   ->  Index Scan using idx_sc on score t2  (cost=0.29..8.31 rows=1 width=8) (actual time=0.013..0.013
rows=1 loops=1)
         Index Cond: (sid = 1)
 Planning Time: 0.326 ms
 Execution Time: 0.047 ms

(7 行记录)
```

2.2.2.4.2.2　归并连接（MergeJoin）

与嵌套循环连接算法不同，归并连接仅适用于自然连接或等值连接场景。该算法要求参与连接的两个表必须是有序的，因此在执行过程中需要对外层表和内层表进行排序操作。具体而言：

- 如果内表或者外表有索引，就不需要进行排序，只对无索引的表进行排序。
- 如果两表都有索引，则两表都不需要排序。

主要适用情况：1）两个表的数据都是基本有序 2）两表可以使用索引或者外表有索引



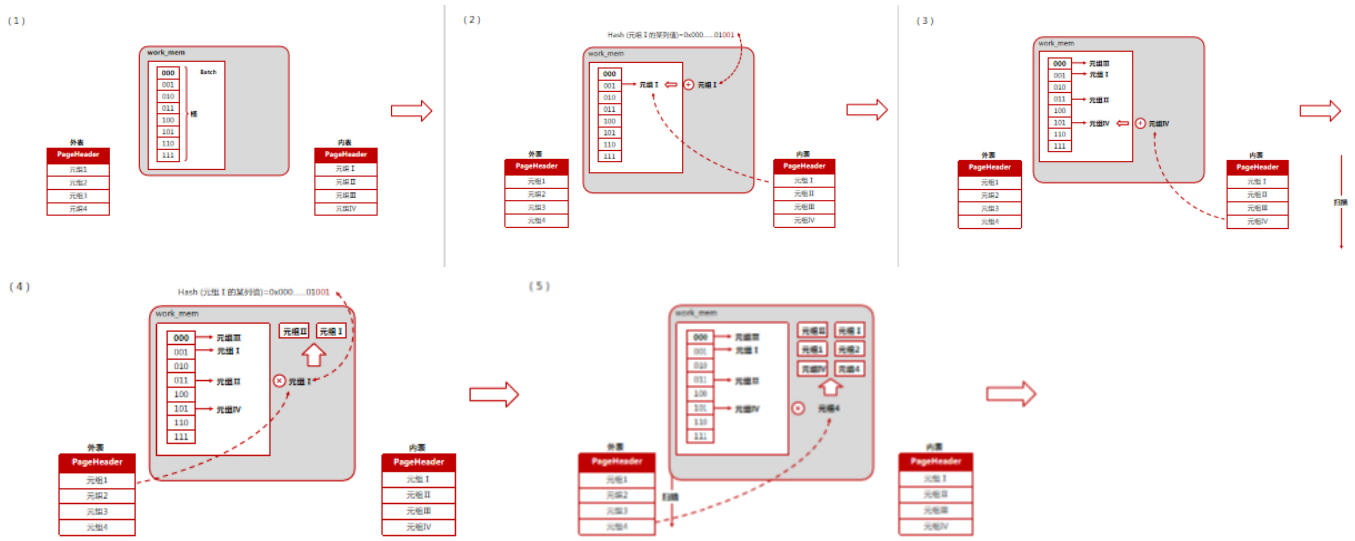
例如：

```
SET max_parallel_workers_per_gather = 0; -- 禁用并行
EXPLAIN ANALYZE SELECT * FROM student t1 JOIN score t2 ON t1.sno=t2.sid WHERE t1.sno<100 AND t2.sid<10 ORDER
BY t1.sno;--使用 order by 让表 student 有序

                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------
----------------------
 Merge Join  (cost=0.57..11.38 rows=1 width=34) (actual time=0.020..0.034 rows=9 loops=1)
   Merge Cond: (t1.sno = t2.sid)
   ->  Index Scan using student_pkey on student t1  (cost=0.28..10.01 rows=99 width=22) (actual time=0.012..
0.015 rows=10 loops=1)
         Index Cond: (sno < 100)
   ->  Index Scan using idx_sc on score t2  (cost=0.29..8.47 rows=10 width=8) (actual time=0.004..0.007
rows=9 loops=1)
         Index Cond: (sid < 10)
 Planning Time: 0.334 ms
 Execution Time: 0.070 ms

(8 行记录)
```

2.2.2.4.2.3　Hash 连接（HashJoin）

与归并连接类似，Hash 连接同样仅适用于自然连接和等值连接场景。

主要适用于：数据分布比较随机无序，重复值不是特别多的情况。HashJoin　原理图：



内存中的 Hash 连接操作在 work_mem 中进行处理。Hash 连接有两个阶段：构建阶段和探测阶段。在构建阶段，内表中的所有元组都会被插入到桶中；在探测阶段，每条外表的元组都会与处理桶中的内表元组比较，如果满足条件则将两条元组连接起来。

例如：

```
SET max_parallel_workers_per_gather = 0; -- 禁用并行
DROP INDEX idx_sc; --删除 score 上的索引防止使用归并连接
EXPLAIN ANALYZE SELECT * FROM student t1 JOIN score t2 ON t1.sno=t2.sid;
                QUERY PLAN
-----------------------------------------------------------------------------------------------------------
--------
 Hash Join  (cost=17.50..1723.01 rows=600 width=30) (actual time=0.158..18.942 rows=600 loops=1)
   Hash Cond: (t2.sid = t1.sno)
   ->  Seq Scan on score t2  (cost=0.00..1443.00 rows=100000 width=8) (actual time=0.007..8.022 rows=100000
loops=1)
   ->  Hash  (cost=10.00..10.00 rows=600 width=22) (actual time=0.144..0.145 rows=600 loops=1)
         Buckets: 1024  Batches: 1  Memory Usage: 40kB
         ->  Seq Scan on student t1  (cost=0.00..10.00 rows=600 width=22) (actual time=0.003..0.057 rows=600
loops=1)
 Planning Time: 0.191 ms
 Execution Time: 18.974 ms

(8 行记录)
```

2.2.2.4.3　连接类型

KingbaseES 支持的连接类型包括：内连接、左外连接、右外连接、全连接、左半连接、左反连接（包含 NA、LNA、RNA）。每种连接类型支持的连接算法如下：

连接类型	连接算法
Inner join	nestloop join、hash join、merge join
Left outer join	nestloop join、hash join、merge join
Right outer join	hash join、merge join
Full join	hash join、merge join
Left semi join	nestloop join、hash join、merge join
Left anti join（包含 NA、LNA、RNA）	nestloop join、hash join、merge join

第3章　执行计划

3.1　查看执行计划

在 KingbaseES 数据库系统中，提供了多种用于查看执行计划的途径。本节旨在详细介绍几种常用的查看方法：

- 使用 EXPLAIN 查看
- 使用对象管理工具查看
- 在日志记录中查看

下表列出了三种方式的特点及其适用场景：

查看方式	特点
使用 EXPLAIN 命令查看	可查看指定 SQL 语句在当前 session 下的执行计划，调试方便
使用对象管理工具查看	
使用 AUTO_EXPLAIN 查看	可记录所有后台 SQL 语句的执行计划：ksql、jdbc 等

3.1.1　使用 EXPLAIN 查看

KingbaseES 中 EXPLAIN 命令来查看执行计划时最常用的方式。其命令格式如下：

```
EXPLAIN [option] statement
```

其中 option 为可选项，可以是以下 5 种情况的组合：

- analyze：执行命令并显示执行事件，默认 false
- verbose：显示附加信息，例如执行计划中每个节点的字段详情，默认值为 false。

- costs：显示执行计划的成本估算，默认值为 true。
- buffers：展示缓冲区使用情况，包括共享块、本地块和临时读写块，默认值为 false，需在启用 ANALYZE 后有效。
- format：指定执行计划的输出格式，支持：TEXT、XML、JSON 或者 YAML，默认是 text

下面举例说明 EXPLAIN 与 EXPLAIN ANALYZE 的不同：

```
CREATE TABLE student(sno INT PRIMARY KEY, sname TEXT, ssex number);
```

```
 EXPLAIN SELECT * FROM student WHERE sname='ada';
                       QUERY PLAN
-----------------------------------------------------------------------------------------------
 Seq Scan on student  (cost=0.00..20.50 rows=4 width=68)
   Filter: (sname = 'ada'::text)
(2 行记录)
```

```
EXPLAIN ANALYZE SELECT * FROM student WHERE sname='ada';
                                          QUERY PLAN
-----------------------------------------------------------------------------------------------
 Seq Scan on student  (cost=0.00..20.50 rows=4 width=68) (actual time=0.006..0.006 rows=0 loops=1)
   Filter: (sname = 'ada'::text)
 Planning Time: 0.117 ms
 Execution Time: 0.054 ms
(4 行记录)
```

通过使用 EXPLAIN 命令进行查询时，系统仅生成预期的查询执行计划，并未真正执行该语句。而当使用 EXPLAIN ANALYZE 时，系统不仅生成执行计划，还会实际执行查询，并提供详细的性能数据，包括执行成本和时间信息。

提示:

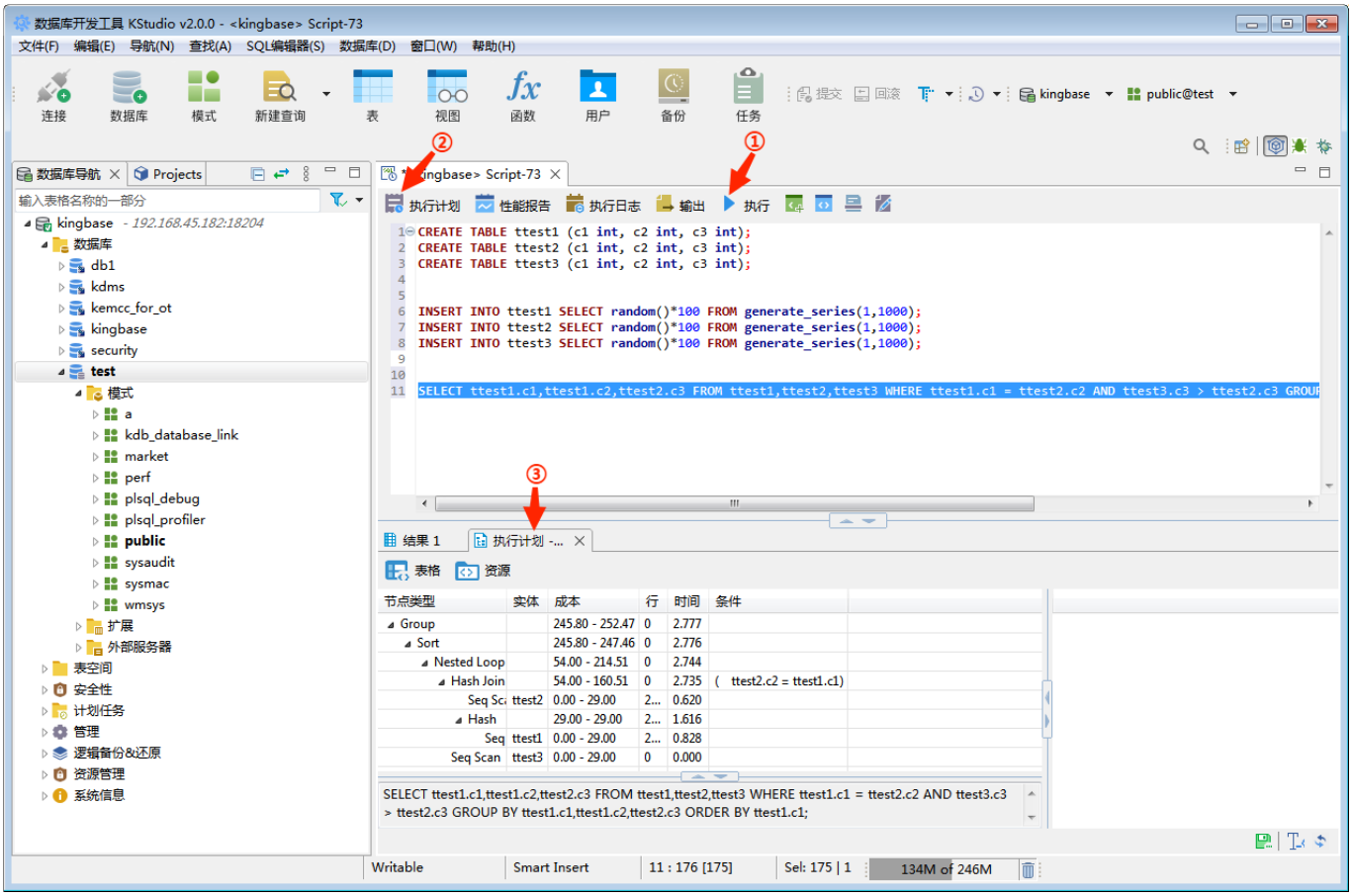
对于某些数据操作语言（DML）语句，若需同时实现运行 EXPLAIN ANALYZE 命令的需求，而不希望对实际数据产生影响，则可将该命令置于事务中执行，并在完成后进行事务回滚：

```
begin;
EXPLAIN ANALYZE update student set ssex=false WHERE sname='ada';
rollback;
```

3.1.2　使用对象管理工具查看

除了文本形式的执行计划查看方式，KingbaseES 还支持通过“对象管理工具”以图形化的方式来查看和展现执行计划。

选定想要查看执行计划的 SQL 语句，然后选择 SQL 编辑器中的解析执行计划，得到语句的完整执行计划，具体情况如下图所示：



3.1.3　在日志记录中查看

KingbaseES 数据库提供了一个名为 AUTO_EXPLAIN 的插件，用于在数据库日志中自动记录运行缓慢 SQL 语句的详细执行计划信息。与 EXPLAIN 和其他对象管理工具相比，AUTO_EXPLAIN 通过持续监控和分析，能够识别出大型应用中存在的未优化 SQL 语句。

根据作用范围不同，AUTO_EXPLAIN 有以下两种方式：

- 作用于当前会话

```
LOAD'AUTO_EXPLAIN';（仅超级用户可用）
```

- 作用于全局

设置 kingbase.conf 中的 shared_preload_libraries 参数为’auto_explain’，可将其预加载到所有会话中。这样做会带来额外的性能开销。

可以通过配置参数，来控制 AUTO_EXPLAIN 的行为：

- AUTO_EXPLAIN.log_min_duration（整数）

最短语句执行时间（ms）；设置为 0：记录所有计划；（默认）设置为 -1：禁用计划记录。

- AUTO_EXPLAIN.log_analyze（boolean）

控制打印 EXPLAIN ANALYZE 输出；

默认情况下，该参数关闭。仅超级用户可更改此设置。

备注:

启用此参数，所有语句都会真正执行，无论其运行时间是否足够长以至于无法真正记录下来，性能成本较高。

- AUTO_EXPLAIN.log_buffers（boolean）

控制打印缓冲区使用情况统计信息，等效于 EXPLAIN 的 BUFFERS 选项。需启用 AUTO_EXPLAIN.log_analyze，默认值为禁用状态，仅超级用户可更改此设置。

- AUTO_EXPLAIN.log_timing（boolean）

控制打印每个节点的计时信息，等效于 EXPLAIN 的 TIMING 选项。需启用 AUTO_EXPLAIN.log_analyze。重复读取系统时钟的开销可能会严重降低某些系统上的查询速度，因此在不需要精确计时时，建议禁用该项功能。

- AUTO_EXPLAIN.log_verbose（boolean）

控制打印详细信息，等效于 EXPLAIN 的 VERBOSE 选项。默认情况下，该参数关闭。仅超级用户可更改此设置。

- AUTO_EXPLAIN.sample_rate（boolean）

控制 AUTO_EXPLAIN 仅对特定会话中的部分查询进行解释。默认值为 1，表示解释所有查询。对于嵌套式查询结构，统会将其整体进行解释或完全跳过。仅超级用户可更改此设置。

3.2　阅读执行计划

本章节包含以下内容：

- 主要内容
- 执行顺序
- 计划节点

3.2.1　主要内容

1. **执行计划里每个节点主要内容包括：**

- 执行动作：包含附加条件，如索引扫描，索引条件等
- 估算成本：启动 cost、总体 cost

- 估计返回的行数和平均宽度
- 如果开启了 Analyze 选项，还会返回：
  - 实际成本：启动 cost、总体 cost
  - 实际返回的行数，节点循环执行次数

我们以下例来说明，执行计划输出结果各部分的内容：

```
EXPLAIN ANALYZE SELECT * FROM student WHERE sname='ada';
QUERY PLAN
----------------------------------------------------------------
Index Scan using idx_stu on student (cost=0.14..8.15 rows=1 width=10)(actual time=0.074..0.074 rows=0
loops=1)
    Index Cond: (sname = 'ada'::text)
Planning Time: 0.122 ms
Execution Time: 0.245 ms
```

2. **执行动作**：Index Scan using idx_stu on student

- Index Scan：表示索引扫描
- Idx_stu：使用的索引名
- Student：访问的表名
- Index Cond: (sname = ‘ada’::text)：索引条件

提示:

整句话表示：访问 student 表的时候，使用的是索引扫描，扫描的索引是 idx_stu，索引条件为：sname = ‘ada’::text。

---

3. **估算成本**：(cost=0.14…8.15 rows=1 width=10)

- Cost=0.14…81.5：启动成本 (返回第一条数据)…总成本 (返回所有数据)
- Rows=1：查询返回行数为 1 条。
- Width=10：返回行的平均宽度为 10 字节。

提示:

在执行计划里，cost 值由下层节点向上层节点传递，cost 值代表的是在此步以下所有操作的代价总和。启动成本是第一行输出开始前的成本，比如进行排序的成本。

---

4. **实际成本**：(actual time=0.074…0.074 rows=0 loops=1)

- Actural time=0.074…0.074：实际的启动/运行时间均为 0.074。

- Rows=0：实际查询返回了 0 行记录。
- Loops=1：索引扫描执行次数为 1 次。

提示:

EXPLAIN ANALYZE 生成信息。估算成本是优化器基于统计信息估算的结果，实际成本是语句执行得出的结果，二者在统计信息不准的情况下，会存在偏差过大的情况。

为了让统计信息更准确，可以运行 Analyze 命令来更新整个数据库、某张表或者某个字段的统计信息，然后再次执行 explain 将得到一个较为准确的结果。

---

5. **计划和执行时间**：(Planning Time: 0.122 ms, Execution Time: 0.245 ms)

EXPLAIN ANALYZE 生成信息。表示生成执行计划用时 0.122ms，真正执行查询用时 0.245ms。

3.2.2　执行顺序

执行计划的结构是一颗计划结点的树，通常会有 0-2 个输入，特殊节点会有多个输入，大部分情况呈现出二叉树的结构。

每一个计划节点对应于树中的一个节点，下层节点的输出作为上层节点输入。数据 (元组) 从底层节点向上层节点流动，直至根节点，而根节点的输出即为整个查询的结果。

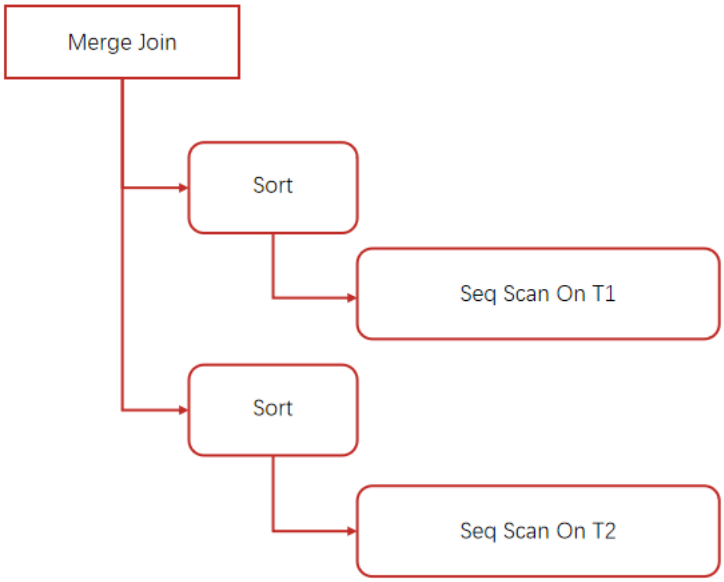
由此可见，查询计划树上的节点就构成了物理元组到执行结果的管道，因此查询计划树的执行过程可以看成是拉动元组穿过管道的过程。KingbaseES 采用了一次一元组的执行模式，每个节点被执行一次仅向上层节点返回一条元组。因此，对于整个查询计划树的执行也是一次一元组的模式。这种模式有很多的优点：

- 减少返回元组的延迟。
- 某些操作 (例如游标、LIMIT 子句等) 无需一次性获取所有的元组，节省了开销。
- 减少了实现过程中缓存结果带来的代码复杂性和执行过程中临时存储的开销。

举一个 join 的例子来说明执行计划里各个部分：

```
EXPLAIN SELECT t1.name, t2.name FROM t1, t2 WHERE t1.id = t2.id;
QUERY PLAN
----------------------------------------------------------------------
Merge Join (cost=176.34..303.67 rows=8064 width=64)
    Merge Cond: (t1.id = t2.id)
    -> Sort (cost=88.17..91.35 rows=1270 width=36)
        Sort Key: t1.id
        -> Seq Scan on t1 (cost=0.00..22.70 rows=1270 width=36)
    -> Sort (cost=88.17..91.35 rows=1270 width=36)
        Sort Key: t2.id
        -> Seq Scan on t2 (cost=0.00..22.70 rows=1270 width=36)
```

该执行计划简化后的模型为一棵树：



3.2.3　计划节点

执行计划内部是树状结构，里面包含的节点主要有 5 类：扫描节点（Scan Node），连接节点（Join Node），物化节点（Materialization Node），控制节点（Control Node），并行节点（Parallel Node），每类节点又包括具体的节点：

1. **扫描节点：**

- 顺序扫描（Seq scan）
- 索引扫描（Index scan）
- 位图扫描（Bitmap scan）
- TID 扫描（TID scan）
- 覆盖索引扫描（Index only scan）

2. **连接节点：**

- 嵌套循环连接（NestLoop join）
- 哈希连接（Hash join）

- 归并连接（Merge join）

3. **物化节点：**

- 物化节点（Material）
- 分组节点（Group）
- 排序节点（Sort）
- 去重节点（Unique）

4. **控制节点：**

- Result 节点
- BitmapAnd/BitmapOr 节点
- Append 节点
- RecursiveUnion 节点

5. **并行节点：**

- Gather/GatherMerge
- 并行全表扫描（Parallel SeqScan）
- 并行 hash（Parallel Hash）
- 并行连接（Parallel HashJoin/MergeJoin）
- 并行聚集（Partial/Finalize Aggregate/HashAggregate/GroupAggregate）
- 并行 Append（Parallel Append）
- 并行 UNION（Parallel UNION）

并行节点详细信息请参考本手册章节使用并行

3.2.3.1　扫描节点

扫描节点包括数据扫描和数据过滤，目前扫描的方式有顺序扫描，索引扫描，位图扫描，TID 扫描，覆盖索引扫描等，存在各自的适用情况。

一般扫描过滤的性能优化建议：

- 提早过滤
- 过滤性越高的字段需要越靠前，过滤性低的靠后
- 核心 SQL 考虑采用覆盖索引 indexonlyscan（更新少），确保高效
- 多 SQL 综合考虑，重复利用索引
- 不干扰过滤前提下，order by 排序字段进入索引

- 根据查询条件，建立索引，优化索引、优化访问方式，限制结果集的数据量。索引应该尽量小，尽量使用字节数小的列建索引，不要对重复值多的列建单一索引。

3.2.3.1.1　顺序扫描（Seq scan）

1. **原理：**

- 根据实际的数据的存取顺序，连续扫描所有数据。

2. **适用情况：**

- 表中数据量较少，如表中仅有 10 条数据；
- 数据选择率较高，如数据选择率高达 80%。

3. **开启/关闭参数**:

```
enable_seqscan=on/off
```

```
EXPLAIN SELECT * FROM student WHERE sname = 'abc';
QUERY PLAN
-------------------------------------------------------
Seq Scan on student (cost=0.00..1.18 rows=1 width=10)
    Filter: (sname = 'abc'::text)
```

3.2.3.1.2　索引扫描（Index scan）

1. **原理：**

使用索引，快速定位符合过滤条件的数据。创建索引存在不同的性质，适用于不同场景，如等值条件使用 hash 索引，可通过键值快速查找到符合条件的索引数据。（具体详见 5.1.1 索引类型）

2. **适用情况：**

数据量较大且选择率较低，如 10000 条数据，选择率仅有 1-20%。

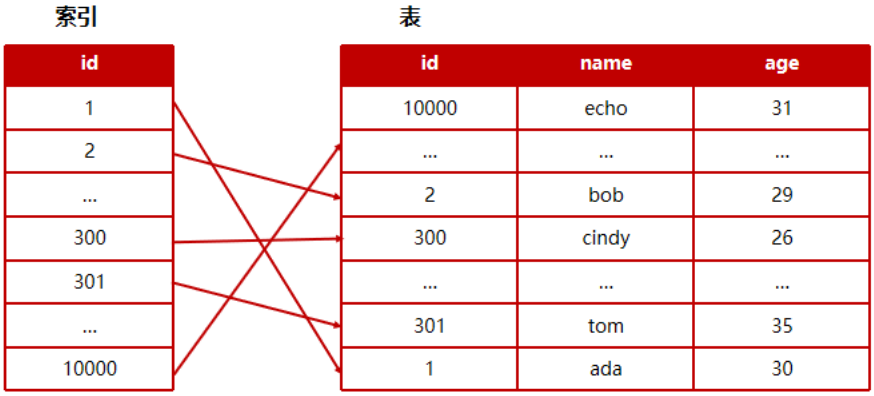
3. **开启/关闭参数**:

```
enable_indexscan=on/off
```

4. **“随机读”问题**:

```
EXPLAIN SELECT * FROM student WHERE sname='ada';
QUERY PLAN
-----------------------------------------------------------------------
Index Scan using idx_stu on student (cost=0.14..8.15 rows=1 width=10)
    Index Cond: (sname = 'ada'::text)
```

使用索引扫描获取表数据，是通过索引的 ctid 映射到堆表。其中索引项“有序”，但存储数据的堆表“无序”。当数据量大，选择率低的情况下，按顺序扫描索引项，再查找堆表，容易导致“随机读”的问题。



3.2.3.1.3　位图扫描（Bitmap scan）

1. **原理：**

- bitmap 扫描是 Bitmap Index Scan 和 Bitmap Heap Scan 的组合。

  先通过 Bitmap Index Scan 索引扫描，在内存中创建一个位图表，每一个 bit 表示一个与过滤条件有关的页面。此页面有可能有数据为 1，不可能为 0。

  然后再通过 Bitmap Heap Scan 表扫描，在内存中创建好的位图表指针对应的页面进行顺序扫描，排除不符合的记录，返回需要的结果。

2. **适用情况：**

适用于列中有重复值，查询中包含 and、or 等范围查找的情况，其中对于有多个条件的组合（AND、OR），可以分别对每个条件做 Bitmap Index Scan，然后再对结果进行 AND 或 OR 操作或当列中的数值散乱在内存中，用于整理数值。

3. **开启/关闭参数:**

```
enable_bitmapscan=on/off
```

3.2.3.1.4　TID 扫描（TID scan）

1. **原理：**

根据数据实际存储位置的 ctid 进行扫描，获取元组。通过隐藏字段 ctid，扫描 ctid 是 KingbaseES 中标记数据位置的字段，通过该字段来查找数据，速度非常快。

2. **适用情况：**

当 where 条件存在 ctid。

3. **开启/关闭参数**:

```
enable_tidscan=on/off
```

3.2.3.1.5　覆盖索引扫描（Index only scan）

1. **原理：**

允许直接从索引得到元组，覆盖索引扫描要求查询中的某个表所需要数据均可以从这个表上的同一个索引的索引页面中获得。

2. **适用情况：**

更新少的表。

3. **开启/关闭参数**:

```
enable_indexonlyscan=on/off
```

备注:

使用覆盖索引之前，需要先对表进行 VACUUM 才能使覆盖索引被优化器使用。

3.2.3.1.6　其他扫描过滤节点

- Sample Scan：

  数据取样，支持查询返回取样数据。当前只在常规表和物化视图上接受 TABLESAMPLE 子句。

- Subquery Scan：

  扫描另一个查询计划树（子计划）的元组，将其转换为子计划的执行。KingbaseES 子查询主要包含如下几个关键字: EXISTS, IN, NOT IN, ANY/SOME, ALL。

- Function Scan:

  扫描返回元组集的函数。在 Scan 的基础上扩展定义了 functions 列表字段，存放 FuncitonScan 涉及的函数，以及 funcordinality 字段 (是否给返回结果加上序号列)。

- Values Scan：

  扫描由值表达式指定的一个行值或者一组行值。更常见的是把它用来生成一个大型命令内的“常量表”，但是它也可以被独自使用。

- CTE Scan：

WITH 提供了一种方式来书写在一个大型查询中使用的辅助语句。这些语句通常被称为公共表表达式或 CTE，它们可以被看成是定义只在一个查询中存在的临时表。在 WITH 子句中的每一个辅助语句可以是一个 SELECT、INSERT、UPDATE 或 DELETE，并且 WITH 子句本身也可以被附加到一个主语句，主语句也可以是 SELECT、INSERT、UPDATE 或 DELETE。

- WorkTable Scan：

  WorkTableScan 与 RecursiveUnion 共同完成递归合并子查询。

- Foreign Scan：

  扫描 KingbaseES 外部数据表，用于 kingbase_fdw 或者 dblink 和外部数据交互的情况。

- Custom Scan：

  自定义扫描接口，用户可以进行自定义扫描方法。

3.2.3.2　连接节点

连接节点主要是根据枚举、动态规划，遗传算法估算连接代价确定连接顺序。

调整方法：

1. ANALYZE 所有查询表;
2. 调整合适的连接顺序，选择率低的 join 先执行
3. 选择合适的连接算法

3.2.3.2.1　嵌套循环连接（NestLoop join）

1. **原理：**

扫描每一条外表的数据（m 条数据），和内表所有的记录（n 条数据）去连接，时间复杂度是 o(mn)。

2. **适用情况：**

数据量不大的情况。

3. **开启/关闭参数**:

```
enable_nestloop=on/off
```

3.2.3.2.2　哈希连接（Hash join）

1. **原理：**

对内表建立 hash 表，扫描所有内表数据到各个 hash 桶里面，建立 hash 桶，再逐行扫描外表数据，对外表数据进行 hash，确定 hash 桶，与桶内数据做连接。

2. **适用情况：**

数据分布无序，重复值较少的情况。

3. **开启/关闭参数**:

```
enable_hashjoin=on/off
```

3.2.3.2.3　归并连接（Merge join）

1. **原理：**

对两个表的数据进行排序，然后做连接。

2. **适用情况：**

两表数据均基本有序。

3. **开启/关闭参数**:

```
enable_mergejoin=on/off
```

3.2.3.3　物化节点

物化节点是一类可缓存元组的节点。在执行过程中，很多扩展的物理操作符需要获取所有的元组后才能进行操作(例如聚集函数操作、没有索引辅助的排序等)，这时要用物化节点将元组缓存起来。

3.2.3.3.1　物化节点（Material）

1. **原理：**

缓存子节点结果，对于重复多次扫描的子节点（特别是多次扫描结果相同）可以减少执行的代价。

2. **适用情况：**

结果在子查询中被多次使用。

3. **开启/关闭参数**:

```
enable_material=on/off
```

3.2.3.3.2　分组节点（Group）

1.HashAggregate

(1) **原理：**

通过 hash 算法，相同的值 hash 到同一桶里面，然后求聚集。

(2) **适用情况：**

适合数据无序的情况。

(3) **开启/关闭参数**:

```
enable_hashagg=on/off
```

2.GroupAggregate

(1) **原理：**

通过排序的方式进行分组，然后执行聚集。

(2) **适用情况：**

适合数据基本有序的情况。

3.Aggregate

(1) **原理：**

执行含有聚集函数的 GROUP BY 操作，该节点能够实现三种执行策略：Plain (不分组的聚集计算)、Sorted(下层节点提供排好序的元组，随后按组执行聚集运算)、Hash (首先对下层节点提供的未排序元组进行分组，再计算聚集值)。

(2) **适用情况：**

含有聚集函数的 GROUP BY 操作。

3.2.3.3.3　排序节点（Sort）

1. **原理：**

对数据进行排序。

2. **适用情况：**

输出结果是有序的的情况。

3. **开启/关闭参数**:

```
enable_sort=on/off
```

3.2.3.3.4　去重节点（Unique）

1. **原理：**

对已排序的元组进行去重操作。UNIQUE 节点负责执行去重操作。由于下层节点能够按顺序提供元组，该节点只需维护一个缓存，用于存储最近接收的元组。对于每个新接收到的元组，系统会比较其与缓存中的元组在特定字段上的值，若存在重复，系统将跳过当前元组，并持续请求后续元组，若无重复，则输出该元组并更新缓存。

2. **适用情况：**

该节点负责执行去重操作，因为下层节点必须返回排序后的元组。当需去重的字段包含在 ORDER BY 子句中时，系统会优先选择使用该 Unique 节点。

3.2.3.3.5　其他物化节点

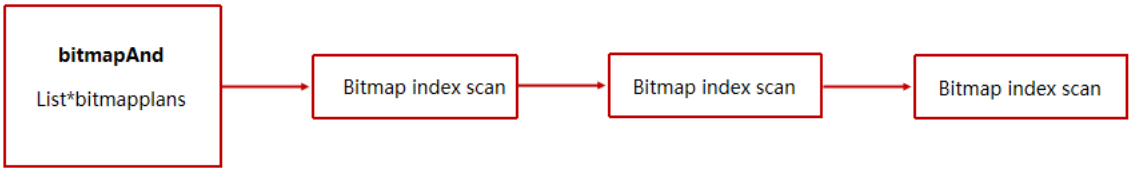
- WindowAgg：窗口函数
- T_SetOp：setop 语法节点
- LockRows：使用锁定子句，比如 FOR UPDATE、FOR SHARE 等
- Limit：当使用 limit 时的节点

3.2.3.4　控制节点

3.2.3.4.1　BitmapAnd/BitmapOr 节点

1. **原理：**

该节点用于处理多个位图的逻辑运算，包括与和或操作。其结构类似于 Append 节点，通过链表组织各子查询计划。首先执行各个子计划以获得位图数据，随后在这些位图上执行逻辑运算。与 Append 节点不同的是，该节点的子计划输出位图而非具体元组。



2. **适用情况：**

BitmapAnd 和 BitmapOr 都是位图 (Bitmap) 类型节点，用于位图计算。

3.2.3.4.2　Result 节点

1. **原理：**

当查询无需扫描表，计算 SELECT 的投影属性或者使用 VALUES 子句构造元组。

2. **适用情况：**

- 针对那些不扫描表的查询
- 用来优化包含仅需计算一次的过滤条件

3.2.3.4.3　Append 节点

1. **原理：**

通过链表结构组织多个子计划，并按顺序依次执行每个子计划。在前一个子计划完成输出后，系统立即启动下一个子计划的执行，直至所有子计划均处理完毕。

2. **适用情况：**

用于处理包含一个或多个子计划的链表。

3.2.3.4.4　RecursiveUnion 节点

1. **原理：**

对子节点进行递归处理。

2. **适用情况：**

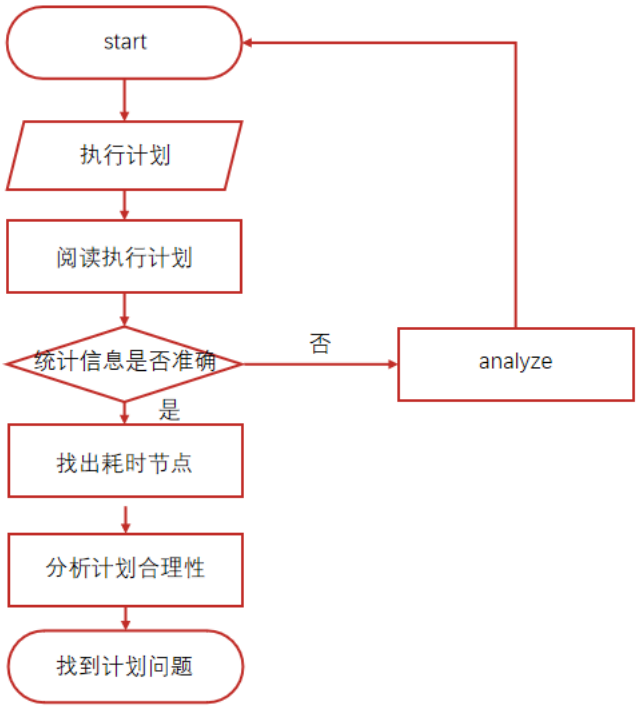
RecursiveUnion 节点用于处理递归定义的 UNION 语句。

3.3　分析执行计划

分析执行计划是 SQL 性能调优需要掌握的一项重要能力，可以帮助用户定位 SQL 语句性能问题产生的原因，进而针对性的实施优化手段。

- 分析步骤
- 执行计划的改变

3.3.1　分析步骤



1. **查看当前慢语句的执行计划**

关于如何查看执行计划详细请参见本手册查看执行计划

2. **自上而下的阅读执行计划，理清层次结构**

执行计划是基于查询语句生成的操作树结构，该操作树中的每个节点代表一个数据库操作，例如表扫描、连接、分组与聚集、排序等。这些操作应按自底向上（从下到上）的顺序依次处理。

3. **对比统计信息是否准确，如果准确执行步骤 4，如果不准确 analyze 相关表后从步骤 1 重新执行。**

4. **关于统计信息详细知识及为什么执行 analyze 参见本手册优化器统计信息**

5. **找出计划中耗时的节点**

识别查询计划中耗时较长的节点有助于快速定位和解决问题。需要注意的是，复杂的查询计划有时可能存在多个需要优化的问题。因此，在进行优化时，应优先关注并解决那些耗时较多的关键节点。

6. **分析耗时节点计划的合理性**

分析耗时节点的合理性可以从以下几个方面进行分析：

- 数据扫描是否可以走索引、分区、物化视图

  对每个表的扫描方式需要进行合理性评估。例如，当查询涉及大量数据时，一般采用顺序扫描；而对于仅需少

量数据的情况，则采用索引扫描方式。对每一步使用的扫描方式进行优化评估至关重要，这种选择通常会对查询性能产生重大影响。

- 多表连接的连接顺序是否合理

  在分析执行计划的层级结构时，需识别表操作的具体顺序并评估其合理性。例如，在涉及三张表（其中两张数据量庞大，另一张数据量较小）的联合查询中，进行表连接时，应优先将数据量大的表与小数据量的表进行连接，然后再将其结果与第三张大表进行连接。这样能够显著减少中间结果并提升查询效率。

- 两表连接的连接算法是否合理

  对于每一步的连接操作，考察采取的连接方式是否合理。比如查看基数估计结果是否准确导致两表连接方式不合理等。

- 返回行数估算是否准确

  比较预期返回行数与实际返回行数，考虑统计信息是否差距过大一致，进而导致选择了次优的执行计划。

- 是否有内存不够的情况

  当存在排序等情况时，操作的表超过了 work_men 时就需要考虑是否存在内存不足的情况。

  随着系统的持续运行，数据规模、数据分布等因素的变化会直接影响执行计划。所以面对相同的语句计划的优劣也不是绝对的。执行计划改变的原因详细请参见本手册执行计划的改变

3.3.2　执行计划的改变

执行计划可能会根据具体情况做出调整，这种调整能够使系统展现出更优的性能表现。特别地，针对那些运行效率较低的语句，通过优化其执行计划，可以有效地缩短其执行所需的时间跨度。值得注意的是，执行计划的变更机制存在多种形式：一方面，系统会自动进行优化调整，例如基于优化器对统计信息收集的结果；另一方面，用户也可以采取强制性修改的方式，这种情况下可以考虑运用 KingbaseES 提供的 HINT 功能来实现预期效果 (详细的使用请参见本手册控制执行计划)，强制修改语句的执行计划，从而达到想要的性能效果。

影响执行计划的几个因素罗列如下：

1. **数据规模的变化**

数据规模对执行计划的选择具有决定性作用。例如，在处理数据量庞大的表查询时，采用全表扫描能够有效缩短时间消耗；而针对数据量相对较少的表查询，则更适合使用索引扫描方式。因此，数据规模的变化会直接影响到系统选择何种执行计划来实现最优性能表现。

2. **数据及索引的膨胀**

在对表内大规模数据执行更新操作时，例如突发性的大量删除或更新记录，如果相关索引未能及时进行回收操作，会导致索引的存储空间仍然维持原有规模不变，从而造成索引页面出现稀疏现象。这种情况会使索引的使用效率降低，并最终引发数据及索引的膨胀问题。

3. **统计信息准确性的影响**

查询优化器在选择执行计划的过程中，是基于所收集的统计信息进行决策的。如果这些统计信息未能准确反映实际数据情况，则可能导致次优执行计划的选择，以及影响执行计划的调整能力，从而对系统性能产生负面影响。

3.4　控制执行计划（使用 HINT）

本章节包含以下内容：

- 概述
- HINT 的使用
- HINT 的功能
- 配置参数
- 示例
- 注意

3.4.1　概述

KingbaseES 采用基于成本的优化器机制。该优化器会根据 SQL 语句生成多种可能的执行路径，并对每种路径进行代价估算，最终选择具有最低执行成本的路径来生成并执行相应的计划。因为优化器本身采用的是代价估算的方式选择最优路径，由于采样精度，统计信息的准确率，以及代价估算算法等因素的限制，有可能存在选择的执行计划不是最优的情况。因此，在这种情况下，需要提供一种手动修正执行计划的可能性，以便更精确地控制和优化查询性能。

HINT 是一种特殊的 SQL 注释形式，用于明确指定在执行 SQL 语句时应采用的具体执行计划。作为一种直接影响执行计划生成的手段，HINT 为用户提供了一个灵活的调整机制，用户可以干预多个关键方面：包括指定连接顺序、选择连接方法以及确定表扫描方式等。此外，还可以利用 HINT 来预估结果行数，从而辅助优化器做出更明智的选择，以提升查询的性能。

3.4.2　HINT 的使用

HINT 通过在目标 SQL 语句 SELECT/UPDATE/DELETE/INSERT/MERGE 之后给出的特殊形式的注释来读取 HINT 注释。注释的形式以字符序列‘/*+’开头以‘*/’结尾，例如/*+SeqScan(tablename)*/，其具体使用方法如下：

- 在 kingbase.conf 配置文件中，配置 `enable_hint = on`，启动数据库。
- 使用 HINT 的注释使 HINT 生效，当 SQL 语句使用的执行计划需要手动调整时，用户可以直接在 SQL 语句中加上 HINT 注释。
- 当用户无法直接在应用层面修改 SQL 语句时，可以结合后续章节介绍的 Query Mapping 功能，在数据库端实现对 SQL 语句的调整。具体方法是通过配置 Query Mapping 规则，建立原始 SQL 语句与添加 HINT 后 SQL 语句之间的对应关系。例如，若希望强制表 t 使用全表扫描，则可以在 Query Mapping 中设置相应的规则。当接收到如 `SELECT * FROM t WHERE id=1;` 时，数据库可以按照 HINT 执行的方式生成执行计划。

```
SELECT create_query_rule('qm1','SELECT * FROM t WHERE id=1',' SELECT/*+seqscan(t)*/* FROM t WHERE id=1',
true,'text');
```

- HINT 可以用于数据库的任何模式下，可以出现在普通的 SQL 语句中，也可用于位于函数中的 SQL 语句。
- 在 HINT 中指定的表名为 SQL 语句中相应表的别名。当需要指定多个表名时，应在 HINT 中依次列出这些表名，并以空格分隔。HINT 可以写在每个子查询中，此时表名可以直接写成该子查询中表的名字；HINT 也可以写在最外层的查询中，此时表名为”子查询名. 表名”的形式，如下面的例子，HINT 和表在同一个子查询时，直接使用名字 T，如果 HINT 控制的是子查询中的表，使用名字 V.T。子查询的名字可以是 SQL 中的名字，用户也可以通过’blockname hint’为子查询定义一个名字。

```
SELECT /*+SeqScan(V.T)*/* FROM (SELECT/*+rows(T #1)*/* FROM T)V;
```

3.4.3　HINT 的功能

HINT 功能可分为以下几种类:

- 指定单表扫描的方式
- 指定单表并行扫描
- 指定多表的连接方式
- 执行多表的连接顺序
- 设置聚合方法
- 设置扫描或连接的基数估计值
- 设置配置参数
- 设置子查询的名字

下面分别介绍各个功能：

- Scan 类型的 HINT：用于指定在对目标表执行单表扫描时所采用的具体扫描方法。这种 HINT 仅适用于普通表、继承表、UNLOGGED 表、临时表以及系统目录等结构。需要注意的是，以下情况不受 SCAN HINT 的影响：外部表、表函数、VALUES 子句、CTE（公共表达式）、视图以及子查询均无法通过 SCAN HINT 进行调整。SCAN HINT 的括号内参数仅允许指定一个表名，并且支持与索引相关的 HINT 格式。若有多余内容需处理，则被视为索引项进行处理，分隔方式可采用逗号或空格形式。

  对于索引扫描，当表为分区表时，目前只能指定 Global Index 的索引名，不支持指定 Local Index 的索引名。

HINT 类型	描述
SeqScan(table)	优先在表上使用顺序扫描。

见续表

表 3.4.1 – 续表

HINT 类型	描述
TidScan(table)	优先在表上使用 Tid 扫描。
IndexScan(table[index…])	优先在表上使用索引扫描，只限制指定的索引。
IndexOnlyScan(table[index…])	优先在表上使用 Index only scan，限制特定的索引。当 Index only scan 不可用时，可以使用索引扫描。
BitmapScan(table[index…])	优先在表上使用位图扫描。
NoSeqScan(table)	优先不在表上使用顺序扫描。
NoTidScan(table)	优先不在表上使用 Tid 扫描。
NoIndexScan(table)	优先不在表上使用索引扫描和 index only scan。
NoIndexScan(table[index…])	优先不在表上使用索引扫描, 优先不使用指定索引。
NoIndexOnlyScan(table)	优先不在表上使用 index only scan。
NoBitmapScan(table)	优先不在表上使用位图索引。
IndexScanRegexp (table[regexp…])	优先在表上使用索引扫描。索引名要满足指定的正则表达式。
BitmapScanRegexp (table[regexp…])	优先在表上使用位图扫描。索引名要满足指定的正则表达式。
IndexO nlyScanRegexp(table[regexp…])	优先在表上使用 Index only scan。索引名要满足指定的正则表达式。
ForceSeqScan(table)	强制在表上使用顺序扫描。
ForceTidScan(table)	强制在表上使用 Tid 扫描。
ForceIndexScan(table[index…])	强制在表上使用索引扫描，只限制指定的索引。
ForceIndexOnlyScan(table[index…])	强制在表上使用 Index only scan，限制特定的索引。当 Index only scan 不可用时，可以使用索引扫描。
ForceBitmapScan(table[index…])	强制在表上使用位图扫描。当指定索引名之后，位图扫描会使用指定的索引进行扫描。

- 并行执行类型的 HINT：在扫描时强制并行执行或禁止并行执行。它可以对普通表，继承父项，UNLOGGED 表和系统目录产生影响。外部表，表函数，值子句，CTE，视图和子查询不受影响。

HINT 类型	描述
Parallel(table_name workers)	强制设置单表扫描并行，并且可以指定并行度
ParallelAppend(workers)	针对 union all 的并行执行，需要在 union all 的外层进行指定，该 hint 指定查询中的 append 节点强制并行执行

- ROWS 类型的 HINT：指明中间结果集的大小，支持绝对值和相对值。该 HINT 可以修正因优化器的限制而连接操作后错估的行数。利用此 HINT 对多个表或复杂结构（如简单表、继承表、分区表、外部表等）的预估行数进行精确调整。

HINT 类型	描述
rows(table_list #|+|-|* const)	设置单表或多个表的返回行数，# 为直接指定行数，+、-、* 为在原来基数估计值的基础上增加一个值，减少一个值或者增长一个倍数。
prows(table_list table #|+|-|* const)	针对参数化路径的 rows，选择使用 prows 来指定，具体用法同 rows

- Join 类型的 HINT：在指定的表进行连接时，强制使用特定的连接方法。它可以影响普通表、继承表、UNLOGGED 表、临时表、外部表、系统目录、表函数、VALUES 命令结果和 CTE 上的连接。但是视图上的连接和子查询不受影响。HINT 中需要包含连接涉及的所有表的别名，如 a,b,c,d 四个表连接，希望 a,b 进行哈希连接，然后和 c 进行嵌套循环连接，则 hint 应写为 hashjoin(a b), nestloop(a b c)，其中 nestloop 要包含参与连接的 3 个表的别名。

HINT 类型	描述
NestLoop(table table[table…])	在对指定的表进行连接时，使用循环嵌套连接。
HashJoin(table table[table…])	在对指定的表进行连接时，使用散列连接。
MergeJoin(table table[table…])	在对指定的表进行连接时，使用排序合并连接。
NoNestLoop(table table[table…])	在对指定的表进行连接时，不使用循环嵌套连接。
NoHashJoin(table table[table…])	在对指定的表进行连接时，不使用散列连接。
NoMergeJoin(table table[table…])	在对指定的表进行连接时，不使用排序合并连接。
materialize(inner_table_list)	多个表进行连接时，强制使用 nestloop 连接且内表为 inner_table_list 指定的一个或多个表，并且对内表进行物化

见续表

表 3.4.4 – 续表

HINT 类型	描述
Use_nl_with_index(inner_table)	多个表进行连接时，强制使用 nestloop 参数化连接，且内表为 inner_table 指定的表，并且内表使用连接条件上的列使用索引扫描

- Leading 类型的 hint：指定在两个或多个表进行连接时的连接顺序，包括内外表顺序。只能指定左深树的连接方式。

  在单个子查询中只能设置一个 leading hint，并且该 hint 必须包含所有参与连接的表，否则无法生效。为了实现对连接顺序和算法的固定，建议先使用 leading hint 明确所有参与连接的表的顺序，然后使用 nestloop/hashjoin/mergejoin hint 指定表的连接算法。在仅指定连接算法而不固定连接顺序的情况下，HINT 通常不会产生预期效果。因此，在调整执行计划时，应优先确保连接顺序的明确性。

HINT 类型	描述
leading(join_table_list)	仅指定 join 顺序，不指定内外表顺序。使用单括号，括号中表名以空格分割，表示首先第一个表和第二个表连接，然后连接第三个表，以此类推。例如 leading(a b c) 仅表示 a 首先和 b 连接，然后连接 c。但 a 和 b 连接顺序为 a join b 还是 b join a 并不指定。
leading((outer_table inner_table))	同时指定连接顺序和内外表顺序。使用双括号，双括号中再使用括号包住连接的两个表，格式为 (((a b) c))
ordered	按照 SQL 中表出现的顺序指定连接的顺序。ordered 比 leading 的优先级更高

- 可以对查询语句块设定用户级别的配置参数，这些参数可以通过 SET 语句进行修改，并影响涉及该子查询所涉表的执行计划生成过程。

  Set(Param_Name Param_Value)

- Blockname 类型的 HINT：允许通过指定块名和表名来控制子查询块中的表。并支持跨查询块的对象引用。对于未命名的子查询块（如子连接），可利用该 HINT 进行命名。

  blockname(subquery_name)

- Aggregate 类型的 HINT：KingbaseES 的聚合运算的算法有两种：排序 (GroupAgggregate) 和哈希 (HashAggregate)。通过 HINT 可以影响优化器选择哪种聚合运算，以及是否选取并行路径。

HINT 类型	描述
Hashagg	子查询中使用哈希聚合。
Groupagg	子查询中使用排序分组聚合。
ParallelHashagg	子查询中使用并行哈希聚合。

-续表-

HINT 类型	描述
ParallelGroupagg	子查询中使用并行排序分组聚合。

3.4.4 配置参数

参数名	描述	默认值
enable_hint	启用 HINT	off
hint_debug_print	是否打印 HINT 的 debug 信息	off
hint_message_level	指定 debug 打印的信息级别。可用的值为 error, warning, notice, info, log, debug1, debug2, debug3, debug4,debug5	log

参数 enable_hint 默认关闭，当此参数关闭后，SQL 查询语句中的 HINT 注释将不会对执行计划产生影响。

当参数 hint_debug_print 设置为 on，并且 hint_message_level 设置为 log 后，运行含 HINT 注释的 SQL 语句会在日志中显示相关调试信息，包含每个表在 HINT 中的名字、used hint、not used hint、duplication hint、error hint。若 HINT 中标注的表名无法识别，则该提示将失效。建议根据日志中的 HINT 表名进行设置。

3.4.5 示例

环境准备：

```
CREATE TABLE t1(id INT, val INT, name varchar(64));
CREATE TABLE t2(id INT, val INT);
CREATE TABLE t3(id INT, val INT);
CREATE TABLE t4(id INT, val INT);
CREATE TABLE t5(id INT, val INT);
CREATE INDEX t1_idx ON t1(id);
CREATE INDEX t2_idx ON t2(id);
CREATE INDEX t3_idx ON t3(id);
CREATE INDEX t4_idx ON t4(id);
INSERT INTO t1 SELECT i, i%5000, 'Kingbase'||(i%5) FROM generate_series(1,3000000) as x(i);
INSERT INTO t2 SELECT i, i%5000 FROM generate_series(1,1000000) as x(i);
INSERT INTO t3 SELECT i, i%5 FROM generate_series(1,100) as x(i);
INSERT INTO t4 SELECT i%5, i%5 FROM generate_series(1,100) as x(i);
```

```
INSERT INTO t5 SELECT i, i%5 FROM generate_series(1,100) as x(i);
ANALYZE t1;
ANALYZE t2;
ANALYZE t3;
CREATE TABLE pt(col1 INT, col2 INT) PARTITION BY RANGE (col1);
CREATE TABLE pt_1 partition of pt for values FROM (0) to (10);
CREATE TABLE pt_2 partition of pt for values FROM (10) to (20);
CREATE INDEX pt_col2_idx ON pt(col2);
CREATE unique index pt_col2_idx_global ON pt(col2) global;
```

为防止并行影响测试结果，以下所有示例结果是在关闭并行扫描的前提下进行

```
set max_parallel_workers_per_gather = 0;
```

3.4.5.1　常用 Scan 类型 HINT 示例

- SeqScan HINT 例子:

```
EXPLAIN SELECT * FROM t1 WHERE id=20;
                        QUERY PLAN
-----------------------------------------------------------------
-> Index Scan using t1_idx on t1 (cost=0.43..8.45 rows=1 width=18)
    Index Cond: (id = 20)
(2 rows)
```

```
EXPLAIN SELECT/*+seqscan(t1)*/ * FROM t1 WHERE id=20;
                        QUERY PLAN
-----------------------------------------------------------------
-> Seq Scan on t1 (cost=0.00..56731.00 rows=1 width=18)
    Filter: (id = 20)
(2 rows)
```

- IndexScan HINT 例子：

```
EXPLAIN SELECT * FROM t1 WHERE id > 10 AND id < 20000000;
                        QUERY PLAN
-----------------------------------------------------------------
-> Seq Scan on t1 (cost=0.00..64231.00 rows=2999990 width=18)
    Filter: ((id > 10) AND (id < 20000000))
(2 rows)
```

```
EXPLAIN SELECT/*+IndexScan(t1)*/ * FROM t1 WHERE id > 10 AND id < 20000000;
                             QUERY PLAN
-------------------------------------------------------------------------
```

```
-> Index Scan using t1_idx on t1 (cost=0.43..112326.23 rows=2999990 width=18
    Index Cond: ((id > 10) AND (id < 20000000))
(2 rows)
```

```
EXPLAIN SELECT * FROM pt WHERE col2 < 10;
                                    QUERY PLAN
-------------------------------------------------------------------------------
 Append  (cost=9.99..66.34 rows=1506 width=8)
   ->  Bitmap Heap Scan on pt_1  (cost=9.99..29.40 rows=753 width=8)
         Recheck Cond: (col2 < 10)
         ->  Bitmap Index Scan on pt_1_col2_idx  (cost=0.00..9.80 rows=753 width=0)
           Index Cond: (col2 < 10)
   ->  Bitmap Heap Scan on pt_2  (cost=9.99..29.40 rows=753 width=8)
         Recheck Cond: (col2 < 10)
         ->  Bitmap Index Scan on pt_2_col2_idx  (cost=0.00..9.80 rows=753 width=0)
           Index Cond: (col2 < 10)
```

```
EXPLAIN SELECT/*+indexscan(pt pt_col2_idx_global)*/ * FROM pt WHERE col2 < 10;
                                     QUERY PLAN
-----------------------------------------------------------------------------------
 Global Index Scan using pt_col2_idx_global on pt  (cost=0.12..13.15 rows=1506 width=8)
   Index Cond: (col2 < 10)
```

- IndexOnlyScan HINT 例子：

```
EXPLAIN SELECT COUNT(0) FROM t1;
                           QUERY PLAN
-------------------------------------------------------------------------------
Aggregate (cost=56731.00..56731.01 rows=1 width=8)
    -> Seq Scan on t1 (cost=0.00..49231.00 rows=3000000 width=0)
```

```
EXPLAIN SELECT/*+IndexOnlyScan(t1)*/ COUNT(0) FROM t1;
                           QUERY PLAN
-------------------------------------------------------------------------------
Aggregate (cost=104826.43..104826.44 rows=1 width=8)
    -> Index Only Scan using t1_idx on t1 (cost=0.43..97326.43 rows=3000000 width=0)
```

- Bitmapscan HINT 例子：

```
EXPLAIN SELECT * FROM t1 WHERE id > 30;
                    QUERY PLAN
-----------------------------------------------------------
-> Seq Scan on t1 (cost=0.00..56731.00 rows=2999970 width=18)
    Filter: (id > 30)
```

```
EXPLAIN SELECT/*+bitmapscan(t1)*/ * FROM t1 WHERE id > 30;
                        QUERY PLAN
----------------------------------------------------------------------
Bitmap Heap Scan on t1 (cost=56342.20..113072.82 rows=2999970 width=18)
    Recheck Cond: (id > 30)
    Bitmap Index Scan on t1_idx (cost=0.00..55592.20 rows=2999970 width=0)
        Index Cond: (id > 30)
```

3.4.5.2　Parallel 类型 HINT 示例

使用并行例子，需要将并行打开 set max_parallel_workers_per_gather = 2;

```
EXPLAIN ANALYZE SELECT/*+Parallel(t2 2)*/ t2.id FROM t2,t3 WHERE t2.id=t3.val GROUP BY t2.id;
                                                        QUERY PLAN
----------------------------------------------------------------------------------------------------------
---------------------------------
Group (cost=5.35..7.15 rows=100 width=4) (actual time=22.341..24.182 rows=4 loops=1)
    Group Key: t2.id
    -> Merge Join (cost=5.35..6.90 rows=100 width=4) (actual time=22.338..24.142 rows=80 loops=1)
        Merge Cond: (t2.id = t3.val)
        -> Gather Merge (cost=0.02..10424.84 rows=1000000 width=4)(actual time=22.100..23.800 rows=5
loops=1)
            Workers Planned: 2
            Workers Launched: 2
            -> Parallel Index Only Scan using t2_idx on t2 (cost=0.00..0.00 rows=416667 width=4) (actual
time=0.051..0.328 rows=421 loops=3)
                Heap Fetches: 1262
        -> Sort (cost=5.32..5.57 rows=100 width=4) (actual time=0.210..0.250 rows=100 loops=1)
            Sort Key: t3.val
            Sort Method: quicksort Memory: 30kB
            -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=4) (actual time=0.042..0.110 rows=100 loops=1)
Planning Time: 0.857 ms
Execution Time: 24.359 ms
```

```
EXPLAIN SELECT * FROM (SELECT * FROM t2 UNION ALL SELECT * FROM t3) a;
                            QUERY PLAN
-----------------------------------------------------------------
 Append  (cost=0.00..19427.50 rows=1000100 width=8)
   ->  Seq Scan on t2  (cost=0.00..14425.00 rows=1000000 width=8)
   ->  Seq Scan on t3  (cost=0.00..2.00 rows=100 width=8)
```

```
EXPLAIN SELECT/*+ParallelAppend(2)*/ * FROM (SELECT * FROM t2 UNION ALL SELECT * FROM t3) a;
                                    QUERY PLAN
```

```
-----------------------------------------------------------------------------
 Gather  (cost=0.00..10676.80 rows=1000100 width=8)
   Workers Planned: 2
   ->  Parallel Append  (cost=0.00..10676.80 rows=416709 width=8)
         ->  Parallel Seq Scan on t2  (cost=0.00..8591.67 rows=416667 width=8)
         ->  Parallel Seq Scan on t3  (cost=0.00..1.59 rows=59 width=8)
```

该用例测试完成后，将并行关闭 set max_parallel_workers_per_gather = 0;

3.4.5.3　ROWS 类型 HINT 示例

测试用例中 t2 表的基数估计值为 1589，而实际执行行数是 5，差距较大。

```
EXPLAIN ANALYZE SELECT t2.id FROM t2,t3 WHERE t2.id=t3.val AND t3.id<5 AND t2.val<8;
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------
-----------
 Merge Join  (cost=2.70..23.47 rows=1 width=4) (actual time=0.036..0.041 rows=4 loops=1)
   Merge Cond: (t2.id = t3.val)
   ->  Index Scan using t2_idx on t2  (cost=0.42..32968.43 rows=1589 width=4) (actual time=0.012..0.014
rows=5 loops=1)
         Filter: (val < 8)
   ->  Sort  (cost=2.27..2.28 rows=3 width=4) (actual time=0.021..0.022 rows=4 loops=1)
         Sort Key: t3.val
         Sort Method: quicksort  Memory: 25kB
         ->  Seq Scan on t3  (cost=0.00..2.25 rows=3 width=4) (actual time=0.007..0.017 rows=4 loops=1)
               Filter: (id < 5)
               Rows Removed by Filter: 96
 Planning Time: 0.174 ms
 Execution Time: 0.069 ms
```

使用 ROWS HINT 修正基数估计，优化器选择了最优的连接方式 NestLoop，而不是修正前的 MergeJoin。

```
EXPLAIN ANALYZE SELECT/*+rows(t2 #5)*/ t2.id FROM t2,t3 WHERE t2.id=t3.val AND t3.id<5 AND t2.val<8;
                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------------
-----
 Nested Loop  (cost=0.42..27.61 rows=1 width=4) (actual time=0.062..0.134 rows=4 loops=1)
   ->  Seq Scan on t3  (cost=0.00..2.25 rows=3 width=4) (actual time=0.034..0.050 rows=4 loops=1)
         Filter: (id < 5)
         Rows Removed by Filter: 96
   ->  Index Scan using t2_idx on t2  (cost=0.42..8.45 rows=1 width=4) (actual time=0.019..0.019 rows=1
loops=4)
         Index Cond: (id = t3.val)
         Filter: (val < 8)
```

```
 Planning Time: 0.255 ms
 Execution Time: 0.188 ms
```

设置用例强制采用 nestloop 的连接方式，t4 表的参数化路径 rows，估算值为 20，实际值为 1，差别较大

```
EXPLAIN ANALYZE  SELECT/*+nestloop(t4 t5)*/ * FROM t5,t4 WHERE t5.id = t4.id AND t5.val < 5;
                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------
--------
 Nested Loop  (cost=0.14..83.50 rows=100 width=16) (actual time=0.031..0.286 rows=80 loops=1)
   ->  Seq Scan on t5  (cost=0.00..2.25 rows=100 width=8) (actual time=0.012..0.049 rows=100 loops=1)
         Filter: (val < 5)
   ->  Index Scan using t4_idx on t4  (cost=0.14..0.61 rows=20 width=8) (actual time=0.001..0.002 rows=1
loops=100)
         Index Cond: (id = t5.id)
 Planning Time: 0.169 ms
 Execution Time: 0.336 ms
```

使用 PROWS HINT 修正基数估计

```
EXPLAIN ANALYZE  SELECT/*+nestloop(t4 t5) PRows(t5 t4 #1)*/ * FROM t5,t4 WHERE t5.id = t4.id AND t5.val < 5;
                        QUERY PLAN
-----------------------------------------------------------------------
 Nested Loop  (cost=0.14..31.25 rows=100 width=16) (actual time=0.102..0.272 rows=80 loops=1)
   ->  Seq Scan on t5  (cost=0.00..2.25 rows=100 width=8) (actual time=0.051..0.068 rows=100 loops=1)
         Filter: (val < 5)
   ->  Index Scan using t4_idx on t4  (cost=0.14..0.28 rows=1 width=8) (actual time=0.001..0.001 rows=1
loops=100)
         Index Cond: (id = t5.id)
 Planning Time: 0.166 ms
 Execution Time: 0.340 ms
```

3.4.5.4　常用 Join 类型 HINT 示例

本章节使用下面的 SQL，演示三种常用 Join 类型的 HINT：

```
SELECT t2.id FROM t2,t3 WHERE t2.id=t3.val AND t2.id>5 GROUP BY t2.id;
```

原始计划使用 MergeJoin 连接：

```
EXPLAIN SELECT t2.id FROM t2,t3 WHERE t2.id=t3.val AND t2.id>5 GROUP BY t2.id;
                            QUERY PLAN
-------------------------------------------------------------------------------------
Group (cost=5.75..7.64 rows=100 width=4)
```

```
    Group Key: t2.id
    -> Merge Join (cost=5.75..7.39 rows=100 width=4)
        Merge Cond: (t2.id = t3.val
        -> Index Only Scan using t2_idx on t2 (cost=0.42..32968.32 rows=999994 width=4)
            Index Cond: (id > 5)
        -> Sort (cost=5.32..5.57 rows=100 width=4)
            Sort Key: t3.val
            -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=4)
```

通过 NestLoop 连接 HINT 进行更改：

```
EXPLAIN SELECT/*+Nestloop(t2 t3)*/ t2.id FROM t2,t3 WHERE t2.id=t3.val AND t2.id>5 GROUP BY t2.id;
                                QUERY PLAN
-----------------------------------------------------------------------------------------------
Group (cost=842.82..843.32 rows=100 width=4)
    Group Key: t2.id
    -> Sort (cost=842.82..843.07 rows=100 width=4)
        Sort Key: t2.id
        -> Nested Loop (cost=0.42..839.50 rows=100 width=4)
            -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=4)
            -> Index Only Scan using t2_idx on t2 (cost=0.42..8.37 rows=1 width=4)
                Index Cond: ((id = t3.val) AND (id > 5))
```

通过 HashJoin 连接 HINT 进行更改：

```
EXPLAIN SELECT/*+Hashjoin(t2 t3)*/ t2.id FROM t2,t3 WHERE t2.id=t3.val AND t2.id>5 GROUP BY t2.id;
                                QUERY PLAN
-------------------------------------------------------------------------------------------
Group (cost=29431.62..29432.12 rows=100 width=4)
    Group Key: t2.id
    -> Sort (cost=29431.62..29431.87 rows=100 width=4)
        Sort Key: t2.id
        Hash Join (cost=29424.93..29428.30 rows=100 width=4)
            Hash Cond: (t3.val = t2.id)
            -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=4)
            -> Hash (cost=16925.00..16925.00 rows=999994 width=4)
                -> Seq Scan on t2 (cost=0.00..16925.00 rows=999994 width=4)
                    Filter: (id > 5)
```

下面例子，执行计划中两表使用 Hashjoin，使用 MergeJoin 连接 HINT 进行更改：

```
EXPLAIN SELECT t5.id FROM t3,t5 WHERE t3.id=t5.val GROUP BY t5.id;
                              QUERY PLAN
-----------------------------------------------------------------------
 HashAggregate  (cost=6.88..7.88 rows=100 width=4)
   Group Key: t5.id
```

```
  ->  Hash Join  (cost=3.25..6.62 rows=100 width=4)
        Hash Cond: (t5.val = t3.id)
        ->  Seq Scan on t5  (cost=0.00..2.00 rows=100 width=8)
        ->  Hash  (cost=2.00..2.00 rows=100 width=4)
              ->  Seq Scan on t3  (cost=0.00..2.00 rows=100 width=4)
```

```
EXPLAIN SELECT /*+Mergejoin(t3 t5)*/ t5.id FROM t3,t5 WHERE t3.id=t5.val GROUP BY t5.id;
                                      QUERY PLAN
-------------------------------------------------------------------------------
 HashAggregate  (cost=7.63..8.63 rows=100 width=4)
   Group Key: t5.id
   ->  Merge Join  (cost=5.56..7.38 rows=100 width=4)
         Merge Cond: (t3.id = t5.val)
         ->  Index Only Scan using t3_idx on t3  (cost=0.14..13.64 rows=100 width=4)
         ->  Sort  (cost=5.32..5.57 rows=100 width=8)
               Sort Key: t5.val
               ->  Seq Scan on t5  (cost=0.00..2.00 rows=100 width=8)
```

3.4.5.5　Leading 连接顺序 HINT 示例

Leading 类型的 HINT 种类 1：仅指定 join 顺序，不指定内外表顺序：

下面例子中各表的连接顺序为（t3 t1 t2），使用 leading HINT 更改连接顺序，但不指定内外表。

```
EXPLAIN SELECT t2.id FROM t1,t2,t3 WHERE t1.id=t3.id AND t1.id<3 AND t3.val=t2.id GROUP BY t2.id;
                        QUERY PLAN
-----------------------------------------------------------------
Group (cost=10.72..10.87 rows=1 width=4)
    Group Key: t2.id
    -> Merge Join (cost=10.72..10.87 rows=1 width=4)
        Merge Cond: (t2.id = t3.val)
        -> Index Only Scan using t2_idx on t2 (cost=0.42..30468.42 rows=1000000 width=4)
        -> Sort (cost=10.29..10.30 rows=1 width=4)
            Sort Key: t3.val
            -> Merge Join (cost=5.92..10.28 rows=1 width=4)
                Merge Cond: (t1.id = t3.id)
                -> Index Only Scan using t1_idx on t1 (cost=0.43..8.46 rows=2 width=4)
                    Index Cond: (id < 3)
                -> Sort (cost=5.32..5.57 rows=100 width=8)
                    Sort Key: t3.id
                    -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=8)
```

```
EXPLAIN SELECT/*+leading(t3 t2 t1)*/ t2.id FROM t1,t2,t3 WHERE t1.id=t3.id AND t1.id<3 AND t3.val=t2.id
GROUP BY t2.id;
```

```
                        QUERY PLAN
-------------------------------------------------------------------
Group (cost=15.67..15.68 rows=1 width=4)
    Group Key: t2.id
    -> Sort (cost=15.67..15.68 rows=1 width=4)
        Sort Key: t2.id
        -> Merge Join (cost=11.30..15.66 rows=1 width=4)
            Merge Cond: (t3.id = t1.id)
            -> Sort (cost=10.70..10.95 rows=100 width=8)
                Sort Key: t3.id
                -> Merge Join (cost=5.75..7.38 rows=100 width=8)
                    Merge Cond: (t2.id = t3.val)
                    -> Index Only Scan using t2_idx on t2 (cost=0.42..30468.42 rows=1000000 width=4)
                    -> Sort (cost=5.32..5.57 rows=100 width=8)
                        Sort Key: t3.val
                        -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=8)
            -> Index Only Scan using t1_idx on t1 (cost=0.43..8.46 rows=2 width=4)
                Index Cond: (id < 3)
```

Leading 类型的 HINT 种类 2：同时指定 join 顺序和内外表顺序：

对上面的例子指定 t1 为外表，t3 为内表进行连接，两表连接的结果作为外表与 t2 作为内表进行连接：

```
EXPLAIN SELECT/*+leading(((t1 t3) t2))*/ t2.id FROM t1,t2,t3 WHERE t1.id=t3.id AND t1.id<3 AND t3.val=t2.id
GROUP BY t2.id;
                                          QUERY PLAN
-----------------------------------------------------------------------------------------------
 Group  (cost=18.66..18.67 rows=1 width=4)
   Group Key: t2.id
   ->  Sort  (cost=18.66..18.67 rows=1 width=4)
         Sort Key: t2.id
         ->  Nested Loop  (cost=6.18..18.65 rows=1 width=4)
               ->  Merge Join  (cost=5.76..10.28 rows=1 width=4)
                     Merge Cond: (t1.id = t3.id)
                     ->  Index Only Scan using t1_idx on t1  (cost=0.43..8.46 rows=2 width=4)
                           Index Cond: (id < 3)
                     ->  Sort  (cost=5.32..5.57 rows=100 width=8)
                           Sort Key: t3.id
                           ->  Seq Scan on t3  (cost=0.00..2.00 rows=100 width=8)
               ->  Index Only Scan using t2_idx on t2  (cost=0.42..8.36 rows=1 width=4)
                     Index Cond: (id = t3.val)
```

3.4.5.6　SET 类型 HINT 示例

下面的 SQL 语句执行计划使用 MergeJoin，通过 SET HINT，设置 enable_mergejoin=off，使用 Nestloop。

```
EXPLAIN SELECT t2.id, COUNT(t3.val) FROM t2,t3 WHERE t2.id=t3.val GROUP BY t2.id;
                        QUERY PLAN
-------------------------------------------------------------------
GroupAggregate (cost=5.75..8.88 rows=100 width=12)
    Group Key: t2.id
    -> Merge Join (cost=5.75..7.38 rows=100 width=8)
        Merge Cond: (t2.id = t3.val)
        -> Index Only Scan using t2_idx on t2 (cost=0.42..30468.42 rows=1000000 width=4)
        -> Sort (cost=5.32..5.57 rows=100 width=4)
            Sort Key: t3.val
            -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=4)
```

```
EXPLAIN SELECT/*+set(enable_mergejoin off)*/ t2.id, COUNT(t3.val) FROM t2,t3 WHERE t2.id=t3.val GROUP BY t2.
id;
                        QUERY PLAN
-------------------------------------------------------------------
GroupAggregate (cost=842.57..844.32 rows=100 width=12)
    Group Key: t2.id
    -> Sort (cost=842.57..842.82 rows=100 width=8)
        Sort Key: t2.id
        -> Nested Loop (cost=0.42..839.25 rows=100 width=8)
            -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=4)
            -> Index Only Scan using t2_idx on t2 (cost=0.42..8.36 rows=1 width=4)
                Index Cond: (id = t3.val)
```

3.4.5.7　Blockname 类型 HINT 示例

在下面的例子中，父查询和子查询都是用表 t2，通过名字无法区分两个表，可以使用 blockname 进行区分，更改如下：

```
EXPLAIN SELECT t2.id FROM t2 WHERE t2.id IN (SELECT id FROM t2 WHERE t2.val < 9);
                        QUERY PLAN
-------------------------------------------------------------------
Nested Loop (cost=16929.55..28344.71 rows=1651 width=4)
    -> HashAggregate (cost=16929.13..16945.64 rows=1651 width=4)
        Group Key: t2_1.id
        -> Seq Scan on t2 t2_1 (cost=0.00..16925.00 rows=1651 width=4)
            Filter: (val < 9)
        -> Index Only Scan using t2_idx on t2 (cost=0.42..6.89 rows=1 width=4)
            Index Cond: (id = t2_1.id)
```

```
EXPLAIN SELECT/*+HashJoin(t2 blk.t2)*/ t2.id FROM t2 WHERE t2.id IN (SELECT/*+blockname(blk)*/ id FROM t2
WHERE t2.val < 9);
```

```
                        QUERY PLAN
----------------------------------------------------------------
Hash Semi Join (cost=16945.64..34014.00 rows=1651 width=4)
    Hash Cond: (t2.id = t2_1.id)
    -> Seq Scan on t2 (cost=0.00..14425.00 rows=1000000 width=4)
    -> Hash (cost=16925.00..16925.00 rows=1651 width=4)
        -> Seq Scan on t2 t2_1 (cost=0.00..16925.00 rows=1651 width=4)
            Filter: (val < 9)
```

3.4.5.8　Materialize 类型 HINT 示例

下面的例子中将（t2 t3）连接的中间结果进行物化处理：

```
EXPLAIN SELECT t2.id FROM t1,t2,t3 WHERE t1.id=t3.id AND t1.id<3  AND t3.val=t2.id GROUP BY t2.id;
QUERY PLAN
----------------------------------------------------------------
Group (cost=10.72..10.87 rows=1 width=4)
    Group Key: t2.id
    -> Merge Join (cost=10.72..10.87 rows=1 width=4)
        Merge Cond: (t2.id = t3.val)
        -> Index Only Scan using t2_idx on t2 (cost=0.42..30468.42 rows=1000000 width=4)
        -> Sort (cost=10.29..10.30 rows=1 width=4)
            Sort Key: t3.val
            -> Merge Join (cost=5.92..10.28 rows=1 width=4)
                Merge Cond: (t1.id = t3.id)
                -> Index Only Scan using t1_idx on t1 (cost=0.43..8.46 rows=2 width=4)
                    Index Cond: (id < 3)
                -> Sort (cost=5.32..5.57 rows=100 width=8)
                    Sort Key: t3.id
                    -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=8)
```

```
EXPLAIN SELECT/*+materialize(t2 t3)*/t2.id FROM t1,t2,t3 WHERE t1.id=t3.id AND t1.id<3 AND t3.val=t2.id
GROUP BY t2.id;
QUERY PLAN
----------------------------------------------------------------
Group (cost=19.10..19.11 rows=1 width=4)
    Group Key: t2.id
    -> Sort (cost=19.10..19.11 rows=1 width=4)
        Sort Key: t2.id
        -> Nested Loop (cost=6.18..19.09 rows=1 width=4)
            Join Filter: (t3.id = t1.id)
            -> Index Only Scan using t1_idx on t1 (cost=0.43..8.46 rows=2 width=4)
                Index Cond: (id < 3)
            -> Materialize (cost=5.75..7.88 rows=100 width=8)
```

```
                -> Merge Join (cost=5.75..7.38 rows=100 width=8)
                    Merge Cond: (t2.id = t3.val)
                    -> Index Only Scan using t2_idx on t2 (cost=0.42..30468.42 rows=1000000 width=4)
                    -> Sort (cost=5.32..5.57 rows=100 width=8)
                        Sort Key: t3.val
                        -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=8)
```

3.4.5.9　Use_nl_with_index 类型 HINT 示例

下面的 SQL 语句中，对表 t2 使用参数化路径：

```
EXPLAIN SELECT t2.id, COUNT(t3.val) FROM t2,t3 WHERE t2.id=t3.val GROUP BY t2.id;
QUERY PLAN
----------------------------------------------------------------
GroupAggregate (cost=5.75..8.88 rows=100 width=12)
    Group Key: t2.id
    -> Merge Join (cost=5.75..7.38 rows=100 width=8)
        Merge Cond: (t2.id = t3.val)
        -> Index Only Scan using t2_idx on t2 (cost=0.42..30468.42 rows=1000000 width=4)
        -> Sort (cost=5.32..5.57 rows=100 width=4)
            Sort Key: t3.val
            -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=4)
```

```
EXPLAIN SELECT/*+use_nl_with_index(t2)*/t2.id, COUNT(t3.val) FROM t2,t3 WHERE t2.id=t3.val GROUP BY t2.id;
QUERY PLAN
----------------------------------------------------------------
GroupAggregate (cost=842.57..844.32 rows=100 width=12)
    Group Key: t2.id
    -> Sort (cost=842.57..842.82 rows=100 width=8)
        Sort Key: t2.id
        -> Nested Loop (cost=0.42..839.25 rows=100 width=8)
            -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=4)
            -> Index Only Scan using t2_idx on t2 (cost=0.42..8.36 rows=1 width=4)
                Index Cond: (id = t3.val)
```

3.4.5.10　Aggregate 类型 HINT 示例

使用 hashagg HINT：

测试并行例子，需要将并行打开 `set max_parallel_workers_per_gather to 2;`

```
EXPLAIN SELECT t2.id, COUNT(t3.val) FROM t2,t3 WHERE t2.id=t3.val GROUP BY t2.id;
QUERY PLAN
```

```
----------------------------------------------------------------
GroupAggregate (cost=5.75..8.88 rows=100 width=12)
    Group Key: t2.id
    -> Merge Join (cost=5.75..7.38 rows=100 width=8)
        Merge Cond: (t2.id = t3.val)
        -> Index Only Scan using t2_idx on t2 (cost=0.42..30468.42 rows=1000000 width=4)
        -> Sort (cost=5.32..5.57 rows=100 width=4)
            Sort Key: t3.val
            -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=4)
```

```
EXPLAIN SELECT/*+hashagg*/t2.id, COUNT(t3.val) FROM t2,t3 WHERE t2.id=t3.val GROUP BY t2.id;
QUERY PLAN
----------------------------------------------------------------
HashAggregate (cost=7.88..8.88 rows=100 width=12)
    Group Key: t2.id
    -> Merge Join (cost=5.75..7.38 rows=100 width=8)
        Merge Cond: (t2.id = t3.val)
        -> Index Only Scan using t2_idx on t2 (cost=0.42..30468.42 rows=1000000 width=4)
        -> Sort (cost=5.32..5.57 rows=100 width=4)
            Sort Key: t3.val
            -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=4)
```

```
EXPLAIN SELECT/*+parallelhashagg*/t2.id, COUNT(t3.val) FROM t2,t3 WHERE t2.id=t3.val GROUP BY t2.id;
                                                    QUERY PLAN
----------------------------------------------------------------------------------------------------
----
 Finalize HashAggregate  (cost=1017.56..1018.56 rows=100 width=12)
   Group Key: t2.id
   ->  Gather Merge  (cost=1005.77..1017.14 rows=84 width=12)
         Workers Planned: 2
         ->  Partial GroupAggregate  (cost=5.75..7.42 rows=42 width=12)
               Group Key: t2.id
               ->  Merge Join  (cost=5.75..6.79 rows=42 width=8)
                     Merge Cond: (t2.id = t3.val)
                     ->  Parallel Index Only Scan using t2_idx on t2  (cost=0.42..24635.09 rows=416667
width=4)
                     ->  Sort  (cost=5.32..5.57 rows=100 width=4)
                           Sort Key: t3.val
                           ->  Seq Scan on t3  (cost=0.00..2.00 rows=100 width=4)
```

该用例测试完成后，将并行关闭 `set max_parallel_workers_per_gather = 0;`

3.4.5.11　Ordered 类型 HINT 示例

下面的例子中表的连接顺序是 t1，t3，t2，加入 ordered HINT 后，执行计划按照表出现的顺序进行连接：

```
EXPLAIN SELECT t2.id FROM t2,t3,t1 WHERE t1.id=t3.id AND t1.id<3 AND t3.val=t2.id GROUP BY t2.id;
QUERY PLAN
----------------------------------------------------------------
Group (cost=10.72..10.87 rows=1 width=4)
    Group Key: t2.id
    -> Merge  Join (cost=10.72..10.87 rows=1 width=4)
        Merge Cond: (t2.id = t3.val)
        -> Index Only Scan using t2_idx on t2 (cost=0.42..30468.42 rows=1000000 width=4)
        -> Sort (cost=10.29..10.30 rows=1 width=4)
            Sort Key: t3.val
            -> Merge Join (cost=5.92..10.28 rows=1 width=4)
                Merge Cond: (t3.id = t1.id)
                -> Sort (cost=5.32..5.57 rows=100 width=8)
                    Sort Key: t3.id
                    -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=8)
                -> Index Only Scan using t1_idx on t1 (cost=0.43..8.46 rows=2 width=4)
                    Index Cond: (id < 3)
```

```
EXPLAIN SELECT/*+ordered*/t2.id FROM t2,t3,t1 WHERE t1.id=t3.id AND t1.id<3 AND t3.val=t2.id GROUP BY t2.id;
QUERY PLAN
----------------------------------------------------------------
Group (cost=15.67..15.68 rows=1 width=4)
    Group Key: t2.id
    -> Sort (cost=15.67..15.68 rows=1 width=4)
        Sort Key: t2.id
        -> Merge Join (cost=11.30..15.66 rows=1 width=4)
            Merge Cond: (t3.id = t1.id)
            -> Sort (cost=10.70..10.95 rows=100 width=8)
                Sort Key: t3.id
                -> Merge Join (cost=5.75..7.38 rows=100 width=8)
                    Merge Cond: (t2.id = t3.val)
                    -> Index Only Scan using t2_idx on t2 (cost=0.42..30468.42 rows=1000000 width=4)
                    -> Sort (cost=5.32..5.57 rows=100 width=8)
                        Sort Key: t3.val
                        -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=8)
            -> Index Only Scan using t1_idx on t1 (cost=0.43..8.46 rows=2 width=4)
                Index Cond: (id < 3)
```

3.4.5.12　HINT 混合使用示例

通常 SQL 调优中会使用多种 HINT，如下例子中使用了连接顺序 leading HINT、连接方式 NestLoop HINT 以及聚集 HINT hashagg。

```
EXPLAIN SELECT/*+leading(((t2 t3) t1)) Nestloop(t2 t3) hashagg*/ t2.id FROM t1,t2,t3 WHERE t1.id=t3.id AND
t1.id<3 AND t3.val=t2.id GROUP BY t2.id;
QUERY PLAN
----------------------------------------------------------------
HashAggregate (cost=1514438.72..1514438.73 rows=1 width=4)
    Group Key: t2.id
    -> Nested Loop (cost=0.43..1514438.72 rows=1 width=4)
        Join Filter: (t3.id = t1.id)
        -> Nested Loop (cost=0.00..1514427.25 rows=100 width=8)
            Join Filter: (t2.id = t3.val)
            -> Seq Scan on t2 (cost=0.00..14425.00 rows=1000000 width=4)
            Materialize (cost=0.00..2.50 rows=100 width=8)
                -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=8)
        Materialize (cost=0.43..8.47 rows=2 width=4)
            -> Index Only Scan using t1_idx on t1 (cost=0.43..8.46 rows=2 width=4)
                Index Cond: (id < 3)
```

3.4.6　注意

Set 类型 HINT 设置并行的 GUC 参数 `max_parallel_workers_per_gather`，若 `max_parallel_worker < max_parallel_workers_per_gather`，系统日志会记录该事件，并行 HINT 不会生效。

Set 类型 HINT 只支持用户级可变的优化器相关的参数从而控制执行计划，其他类型参数（例如执行器参数 enable_upper_colname, work_mem,…）不支持。

第4章　手动 SQL 优化示例

4.1　概述

KingbaseES 的查询优化器虽然在不断优化自身能力，但仍存在无法完全覆盖所有场景的情况。针对优化器未能自动处理的情形，本章将详细介绍通过手动方式进行 SQL 性能调优的具体方法，涵盖以下主要内容：

- 使用索引
- 更新统计信息
- 增大 work_mem
- 使用分区
- 使用物化视图
- 缓存执行计划
- 调整性能参数
- 使用并行
- 改写 SQL

4.2　使用索引

4.2.1　概述

索引是一种有序存储结构，是实现高效数据检索的关键技术手段之一，在 SQL 优化中具有重要作用。通过建立适当的索引，可以显著提高数据查询的速度和效率。目前 KES 支持的索引类型有：

索引类型	适用范围
Btree	KES 默认索引；支持的查询操作包括 >,<,>=,<=,=,IN,LIKE 等；适用于 ORDER BY、MIN、MAX、MERGE JOIN 等有序操作
Hash	仅适用于等值查询。在数据列存在较高重复值的情况下，不建议使用
Bitmap	适用于需要处理多属性组合查询的场景，例如表包含多个字段且查询条件涉及这些字段的任意组合时
GIN	支持复杂的数据结构查询，如数组和全文检索等操作
GiST	适用于多维数据类型和集合数据类型；可用于位置搜索，如包含、相交、左边、右边等
SP-GiST	适用于空间可以递归分割成不相交区域的结构，支持四叉树、k-D 树和基数树等嵌套式查询结构
BRIN	适用于按顺序插入或追加的流数据，如时间序列日志数据

提示:

不合适建索引的场景：

- 索引列存在函数（确定性的函数可以创建函数索引）
- 索引列选择率较低，此时全表扫描更高效
- 数据量较小的表，维护索引的成本可能超过其收益
- 大字段类型（如大文本或大二进制），索引占用太多的存储空间

通过查询系统表 sys_stat_user_indexes 可以识别从未被使用的索引。针对这些长期未被利用的索引，可以执行删除操作以减少磁盘空间占用并降低维护成本。同时，可借助数据库提供的空间管理工具 VACUUM 删除已被标记为无效的数据，释放占用的空间，并根据需要手动重建必要的索引以支持查询性能。

4.2.2　索引使用问题示例

环境准备：

```
CREATE TABLE t1(id INT, val INT, name VARCHAR(64));
INSERT INTO t1 SELECT i%10, i%10, 'Kingbase'||(i%5) FROM generate_series(1,1000) AS x(i);
CREATE TABLE t2(id INT, val INT);
INSERT INTO t2 SELECT i%15, i%5 FROM generate_series(1,1000) AS x(i);
CREATE TABLE t3(id INT, val INT);
INSERT INTO t3 SELECT i%2000, i%200 FROM generate_series(1,50000) AS x(i);
CREATE TABLE t4(id INT, val INT);
INSERT INTO t4 SELECT i, i%2000 FROM generate_series(1,10000) AS x(i);
```

```
CREATE TABLE t5(id INT, first_name TEXT, middle_name VARCHAR(20), last_name TEXT COLLATE "C");
CREATE view v1(id, total) AS (SELECT t1.id, sum(t1.val) FROM t1,t2 WHERE t1.id=t2.val group by t1.id);
ANALYZE;

set max_parallel_workers_per_gather = 0; -- 禁用并行
```

4.2.2.1　缺少索引

- 示例 1：未创建适当的索引，仅执行全表扫描，执行速度慢

```
EXPLAIN ANALYZE SELECT * FROM t1 WHERE id < 1;
                        QUERY PLAN
-----------------------------------------------------------------------------------------------
Seq Scan on t1  (cost=0.00..19.50 rows=100 width=18) (actual time=0.012..0.118 rows=100 loops=1)
   Filter: (id < 1)
   Rows Removed by Filter: 900
 Planning Time: 0.073 ms
 Execution Time: 0.133 ms
(5 行记录)
```

当前查询条件下的选择率为 1%。由于表 t1 未创建索引，查询只能执行全表扫描。

根据谓词过滤条件，在 id 列创建索引以提升查询效率：

```
CREATE INDEX ON t1(id); -- 创建索引
ANALYZE t1;
```

此时查询速度变快：

```
EXPLAIN ANALYZE SELECT * FROM t1 WHERE id < 1;
                                                        QUERY PLAN
-------------------------------------------------------------------------------------------------------------
----------
 Bitmap Heap Scan on t1  (cost=5.05..13.30 rows=100 width=18) (actual time=0.039..0.075 rows=100 loops=1)
   Recheck Cond: (id < 1)
   Heap Blocks: exact=7
   ->  Bitmap Index Scan on t1_id_idx  (cost=0.00..5.03 rows=100 width=0) (actual time=0.032..0.032 rows=100
loops=1)
         Index Cond: (id < 1)
 Planning Time: 0.211 ms
 Execution Time: 0.096 ms
(7 行记录)
```

- 示例 2：未创建适当的索引导致连接方式不准确，导致无法生成最优执行计划

```
EXPLAIN ANALYZE SELECT * FROM t1, t2, v1 WHERE t1.id=t2.id AND t2.val=v1.id AND t1.id < v1.total AND t2.id <
10;
                        QUERY PLAN
-------------------------------------------------------------------------------------------------------------
--------------------------
Hash Join (cost=1700.69..2363.39 rows=14919 width=38) (actual time=71.315..96.564 rows=53600 loops=1)
    Hash Cond: (t1.id = t2.id)
    Join Filter: (t1.id < (sum(t1_1.val)))
    Rows Removed by Join Filter: 13300
    -> Seq Scan on t1 (cost=0.00..17.00 rows=1000 width=18) (actual time=0.039..0.200 rows=1000 loops=1)
    -> Hash (cost=1692.33..1692.33 rows=669 width=20) (actual time=71.262..71.264 rows=669 loops=1)
        Buckets: 1024 Batches: 1 Memory Usage: 45kB
        -> Hash Join (cost=1672.32..1692.33 rows=669 width=20) (actual time=70.526..71.044 rows=669 loops=1)
            Hash Cond: (t2.val = t1_1.id)
            -> Seq Scan on t2 (cost=0.00..17.50 rows=669 width=8) (actual time=0.040..0.254 rows=669
loops=1)
                Filter: (id < 10)
                Rows Removed by Filter: 331
            -> Hash (cost=1672.20..1672.20 rows=10 width=12) (actual time=70.478..70.479 rows=5 loops=1)
                Buckets: 1024 Batches: 1 Memory Usage: 9kB
                -> HashAggregate (cost=1672.00..1672.10 rows=10 width=12) (actual time=70.473..70.476 rows=5
loops=1)
                    Group Key: t1_1.id
                    -> Hash Join (cost=29.50..1172.00 rows=100000 width=8) (actual time=1.694..31.701
rows=100000 loops=1)
                        Hash Cond: (t2_1.val = t1_1.id)
                        -> Seq Scan on t2 t2_1 (cost=0.00..15.00 rows=1000 width=4)(actual time=0.024..0.210
rows=1000 loops=1)
                        -> Hash (cost=17.00..17.00 rows=1000 width=8) (actual time=1.655..1.656 rows=1000
loops=1)
                            Buckets: 1024 Batches: 1 Memory Usage: 48kB
                            -> Seq Scan on t1 t1_1 (cost=0.00..17.00 rows=1000 width=8)(actual time=0.021..
0.786 rows=1000 loops=1)
Planning Time: 0.938 ms
Execution Time: 98.882 ms
(24 rows)
```

rows 很小的情况下，不建议使用 HashJoin 进行连接操作，此时 NestLoop 或 MergeJoin 更合适。现对相关列添加索引：

```
CREATE INDEX t1_id ON t1(id);
```

重新执行：

```
EXPLAIN ANALYZE SELECT * FROM t1, t2, v1 WHERE t1.id=t2.id AND t2.val=v1.id AND t1.id < v1.total AND t2.id <
10;
                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------
----------------------------------
 Merge Join  (cost=2058.44..2322.85 rows=14919 width=38) (actual time=61.944..85.296 rows=53600 loops=1)
   Merge Cond: ((t2.id = t1.id) AND (t2.val = t1_1.id))
   ->  Sort  (cost=48.90..50.57 rows=669 width=8) (actual time=0.388..0.446 rows=669 loops=1)
         Sort Key: t2.id, t2.val
         Sort Method: quicksort  Memory: 61kB
         ->  Seq Scan on t2  (cost=0.00..17.50 rows=669 width=8) (actual time=0.022..0.230 rows=669 loops=1)
               Filter: (id < 10)
               Rows Removed by Filter: 331
   ->  Sort  (cost=2009.55..2017.88 rows=3333 width=30) (actual time=61.430..65.319 rows=56734 loops=1)
         Sort Key: t1.id, t1_1.id
         Sort Method: quicksort  Memory: 440kB
         ->  Nested Loop  (cost=1672.28..1814.52 rows=3333 width=30) (actual time=56.375..58.865 rows=4000
loops=1)
               ->  HashAggregate  (cost=1672.00..1672.10 rows=10 width=12) (actual time=56.341..56.347
rows=5 loops=1)
                     Group Key: t1_1.id
                     ->  Hash Join  (cost=29.50..1172.00 rows=100000 width=8) (actual time=0.511..25.216
rows=100000 loops=1)
                           Hash Cond: (t2_1.val = t1_1.id)
                           ->  Seq Scan on t2 t2_1  (cost=0.00..15.00 rows=1000 width=4) (actual time=0.010.
.0.157 rows=1000 loops=1)
                           ->  Hash  (cost=17.00..17.00 rows=1000 width=8) (actual time=0.491..0.492
rows=1000 loops=1)
                                 Buckets: 1024  Batches: 1  Memory Usage: 48kB
                                 ->  Seq Scan on t1 t1_1  (cost=0.00..17.00 rows=1000 width=8) (actual
time=0.011..0.239 rows=1000 loops=1)
               ->  Index Scan using t1_id on t1  (cost=0.28..10.90 rows=333 width=18) (actual time=0.017..0.
321 rows=800 loops=5)
                     Index Cond: (id < (sum(t1_1.val)))
 Planning Time: 0.750 ms
 Execution Time: 87.328 ms
(24 行记录)
```

从执行计划中可以看出，选择 NestLoop 和 MergeJoin 的连接方式查询性能得到提升。

4.2.2.2　使用表达式索引

在表 t1 的 name 列创建普通索引：

```
CREATE INDEX idx_t1_name ON t1(name text_pattern_ops);
ANALYZE t1;

EXPLAIN ANALYZE SELECT * FROM t1 WHERE UPPER(name) = 'KINGBASE1';
                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------
-----------------
 Seq Scan on t1  (cost=0.00..27.00 rows=5 width=18) (actual time=0.013..0.432 rows=200 loops=1)
   Filter: ((((upper((name)::TEXT))::character varying(8000 char))::character varying(64 char))::TEXT =
'KINGBASE1'::TEXT)
   Rows Removed by Filter: 800
 Planning Time: 0.215 ms
 Execution Time: 0.451 ms
(5 行记录)
```

从执行计划中可以看出，针对表 t1 采用了全表扫描的方式，未能有效利用 name 字段上的普通索引。这是因为查询中使用了 upper(name) 函数进行了大小写无关的比较操作，导致数据库无法利用现有的普通索引来加速查询。为了解决这一问题，应在 name 字段上创建合适的函数索引：

```
CREATE INDEX idx_t1_name_expr ON t1(UPPER(name)); -- 创建函数索引
ANALYZE t1;
```

重新执行：

```
EXPLAIN ANALYZE SELECT * FROM t1 WHERE UPPER(name) = 'KINGBASE1';
                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------
-------------------------
 Bitmap Heap Scan on t1  (cost=9.83..20.83 rows=200 width=18) (actual time=0.040..0.073 rows=200 loops=1)
   Recheck Cond: ((((upper((name)::TEXT))::character varying(8000 char))::character varying(64 char))::TEXT
= 'KINGBASE1'::TEXT)
   Heap Blocks: exact=7
   ->  Bitmap Index Scan on idx_t1_name_expr  (cost=0.00..9.78 rows=200 width=0) (actual time=0.032..0.032
rows=200 loops=1)
         Index Cond: ((((upper((name)::TEXT))::character varying(8000 char))::character varying(64 char))::
TEXT = 'KINGBASE1'::TEXT)
 Planning Time: 0.250 ms
 Execution Time: 0.099 ms
(7 行记录)
```

对比上述两个执行计划可以看出，选择合适的函数索引能够提升执行速度，优化查询性能。

4.2.2.3　使用联合索引

当 WHERE 子句中引用了联合索引中的所有列或前导列时，可以利用该联合索引来提高数据检索效率。

```
EXPLAIN ANALYZE SELECT * FROM t3,t4 WHERE t3.id=t4.id AND t4.val = 50 AND t3.val < 150;

                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------
--------
 Hash Join  (cost=170.06..1157.74 rows=19 width=16) (actual time=1.140..10.731 rows=25 loops=1)
   Hash Cond: (t3.id = t4.id)
   ->  Seq Scan on t3  (cost=0.00..847.00 rows=37462 width=8) (actual time=0.010..6.434 rows=37500 loops=1)
         Filter: (val < 150)
         Rows Removed by Filter: 12500
   ->  Hash  (cost=170.00..170.00 rows=5 width=8) (actual time=1.115..1.115 rows=5 loops=1)
         Buckets: 1024  Batches: 1  Memory Usage: 9kB
         ->  Seq Scan on t4  (cost=0.00..170.00 rows=5 width=8) (actual time=0.010..1.112 rows=5 loops=1)
               Filter: (val = 50)
               Rows Removed by Filter: 9995
 Planning Time: 0.153 ms
 Execution Time: 10.752 ms
(12 行记录)
```

对表 t3 创建 id 和 val 两列的联合索引：

```
CREATE INDEX t3_id_val_idx ON t3(id,val);
```

再次执行：

```
EXPLAIN ANALYZE SELECT * FROM t3,t4 WHERE t3.id=t4.id AND t4.val = 50 AND t3.val < 150;
                                                          QUERY PLAN
-----------------------------------------------------------------------------------------------------------
-----------------
 Nested Loop  (cost=4.48..450.37 rows=19 width=16) (actual time=0.032..1.226 rows=25 loops=1)
   ->  Seq Scan on t4  (cost=0.00..170.00 rows=5 width=8) (actual time=0.014..1.160 rows=5 loops=1)
         Filter: (val = 50)
         Rows Removed by Filter: 9995
   ->  Bitmap Heap Scan on t3  (cost=4.48..55.88 rows=19 width=8) (actual time=0.005..0.011 rows=5 loops=5)
         Recheck Cond: ((id = t4.id) AND (val < 150))
         Heap Blocks: exact=25
         ->  Bitmap Index Scan on t3_id_val_idx  (cost=0.00..4.48 rows=19 width=0) (actual time=0.003..0.003
rows=5 loops=5)
               Index Cond: ((id = t4.id) AND (val < 150))
 Planning Time: 0.303 ms
 Execution Time: 1.250 ms
(11 行记录)
```

4.2.2.4　使用索引提升 Like 模式匹配性能

4.2.2.4.1　前匹配或者精确匹配使用 Btree 索引

当使用 Like 表达式进行通配符模式匹配时，BTree 索引能够有效提高查询效率。然而，在处理 Like 模糊查询时，BTree 索引仅适用于前缀匹配（`a LIKE'abc%'`）或精确匹配（`a LIKE'abc'`），而不支持后缀匹配（`a LIKE'%abc'`）和中间匹配（`a LIKE'%abc%'`）。这一限制源于 BTree 索引的操作特性，即仅限于处理诸如大于、等于和小于等的操作，因此在某些模糊查询场景下表现有限。

1. 精确匹配型 like

创建索引:

```
CREATE INDEX t5_first_name_idx ON t5(first_name);
EXPLAIN SELECT * FROM t5 WHERE first_name LIKE 'abc';
                                    QUERY PLAN
-------------------------------------------------------------------------------
 Bitmap Heap Scan on t5  (cost=4.17..11.28 rows=3 width=126)
   Filter: (first_name ~~ 'abc'::TEXT)
   ->  Bitmap Index Scan on t5_first_name_idx  (cost=0.00..4.17 rows=3 width=0)
         Index Cond: (first_name = 'abc'::TEXT)
(4 行记录)
```

在精确匹配下，优化器能够将 `name LIKE 'abc'` 转换为 `name = 'abc'`的形式，高效地利用索引条件进行查询。

2. 索引字段为“C”collate

```
CREATE INDEX ON t5(last_name);
EXPLAIN SELECT * FROM t5 WHERE last_name LIKE 'abc%';
                                    QUERY PLAN
--------------------------------------------------------------------------------
 Bitmap Heap Scan on t5  (cost=4.18..11.29 rows=3 width=126)
   Filter: (last_name ~~ 'abc%'::TEXT)
   ->  Bitmap Index Scan on t5_last_name_idx  (cost=0.00..4.18 rows=3 width=0)
         Index Cond: ((last_name >= 'abc'::TEXT) AND (last_name < 'abd'::TEXT))
(4 行记录)
```

对于配置为 Collate ”C” 的字段，在执行 LIKE 操作时，同样会被转换为`>= AND <` 的一对索引条件。

在其他情况下，则需要使用特定的操作符类（如 text_pattern_ops、bpchar_pattern_ops 等）指定相应的 BTree 索引以支持模式匹配查询：

3. LIKE const 表达式

```
CREATE INDEX ON t5(first_name text_pattern_ops);
EXPLAIN SELECT * FROM t5 WHERE first_name LIKE 'abc%';
                                       QUERY PLAN
-----------------------------------------------------------------------------------
```

```
 Bitmap Heap Scan on t5  (cost=4.18..11.29 rows=3 width=126)
   Filter: (first_name ~~ 'abc%'::TEXT)
   ->  Bitmap Index Scan on t5_first_name_idx1  (cost=0.00..4.18 rows=3 width=0)
         Index Cond: ((first_name ~>=~ 'abc'::TEXT) AND (first_name ~<~ 'abd'::TEXT))
(4 行记录)
```

4. LIKE var 表达式

KingbaseES 数据库系统支持将模式匹配条件指定为一个变量表达式的场景，例如 a LIKE b 的查询，并通过 BTree 索引来实现高效的查询处理。

```
EXPLAIN SELECT count(*) FROM t5,t1 WHERE t5.first_name LIKE t1.name || '%';
                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------
---------------------------------------------
 Aggregate  (cost=1707.50..1707.51 rows=1 width=8)
   ->  Nested Loop  (cost=0.16..1701.00 rows=2600 width=0)
         ->  Seq Scan on t1  (cost=0.00..17.00 rows=1000 width=10)
         ->  Index Only Scan using t5_first_name_idx1 on t5  (cost=0.16..1.65 rows=3 width=32)
               Index Cond: ((first_name ~>=~ low_prefix_text(((t1.name)::TEXT || '%'::TEXT), '100'::oid))
AND (first_name ~<~ high_prefix_text(((t1.name):
:TEXT || '%'::TEXT), '100'::oid, '2095'::oid, '100'::oid)))
               Filter: (first_name ~~ ((t1.name)::TEXT || '%'::TEXT))
(6 行记录)
```

4.2.2.4.2　后匹配 reverse() 表达式

对于仅需执行后缀匹配查询的情况（例如字符串 LIKE ‘%abc’），可以利用带有 COLLATE “C”设置的 reverse() 函数及其 BTree 索引来实现高效的匹配操作。通过将原查询条件转换为 reverse(字符串) LIKE ‘cba%’ 的形式，将其视为前缀匹配问题进行处理。

```
EXPLAIN SELECT * FROM t5 WHERE reverse(t5.first_name) LIKE '2%';
                     QUERY PLAN
-----------------------------------------------------
 Seq Scan on t5  (cost=0.00..17.80 rows=3 width=126)
   Filter: (reverse(first_name) ~~ '2%'::TEXT)
(2 行记录)
```

```
CREATE INDEX ON t5(reverse(first_name) COLLATE "C"); -- 创建索引
EXPLAIN SELECT * FROM t5 WHERE reverse(t5.first_name) LIKE '2%';
                                          QUERY PLAN
-------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on t5  (cost=4.18..11.30 rows=3 width=126)
   Filter: (reverse(first_name) ~~ '2%'::TEXT)
   ->  Bitmap Index Scan on t5_reverse_idx  (cost=0.00..4.18 rows=3 width=0)
```

```
        Index Cond: ((reverse(first_name) >= '2'::TEXT) AND (reverse(first_name) < '3'::TEXT))
(4 行记录)
```

4.2.2.4.3　中间匹配使用 TRGM 索引

sys_trgm 是一个扩展模块，主要用于提供基于三元模型的字母数字文本相似度计算函数和操作符，并支持快速搜索相似字符串的索引操作符类。该模块适用于以下场景：

- 适用范围：- 提供 GiST 和 GIN 索引操作符类。- 支持基于三元模型的索引搜索，包括 LIKE，ILIKE，~ 和 ~* 查询。
- 局限性：- 在处理部分匹配和全匹配的 LIKE 查询时，trgm 索引的性能可能不如 BTree 索引。- trgm 索引不支持等值比较以及简单的比较操作。

sys_trgm 支持 like 模糊匹配示例如下：

```
CREATE extension sys_trgm; -- 开启扩展 sys_trgm
DROP INDEX t5_first_name_idx1;
CREATE INDEX ON t5 using gin(first_name gin_trgm_ops); -- 创建 gin 索引
EXPLAIN SELECT count(*) FROM t5,t1 WHERE t5.first_name LIKE t1.name || '%';
                                        QUERY PLAN
-----------------------------------------------------------------------------------------------
 Aggregate  (cost=4161.25..4161.26 rows=1 width=8)
   ->  Nested Loop  (cost=0.06..4154.75 rows=2600 width=0)
         ->  Seq Scan on t1  (cost=0.00..17.00 rows=1000 width=10)
         ->  Bitmap Heap Scan on t5  (cost=0.06..4.11 rows=3 width=32)
               Recheck Cond: (first_name ~~ ((t1.name)::TEXT || '%'::TEXT))
               ->  Bitmap Index Scan on t5_first_name_idx2  (cost=0.00..0.06 rows=3 width=0)
                     Index Cond: (first_name ~~ ((t1.name)::TEXT || '%'::TEXT))
(7 行记录)
```

4.3　更新统计信息

本章节使用实例构造方法为：

```
CREATE TABLE tt1 (a INT,b VARCHAR,c CHAR(50000), d CHAR(50000)); -- 建表 tt1
INSERT INTO tt1(a,b,c,d)
SELECT MOD(i,100),MOD(i,50),i, i FROM generate_series(1,5000) s(i);
CREATE TABLE tt2 (a INT,b VARCHAR,c INT); -- 建表 tt2
INSERT INTO tt2(a,b,c)
SELECT MOD(i,50),MOD(i,25),i FROM generate_series(1,5000) s(i);
CREATE TABLE tt3 (a INT,b VARCHAR,c INT); -- 建表 tt3
INSERT INTO tt3(a,b,c)
```

```
SELECT MOD(i,50),MOD(i,25),i FROM generate_series(1,5000) s(i);
CREATE INDEX ind ON tt3 USING btree(a);
```

4.3.1　统计信息未及时更新

创建两表后，直接执行 SQL 语句：

```
EXPLAIN ANALYZE SELECT * FROM tt3 WHERE a > 10;
                          QUERY PLAN
-----------------------------------------------------------------------------------------------------
---------
Bitmap Heap Scan on tt3  (cost=37.20..86.04 rows=1667 width=40) (actual time=0.559..1.352 rows=3900 loops=1)

   Recheck Cond: (a > 10)
   Heap Blocks: exact=28
   ->  Bitmap Index Scan on ind  (cost=0.00..36.78 rows=1667 width=0) (actual time=0.536..0.537 rows=3900
loops=1)
         Index Cond: (a > 10)
Planning Time: 0.186 ms
Execution Time: 1.743 ms
(7 rows)
```

根据查询执行计划的结果分析，在查询条件设定为 a > 10 时，预估将返回的记录数为 1667 条，然而实际符合条件的记录数量却达到了 3900 条。这一偏差是由于统计信息没有及时收集，导致基数估计出现误差，进而选择了位图扫描。

```
ANALYZE; -- 更新统计信息
-- 更新统计信息后再次执行 SQL 语句:
EXPLAIN ANALYZE SELECT * FROM tt3 WHERE a > 10;
                             QUERY PLAN
-------------------------------------------------------------------------------------------------------
 Seq Scan on tt3  (cost=0.00..90.50 rows=3900 width=10) (actual time=0.018..1.191 rows=3900 loops=1)
   Filter: (a > 10)
   Rows Removed by Filter: 1100
 Planning Time: 0.149 ms
 Execution Time: 1.365 ms
 (5 rows)
```

在执行 ANALYZE 以更新统计信息之后，查询优化器能够准确预估将返回 3900 条记录，并选择顺序扫描作以实现最优性能。

4.3.2　基数估计不准

一些情况下由于优化器的局限性会导致返回的行数不准。特别是在处理多列统计或多条件查询等复杂情形时，优化器的预估值可能出现偏差。下面举例说明：

执行 SQL 语句：

```
EXPLAIN ANALYZE SELECT * FROM tt1,tt2 WHERE tt1.b=tt2.a AND tt1.a = 1 AND tt1.b='1' AND tt2.b ='1';
                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------
Nested Loop  (cost=0.00..1002.50 rows=1 width=1192) (actual time=0.030..93.655 rows=5000 loops=1)
   Join Filter: ((tt1.b)::integer = tt2.a)
   Rows Removed by Join Filter: 5000
   ->  Seq Scan on tt1  (cost=0.00..909.00 rows=1 width=1182) (actual time=0.013..2.405 rows=50 loops=1)
         Filter: ((a = 1) AND ((b)::text = '1'::text))
         Rows Removed by Filter: 4950
   ->  Seq Scan on tt2  (cost=0.00..90.50 rows=200 width=10) (actual time=0.008..1.714 rows=200 loops=50)
         Filter: ((b)::text = '1'::text)
         Rows Removed by Filter: 4800
Planning Time: 0.281 ms
Execution Time: 93.953 ms
```

从执行计划中可以看出，针对基表 tt1 的基数估计存在偏差，实际结果为 50 行，而优化器仅估算了 1 行。为了解决这一问题，考虑在 tt1 表的 a 和 b 两列上创建多列 MCV 扩展统计信息：

```
CREATE STATISTICS tt1_stat(mcv) ON a,b FROM tt1;
ANALYZE tt1;
```

重新执行

```
EXPLAIN ANALYZE SELECT * FROM tt1,tt2 WHERE tt1.b=tt2.a AND tt1.a = 1 AND tt1.b='1' AND tt2.b ='1';
                                          QUERY PLAN
-----------------------------------------------------------------------------------------------------
--
Hash Join  (cost=909.62..1003.62 rows=50 width=1192) (actual time=2.296..7.302 rows=5000 loops=1)
Hash Cond: (tt2.a = (tt1.b)::integer)
->  Seq Scan on tt2  (cost=0.00..90.50 rows=200 width=10) (actual time=0.016..1.603 rows=200 loops=1)
      Filter: ((b)::text = '1'::text)
      Rows Removed by Filter: 4800
->  Hash  (cost=909.00..909.00 rows=50 width=1182) (actual time=2.268..2.269 rows=50 loops=1)
      Buckets: 1024  Batches: 1  Memory Usage: 68kB
      ->  Seq Scan on tt1  (cost=0.00..909.00 rows=50 width=1182) (actual time=0.004..2.218 rows=50 loops=1)
            Filter: ((a = 1) AND ((b)::text = '1'::text))
            Rows Removed by Filter: 4950
Planning Time: 0.326 ms
Execution Time: 7.553 ms
```

在基表 tt1 的基数估计得到校正后，查询执行计划变更为使用 HashJoin 而非 NestedLoop 连接。这一调整使查询响应时间从 94 毫秒优化至 8 毫秒。

4.4　增大 work_mem

当查询包含排序操作时，若排序过程所需的内存超出系统配置的 work_mem 参数设定值，数据库将利用临时磁盘文件来辅助完成排序。继续使用上一章节实例进行举例说明：

```
EXPLAIN ANALYZE SELECT * FROM tt1 ORDER BY a;
                       QUERY PLAN
-----------------------------------------------------------------------------------------------------
---
Sort  (cost=3774.19..3786.69 rows=5000 width=1182) (actual time=10.293..12.644 rows=5000 loops=1)
   Sort Key: a
   Sort Method: external merge  Disk: 5872kB
   ->  Seq Scan on tt1  (cost=0.00..884.00 rows=5000 width=1182) (actual time=0.012..1.769 rows=5000
loops=1)
Planning Time: 0.064 ms
Execution Time: 13.935 ms
```

根据查询执行计划显示，排序操作采用了外部归并排序机制 (external merge)，并生成了 5872KB 的磁盘临时文件。这表明系统的工作内存资源不足以支持该排序操作。现将 work_mem 参数值调高以优化排序性能：

```
set work_mem= 10240;
```

再次执行：

```
EXPLAIN ANALYZE SELECT * FROM tt1 ORDER BY a;
                         QUERY PLAN
-----------------------------------------------------------------------------------------------------
--
Sort  (cost=1191.19..1203.69 rows=5000 width=1182) (actual time=4.229..5.015 rows=5000 loops=1)
   Sort Key: a
   Sort Method: quicksort  Memory: 10235kB
   ->  Seq Scan on tt1  (cost=0.00..884.00 rows=5000 width=1182) (actual time=0.012..1.796 rows=5000
loops=1)
Planning Time: 0.068 ms
Execution Time: 6.474 ms
```

根据查询执行计划显示，在调整相关配置参数后，排序操作不再需要使用磁盘临时文件进行处理。同时，SQL 语句的执行时间也随之下降。

4.5　使用分区

数据库通过分区功能，可将大规模数据表及其索引划分为多个较小且便于管理的子分区。随着子分区数量的增加，在对分区表执行 SELECT、UPDATE 或 DELETE 操作时，其执行计划生成所需的时间会显著增加。因此，本节将重点介绍为解决此类性能瓶颈而设计的配置参数及其相关设置方法。

- 参数名称：partition_table_limit
  - 默认值：20
  - 取值范围：0~2,147,483,647
- 配置方法：
  - 全局设置：配置参数中指定（需执行 sys_reload_conf() 重载配置文件生效），例：

```
ALTER SYSTEM SET partition_table_limit = 3;
SELECT sys_reload_conf();
```

  - 会话级：

```
SET partition_table_limit = 30;
```

- 功能：
  - 当参数值不为 0 时，经过剪枝后的分区表子表数量达到或超过该参数设定值时，系统将优化执行计划的生成过程。具体而言，所有子表的执行计划将基于父表信息统一生成，从而提高效率。
  - 当该参数值为 0 时，则不启用分区表执行计划快速生成的功能。
- 示例：
  - 创建分区表

```
CREATE TABLE measurement_range
    (
    city_id INT NOT NULL,
    logdate DATE NOT NULL,
    peaktemp INT,
    unitsales INT
    )
PARTITION BY RANGE (logdate)
    ( PARTITION q1 VALUES LESS THAN ('2006-04-01'),
    PARTITION q2 VALUES LESS THAN ('2006-07-01'),
    PARTITION q3 VALUES LESS THAN ('2006-10-01'),
    PARTITION q4 VALUES LESS THAN ('2007-01-01')
    );
```

  - 插入数据

```
INSERT INTO measurement_range VALUES(1, '2006-01-01', 32, 2000);
INSERT INTO measurement_range VALUES(2, '2006-03-13', 30, 1000);
INSERT INTO measurement_range VALUES(3, '2006-04-01', 33, 5000);
INSERT INTO measurement_range VALUES(1, '2006-07-09', 24, 4000);
INSERT INTO measurement_range VALUES(2, '2006-08-11', 41, 4000);
INSERT INTO measurement_range VALUES(6, '2006-11-11', 23, 2000);
ANALYZE measurement_range;
```

– 查看参数值

```
show partition_table_limit ;

partition_table_limit
-----------------------
30
(1 row)
```

– 会话级修改

```
SET partition_table_limit TO 3;
```

– 使用优化的计划（所有子计划均根据父表生成）

```
EXPLAIN SELECT * FROM measurement_range WHERE logdate < '2006-09-01';
                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------
--------
Append  (cost=0.00..3.08 rows=4 width=20)
->  Seq Scan on measurement_range_q1  (cost=0.00..1.03 rows=1 width=20)
        Filter: ((logdate)::timestamp without time zone < '2006-09-01 00:00:00'::timestamp without time
zone)
->  Seq Scan on measurement_range_q2  (cost=0.00..1.03 rows=1 width=20)
        Filter: ((logdate)::timestamp without time zone < '2006-09-01 00:00:00'::timestamp without time
zone)
->  Seq Scan on measurement_range_q3  (cost=0.00..1.03 rows=1 width=20)
        Filter: ((logdate)::timestamp without time zone < '2006-09-01 00:00:00'::timestamp without time
zone)
(7 rows)
```

– 未使用优化（子表计划根据各自的统计信息生成）

```
EXPLAIN SELECT * FROM measurement_range WHERE logdate < '2006-06-01';
                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------
--------
Append  (cost=0.00..2.05 rows=3 width=20)
```

```
->  Seq Scan on measurement_range_q1  (cost=0.00..1.02 rows=2 width=20)
        Filter: ((logdate)::timestamp without time zone < '2006-06-01 00:00:00'::timestamp without time
zone)
->  Seq Scan on measurement_range_q2  (cost=0.00..1.01 rows=1 width=20)
        Filter: ((logdate)::timestamp without time zone < '2006-06-01 00:00:00'::timestamp without time
zone)
(5 rows)
```

- 适用场景：
  - 分区表子表数量较多；
  - 分区表上索引数量较多；
  - 对分区表执行查询/更新/删除操作时，并发数量较多（100 以上）；
- 不适用于以下分区表
  - 多级分区表；
  - 存在一个或多个子表的列顺序与父表不一致的情况；
- 不适用于以下 SQL 命令
  - SELECT FOR SHARE/UPDATE

```
EXPLAIN SELECT * FROM measurement_range WHERE logdate < '2006-09-01' FOR SHARE;
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------
--------------
LockRows  (cost=0.00..3.14 rows=5 width=30)
->  Append  (cost=0.00..3.09 rows=5 width=30)
        ->  Seq Scan on measurement_range_q1  (cost=0.00..1.02 rows=2 width=30)
            Filter: ((logdate)::timestamp without time zone < '2006-09-01 00:00:00'::timestamp without
time zone)
        ->  Seq Scan on measurement_range_q2  (cost=0.00..1.01 rows=1 width=30)
            Filter: ((logdate)::timestamp without time zone < '2006-09-01 00:00:00'::timestamp without
time zone)
        ->  Seq Scan on measurement_range_q3  (cost=0.00..1.02 rows=2 width=30)
            Filter: ((logdate)::timestamp without time zone < '2006-09-01 00:00:00'::timestamp without
time zone)
(8 rows)
```

```
EXPLAIN SELECT * FROM measurement_range WHERE logdate < '2006-09-01' FOR UPDATE;
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------
--------------
LockRows  (cost=0.00..3.14 rows=5 width=30)
->  Append  (cost=0.00..3.09 rows=5 width=30)
```

```
        ->  Seq Scan on measurement_range_q1  (cost=0.00..1.02 rows=2 width=30)
            Filter: ((logdate)::timestamp without time zone < '2006-09-01 00:00:00'::timestamp without
time zone)
        ->  Seq Scan on measurement_range_q2  (cost=0.00..1.01 rows=1 width=30)
            Filter: ((logdate)::timestamp without time zone < '2006-09-01 00:00:00'::timestamp without
time zone)
        ->  Seq Scan on measurement_range_q3  (cost=0.00..1.02 rows=2 width=30)
            Filter: ((logdate)::timestamp without time zone < '2006-09-01 00:00:00'::timestamp without
time zone)
(8 rows)
```

– INSERT SELECT

```
CREATE TABLE mr_t
(
city_id INT NOT NULL,
logdate DATE NOT NULL,
peaktemp INT,
unitsales INT
);

EXPLAIN INSERT INTO mr_t SELECT * FROM measurement_range;
                                QUERY PLAN
--------------------------------------------------------------------------------
Insert on mr_t  (cost=0.00..4.09 rows=6 width=20)
->  Append  (cost=0.00..4.09 rows=6 width=20)
        ->  Seq Scan on measurement_range_q1  (cost=0.00..1.02 rows=2 width=20)
        ->  Seq Scan on measurement_range_q2  (cost=0.00..1.01 rows=1 width=20)
        ->  Seq Scan on measurement_range_q3  (cost=0.00..1.02 rows=2 width=20)
        ->  Seq Scan on measurement_range_q4  (cost=0.00..1.01 rows=1 width=20)
(6 rows)
```

– UPDATE/DELETE RETURNING

```
EXPLAIN UPDATE measurement_range SET unitsales = 6500 WHERE logdate < '2006-09-01' AND peaktemp = 41
returning *;
                                                        QUERY PLAN
------------------------------------------------------------------------------------------------------
------------------------------
Update on measurement_range  (cost=0.00..3.08 rows=3 width=26)
Update on measurement_range_q1
Update on measurement_range_q2
Update on measurement_range_q3
->  Seq Scan on measurement_range_q1  (cost=0.00..1.03 rows=1 width=26)
        Filter: (((logdate)::timestamp without time zone < '2006-09-01 00:00:00'::timestamp without
```

```
time zone) AND (peaktemp = 41))
->  Seq Scan on measurement_range_q2  (cost=0.00..1.01 rows=1 width=26)
        Filter: (((logdate)::timestamp without time zone < '2006-09-01 00:00:00'::timestamp without
time zone) AND (peaktemp = 41))
->  Seq Scan on measurement_range_q3  (cost=0.00..1.03 rows=1 width=26)
        Filter: (((logdate)::timestamp without time zone < '2006-09-01 00:00:00'::timestamp without
time zone) AND (peaktemp = 41))
(10 rows)
```

– UPDATE/DELETE 包含 WITH CHECK 的指向分区表的视图

```
CREATE VIEW v_mr AS SELECT * FROM measurement_range WHERE unitsales > 3000 WITH CHECK OPTION;
EXPLAIN UPDATE v_mr SET unitsales = 2500 WHERE logdate < '2006-09-01' AND peaktemp = 41;
                                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------
-----------------------------------------------------
Update on measurement_range  (cost=0.00..3.09 rows=3 width=26)
Update on measurement_range_q1
Update on measurement_range_q2
Update on measurement_range_q3
->  Seq Scan on measurement_range_q1  (cost=0.00..1.03 rows=1 width=26)
        Filter: ((unitsales > 3000) AND ((logdate)::timestamp without time zone < '2006-09-01 00:00:00
'::timestamp without time zone) AND (peaktemp = 41))
->  Seq Scan on measurement_range_q2  (cost=0.00..1.02 rows=1 width=26)
        Filter: ((unitsales > 3000) AND ((logdate)::timestamp without time zone < '2006-09-01 00:00:00
'::timestamp without time zone) AND (peaktemp = 41))
->  Seq Scan on measurement_range_q3  (cost=0.00..1.03 rows=1 width=26)
        Filter: ((unitsales > 3000) AND ((logdate)::timestamp without time zone < '2006-09-01 00:00:00
'::timestamp without time zone) AND (peaktemp = 41))
(10 rows)
```

– UPDATE/DELETE 语句中涉及到多个表

```
EXPLAIN DELETE FROM measurement_range WHERE city_id IN (SELECT city_id FROM mr_t);
                                    QUERY PLAN
-------------------------------------------------------------------------------------
Delete on measurement_range  (cost=32.17..139.69 rows=4 width=12)
Delete on measurement_range_q1
Delete on measurement_range_q2
Delete on measurement_range_q3
Delete on measurement_range_q4
->  Hash Join  (cost=32.17..34.94 rows=1 width=12)
        Hash Cond: (mr_t.city_id = measurement_range_q1.city_id)
        ->  HashAggregate  (cost=31.12..33.12 rows=200 width=10)
            Group Key: mr_t.city_id
```

```
            ->  Seq Scan on mr_t  (cost=0.00..26.90 rows=1690 width=10)
        ->  Hash  (cost=1.02..1.02 rows=2 width=10)
            ->  Seq Scan on measurement_range_q1  (cost=0.00..1.02 rows=2 width=10)
->  Hash Join  (cost=32.15..34.91 rows=1 width=12)
        Hash Cond: (mr_t.city_id = measurement_range_q2.city_id)
        ->  HashAggregate  (cost=31.12..33.12 rows=200 width=10)
            Group Key: mr_t.city_id
            ->  Seq Scan on mr_t  (cost=0.00..26.90 rows=1690 width=10)
        ->  Hash  (cost=1.01..1.01 rows=1 width=10)
            ->  Seq Scan on measurement_range_q2  (cost=0.00..1.01 rows=1 width=10)
->  Hash Join  (cost=32.17..34.94 rows=1 width=12)
        Hash Cond: (mr_t.city_id = measurement_range_q3.city_id)
        ->  HashAggregate  (cost=31.12..33.12 rows=200 width=10)
            Group Key: mr_t.city_id
            ->  Seq Scan on mr_t  (cost=0.00..26.90 rows=1690 width=10)
        ->  Hash  (cost=1.02..1.02 rows=2 width=10)
            ->  Seq Scan on measurement_range_q3  (cost=0.00..1.02 rows=2 width=10)
->  Hash Join  (cost=32.15..34.91 rows=1 width=12)
        Hash Cond: (mr_t.city_id = measurement_range_q4.city_id)
        ->  HashAggregate  (cost=31.12..33.12 rows=200 width=10)
            Group Key: mr_t.city_id
            ->  Seq Scan on mr_t  (cost=0.00..26.90 rows=1690 width=10)
        ->  Hash  (cost=1.01..1.01 rows=1 width=10)
            ->  Seq Scan on measurement_range_q4  (cost=0.00..1.01 rows=1 width=10)
(33 rows)
```

4.6　使用物化视图

物化视图用于存储特定查询或中间过程的计算结果。其存储位置依据数据量大小有所不同：较小规模的数据存放在内存中以提升访问速度，较大规模的数据则存储于磁盘上。使用物化视图可以预先计算并存储那些耗时较长的数据库操作结果（如表连接和聚合操作），以便避免重复执行这些操作，从而提高查询效率。

然而，KingbaseES 数据库管理系统目前尚不支持物化视图的自动更新与增量更新功能。因此，在原始数据表更新频率较高且物化视图刷新速度相对较慢的情况下，使用物化视图可能并非理想选择。

因此，是否采用物化视图技术，需根据具体应用场景决定。

物化视图主要适用于以下两种场景：

- 当基础数据表的更新频率较低，但涉及复杂操作（如表关联）或需要高频访问时。
- 在查询过程中频繁引用外部数据库表且扫描操作耗时较长的情况下，可通过物化视图预先存储相关数据，并定期手动更新以保持数据同步。

示例：

```
CREATE extension file_fdw;
CREATE server local_file foreign data wrapper file_fdw;
CREATE FOREIGN TABLE words(word TEXT) server local_file options (filename '/data5.8T_3/coder/JOB/job_csv/
keyword.csv'); -- 创建外部表
CREATE materialized view wrd AS SELECT * FROM words; -- 创建物化视图
CREATE UNIQUE INDEX wrd_word ON wrd(word); -- 创建索引
ANALYZE wrd;
```

```
EXPLAIN ANALYZE SELECT COUNT(*) FROM words WHERE word = '224,cooking,C252';
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------
----
Aggregate  (cost=16657.51..16657.52 rows=1 width=8) (actual time=95.606..95.608 rows=1 loops=1)
   ->  Foreign Scan on words  (cost=0.00..16655.54 rows=790 width=0) (actual time=0.069..95.594 rows=1
loops=1)
         Filter: (word = '224,cooking,C252'::text)
         Rows Removed by Filter: 134169
         Foreign File: /data5.8T_3/coder/JOB/job_csv/keyword.csv
         Foreign File Size: 3791416 b
Planning Time: 0.129 ms
Execution Time: 95.705 ms
(8 行记录)
```

使用物化视图，上述查询执行时间由 95.7 毫秒变成 0.11 毫秒：

```
EXPLAIN ANALYZE SELECT COUNT(*) FROM wrd WHERE word = '224,cooking,C252';
                                                      QUERY PLAN
-----------------------------------------------------------------------------------------------------------
-------------
Aggregate  (cost=8.44..8.45 rows=1 width=8) (actual time=0.049..0.050 rows=1 loops=1)
   ->  Index Only Scan using wrd_word on wrd  (cost=0.42..8.44 rows=1 width=0) (actual time=0.039..0.041
rows=1 loops=1)
         Index Cond: (word = '224,cooking,C252'::text)
         Heap Fetches: 1
Planning Time: 0.279 ms
Execution Time: 0.110 ms
(6 行记录)
```

4.7　缓存执行计划

KingbaseES 数据库具备查询执行计划缓存功能，能够存储先前生成的执行策略。当接收到相同的查询请求时，系统将直接使用已缓存的执行方案，避免重复计算，从而提升系统运行效率。

KingbaseES 数据库中支持 PBE 扩展协议和 Q 报文简单协议下的执行计划缓存。这两种不同协议下，KingbaseES 采用不同的缓存方式。

- PBE 扩展协议下执行计划的缓存
- Q 报文简单协议下执行计划的缓存

4.7.1 PBE 扩展协议下执行计划的缓存

KingbaseES 数据库在通过 PBE 报文接收到 SQL 语句后，系统首先对其进行解析并生成相应的执行计划，随后将此执行计划缓存在本地内存中。在后续的消息处理过程中，通过 PBE 报文中的名字字段来检索缓存的执行计划。

当涉及 SQL 语句的相关数据库对象（如表结构、函数定义或统计信息）发生变化时，其对应的执行计划将被自动失效并移除。同时，KingbaseES 支持使用 DISCARD PLANS 命令手动清除所有缓存的执行计划，并通过配置 plan_cache_mode 参数调节系统的缓存策略：

- auto：默认值。对同一个 SQL，前五次不执行计划缓存，计算出一个执行代价的平均值，然后和缓存的执行计划代价对比。如果缓存的执行计划代价大，则不进行计划缓存。
- force_generic_plan：强制执行计划缓存
- force_custom_plan：强制不执行计划缓存，将 SQL 中的参数都替换为常量然后才生成执行计划。

4.7.2 Q 报文简单协议下执行计划的缓存

KingbaseES 在接收到 Q 报文中的 SQL 语句后，会生成相应的执行计划并将其存储在共享内存区域中。在后续处理过程中，系统能够通过计算得出的 SQLID 值来快速查询和获取缓存的执行计划。

该机制具备以下特点：

- 当与 SQL 相关的对象（如表结构、函数定义、统计信息等）发生变更时，系统会自动清空对应的执行计划。
- 用户可以通过特定命令对执行计划进行手动管理和控制。
- `SELECT remove_all_plan_cache()`：删除所有缓存的执行计划
- `SELECT remove_plan_cache(sqlid,sqlowner,sqldb)`：删除指定的缓存执行计划。id 为根据 SQL 字符串计算出的整数值。sqlowner 为数据库用户 OID。sqldb 为数据库 OID 。这三个值都可以为 NULL。
- `SELECT * from get_plan_cache()`：显示所有的缓存执行计划的 SQL 的列表。返回的值包括

属性	描述
sqlid	根据 SQL 字符串生成的 HASH 值
sqlowner	用户 OID
sqldb	数据库 OID

-续表-

属性	描述
usesize	缓存计划占用的共享内存空间
usecount	缓存计划被重用的次数
sqltext	SQL 文本
costs	缓存的执行计划代价。当没有缓存计划时，代价为-1

- `SELECT * from get_plan_cache(sqlid)`：显示执行 SQLID 的执行计划

通过以下系统级参数控制如何为 SQL 缓存执行计划：

- `simple_plan_cache_mode`：开关控制是否开启或关闭计划的缓存。取值范围为 OFF(不开启，默认值)，EXACT(SQL 语句完全相同时才会重用，否则会被重新执行硬解析操作)，FORCE(将 SQL 常量变为参数生成计划)。
- `simple_plan_cache_size`：控制支持的缓存执行计划的数量。取值为大于等于 0 的整数，默认值为 1000，即最多缓存 1000 个执行计划。
- `simple_plan_cache_custom`：取值为大于等于 0 的整数，默认值为 5。即对同一 SQL，前 5 次不使用执行计划缓存。将前 5 次生成的执行计划代价计算一个平均值，当要缓存的执行计划代价不大于该值才使用缓存的执行计划。通过该参数可以防止缓存了一个代价很高的执行计划。
- `simple_plan_cache_notcache_sql`：当用户指定一些 SQL 不希望进行缓存时，可以在配置文件中配置这些 SQL 的列表。取值为以分号分割的 SQL 字符串。采用模糊匹配的方式，所以对每个 SQL 我们可以只写出 SQL 的一部分即可。

在 simple_plan_cache_mode 设置模式为 FORCE 时，以下 SQL 语句不会进行缓存：

- Order，Group 后带有常量，则不进行计划缓存
- 查询条件中包含 LIKE，则不进行计划缓存
- SQL 中包含类型转换，则不进行缓存
- SQL 中包含属性为 immutable 并且没有参数的函数，则不进行缓存
- SQL 中包含 interval 或 Coalesc 关键字，则不进行缓存
- SQL 中包含临时表
- SQL 语句为 DDL
- SQL 中包含列表或数组，则不进行缓存

4.8　调整性能参数

SQL 性能优化涉及多种类型的配置参数设置，主要包括以下几类：查询成本参数、节点控制参数、多表连接参数以及内存管理相关参数等。在实际应用中应当充分考虑其作用域，可以优先采用 HINT 方式来指定参数，以确保对系统的影响范围可控。

这些参数的设置可以按照以下层级进行配置：

- 系统级配置：通过修改数据库配置文件 kingbase.conf，添加相应参数（如 `seq_page_cost=2`），并重启数据库服务以生效。
- 会话级配置：在数据库环境中执行相关命令（如 `seq_page_cost=2;`）进行临时设置，仅影响当前连接会话。
- 语句级配置：具体实现方式可参考官方文档中关于HINT的说明。

在参数管理方面，可以通过查询系统状态视图来查看当前设置情况。例如，使用 `SHOW seq_page_cost;` 命令可以验证参数的实际生效值。

4.8.1　成本参数

KingbaseES 优化器采用基于成本的算法来选择总成本最低的访问路径。在计算过程中，系统会调用一系列预定义的参数因子，并将其作为影响最终计算结果的关键变量。这些参数因子的设置直接影响了最终生成的总成本评估值，主要包括：

参数名称	取值范围	功能描述
seq_page_cost	数值，默认值为 1.0	表示全表扫描时，读取单个数据块的扫描成本，与磁盘性能有关
random_page_cost	数值，默认值为 4.0	表示使用索引扫描时，单个数据块的扫描成本，与磁盘性能有关
cpu_tuple_cost	数值，默认值为 0.01	表示读取一个元组时，CPU 消耗成本，与 CPU 性能有关
cpu_index_tuple_cost	数值，默认值为 0.005	表示从索引上读取一个元组时，CPU 消耗成本，与 CPU 性能有关
cpu_operator_cost	数值，默认值为 0.0025	表示执行一个操作符的 CPU 消耗成本，与 CPU 性能有关

当服务器硬件配置升级为更高性能的磁盘存储设备时，可以考虑对磁盘 I/O 相关的成本参数进行优化调谐。具体而言，适当下调 `seq_page_cost` 和 `random_page_cost` 的值，例如将其分别设置为 0.5 和 2.0。类似地，CPU 消耗相关的成本参数可以根据实际硬件性能情况灵活调整：当 CPU 处理能力较弱时，可适当提高其权重；而当 CPU 性能得到显著提升后，则可以考虑降低相关配置。

4.8.2　节点开关控制参数

KingbaseES 提供了灵活的优化器控制机制，支持针对具体查询关闭特定的优化特性。在实际应用中，如果优化器自动生成的执行计划并非最优选择，可以通过配置相关参数强制优化器采用更优的访问路径来解决问题。

扫描类开关	enable_seqscan	on/off，默认值为 on	表示是否允许使用全表扫描
	enable_indexscan	on/off，默认值为 on	表示是否允许使用索引扫描
	enable_bitmapscan	on/off，默认值为 on	表示是否允许使用 bitmap 扫描
	enable_tidscan	on/off，默认值为 on	表示是否允许使用 tid 扫描
连接类开关	enable_hashjoin	on/off，默认值为 on	表示是否允许使用 hash 连接
	enable_nestloop	on/off，默认值为 on	表示是否允许使用 nestloop 连接
	enable_mergejoin	on/off，默认值为 on	表示是否允许使用合并连接
其它开关	enable_hashagg	on/off，默认值为 on	表示是否允许使用 hash 的方式来计算 Aggregate 值
	enable_sort	on/off，默认值为 on	表示是否允许使用显式的 sort 节点
	enable_material	on/off，默认值为 on	表示是否允许使用物化节点

在采用参数开关机制对特定操作进行控制时，可以通过将其使用成本设置为较高的数值，从而降低该方法在优化器选择最优执行路径时被选中的概率。需要注意的是，这种设置并不能完全禁止该特性在某些情况下的应用。

由于这类全局性的配置调整会对所有 SQL 语句产生影响，因此并不推荐将其作为控制单个查询执行方式的主要手段。在这种情况下，建议优先采用 HINT 来进行更为精确和针对性的控制。

4.8.3　多表连接参数

当单个查询涉及的表数量达到或超过 12 个时，系统内部可能存在的访问路径组合将呈现指数级增长。为避免过多的可选路径导致 CPU 资源消耗过大，在这种情况下 KingbaseES 采用了基于遗传算法（Genetic Algorithm for Query Optimization, GEQO）的启发式搜索方法。

GEQO 通过限定搜索范围的方式来减少计算量，同时仍然能够有效地找到较优的执行计划。然而，由于其随机性和 heuristic 特性，生成的具体执行计划可能会存在一定的不确定性。

与遗传算法相关的关键参数包括：

- geqo
  - 参数说明：控制基因查询优化的使用
  - 取值范围：布尔类型，on 为开启，off 为关闭，默认值为 on。
- geqo_threshold
  - 参数说明：geqo 算法开启的阈值，当关系表的个数超过该值的时候，启用基因查询来做路径选择。
  - 取值范围：整型，2~INT_MAX，默认值为 12。
- geqo_effort
  - 参数说明：控制基因查询里规划时间和查询计划有效性之间的平衡，选的数值越大，则优化器花费更多的时间，选到更优的访问路径概率也更大。
  - 取值范围：整型，1~10，默认值为 5。

还有 4 个参数：`geqo_pool_size`, `geqo_generations`, `geqo_selection_bias`, `geqo_seed` 可以进一步控制基因查询的行为，请参考“遗传查询优化”章节。

4.8.4 内存设置相关参数

- work_mem
  - 参数说明：内部排序和哈希操作可使用的工作内存大小，该内存是在开始使用临时磁盘文件之前使用的内存数目，索引扫描不用 work_mem。
  - 取值范围：存储容量，默认值为 4MB。
  - 使用场景：
    * 排序操作：主要应用于 `ORDER BY`、`DISTINCT` 以及 `MERGE JOIN` 等查询操作
    * 哈希操作：广泛用于哈希连接（Hash Join）、哈希聚合（Hash Aggregation）以及基于哈希的 IN 子句处理

在处理复杂查询时，可能会并行执行多个排序或哈希操作，每个操作都会预先分配由 work_mem 参数声明的内存空间。当所有可用内存都被占用后，才会转而使用临时文件存储中间结果。值得注意的是，在多会话环境下，多个排序操作可能并行执行，导致整体内存消耗显著增加，通常为单个会话 work_mem 值的数倍。

- maintenance_work_mem
  - 参数说明：在维护性操作 (如 `VACUUM`、`CREATE INDEX`、`ALTER TABLE ADD FOREIGN KEY` 等) 中使用的最大的内存数。
  - 取值范围：存储容量，默认是 64MB

  - 推荐设置：建议根据具体场景进行调整。例如，在创建大型索引（如 10G 规模的数据）时，若内存资源充足，可适当增大该参数值（如设置为 1G）。通常在数据导入阶段可调高此参数以提升操作效率；在测试验证阶段可根据实际需求逐步优化并减小该值。

由于在一个数据库会话中，任一时刻仅有一个维护性操作可以执行，并且一般情况下数据库实例的并发维护操作数量有限，因此将 `maintenance_work_mem` 设置为比 `work_mem` 更大的值是安全可行的。较大的参数值有助于提升数据清理和恢复操作的速度，但需注意在测试环境中避免过度配置该参数以免造成资源浪费。

- temp_buffers
  - 参数说明：定义了数据库会话可使用的临时缓冲区最大数量。这些缓冲区仅用于访问本地临时表，并且每个会话可以根据需要按此限制分配临时缓冲区。需要注意的是，该参数仅在会话首次使用临时表之前方可修改；一旦会话中已使用过临时表，则后续对该值的修改将无效。
  - 内存管理机制：一个会话将根据 `temp_buffers` 给出的限制根据需要分配临时缓冲区。若在一个未充分利用临时缓冲区的会话中设置了较大的参数值，其资源开销仅体现为缓冲区描述符占用（每增加一个缓冲区描述符约需 64 字节）。然而，当某个实际使用的缓冲区超出初始分配时，则会额外消耗 BLCKSZ（默认 8192 字节）的内存空间。
  - 取值范围：整型，100~INT_MAX/2，默认值为 8MB。
- shared_buffers
  - 参数说明：数据库服务器使用的共享内存缓冲区的数量。主要用于缓存数据，根据需求一般不能设置超过 80% 的内存，但至少是 20%。
  - 取值范围：存储容量，默认值 128MB。

  KingbaseES 使用内存总大小为：

  ```
  shared_buffers(数据) + wal_buffers（日志 ）+ maintenance_work_mem（创建索引排序时使用）+ n*work_mem(n 为并发做排序的连接数) + 服务进程上下文使用的内存 (无法精确估计大小) + m*thread_stack_size
  ```

  `thread_stack_size` 为进程栈的大小，KingbaseES 默认为 1MB;`m` 为当前连接数。由于 32bit 平台的用户进程内存寻址空间较小 (window 为 2G、linux 为 3G)，所以这些内存的设置在 32bit 平台上的设置不能过大，防止内存溢出。

4.8.5 其他相关参数

- cursor_tuple_fraction
  - 参数说明：该参数用于控制优化器在处理游标查询时的规划策略。较小的参数值（0.1~1.0 之间）会导致优化器优先考虑游标快速返回第一条记录的需求，而较大的参数值则更注重总体查询代价的最优性。当设置为较小值时，优化器倾向于为游标生成执行计划，以实现快速获取首条记录的目标。然而，这种规划可能导致整体查询时间增加。反之，若设置为较大值（如 1.0），则优化器将与普通查询一样基于总体代价最低的原则进行规划，而不特别考虑前几行数据的返回速度。
  - 取值范围：浮点型数值，默认值为 0.1，有效区间为 0.1 至 1.0。

- from_collapse_limit
  - 参数说明：该参数决定了当生成的 FROM 列表项数不超过此值时，优化器将尝试将子查询与上层查询进行融合。较小的参数设置可能会缩短规划时间，但可能导致生成次优的查询执行计划。若将该参数设置为 `geqo_threshold` 或更大的值，则可能触发使用遗传查询优化（GEQO）算法进行规划，从而可能导致非最优的查询计划。
  - 取值范围：整数类型，默认值为 8。
- join_collapse_limit
  - 参数说明：该参数用于控制显式 JOIN 结构重写为 FROM 列表项的操作。若查询重写的 FROM 列表项目数不超过此值，则优化器将把除 FULL JOIN 外的所有显式 JOIN 结构调整到 FROM 项列表中。较小的参数设置可能会减少查询计划时间，但可能生成次优的执行计划。默认情况下，该参数与 `from_collapse_limit` 保持相同值以满足大多数场景需求。需要注意的是，若将此参数设置为 `geqo_threshold` 或更大的值，则可能触发使用遗传查询优化（GEQO）算法进行规划，从而可能导致非最优的查询计划。
  - 取值范围：整数，默认值为 8。

4.9 使用并行

KingbaseES 支持通过多核 CPU 并行执行单个 SQL 语句，这一功能被称为并行查询。本章将详细阐述并行查询的工作原理及其适用场景。

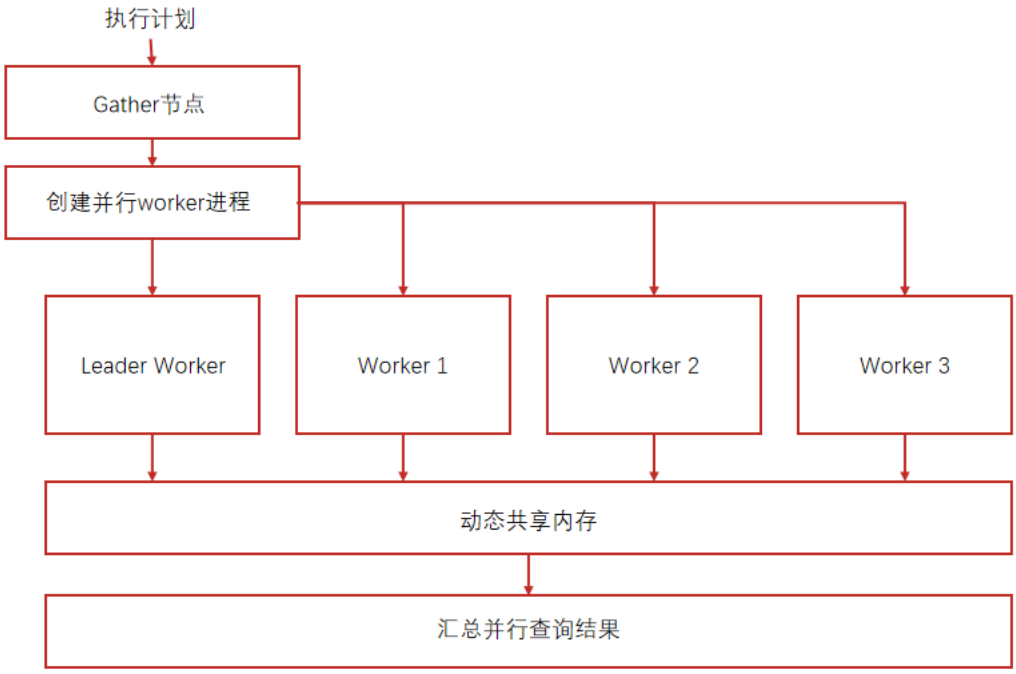
4.9.1 并行查询原理

在优化器评估某个特定查询时，若判断并行执行为最优策略，则会生成包含 Gather 或 GatherMerge 操作节点的查询执行计划。以一个普通并行查询为例：

```
CREATE TABLE student (id INT PRIMARY KEY, age INT); --
INSERT INTO student SELECT id, (random()*100)::INTEGER AS age FROM generate_series(1,10000000) AS id;
ANALYZE student;

EXPLAIN SELECT * FROM student WHERE age =6;

                                QUERY PLAN
-------------------------------------------------------------------------------
Gather (cost=1000.00..80481.52 rows=102334 width=8)
    Workers Planned: 5
    -> Parallel Seq Scan on student (cost=0.00..69248.12 rows=20467 width=8)
        Filter: (age = 6)
(4 行记录)
```

通过 EXPLAIN 工具可以查看并行查询的执行计划。在用户提交查询语句后，KingbaseES 的分析器、重写器以及优化器组件仍按常规流程运作，仅在生成执行计划时触发并行查询机制。

在执行计划中，Gather 节点作为其中一个操作符节点，标志着并行处理的开始。当查询执行到达该节点时，会话进程将根据配置参数及成本估算申请一定数量的工作线程（worker processes）。其中，一个特定的 worker 被指定为主工作线程（leader worker），负责协调和汇总其他 worker 线程的查询结果。主工作线程不仅承担协调任务，还会根据实际数据量参与部分并行处理工作。整个执行流程如下：



Leader 工作线程与其他工作线程通过动态共享内存实现通信机制。在并行查询过程中，所有其他 worker 进程（包括主工作线程）将各自处理的数据结果写入到共享内存区域中。主工作线程负责从共享内存中读取各 worker 的处理成果，并完成最终结果的汇总操作。

需要注意的是，并行查询功能依赖于动态共享内存机制，因此要求 dynamic_shared_memory_type 参数必须配置为“none”以外的有效值。

4.9.2　并行查询相关参数

要想使用并行查询，需要在 kinbase.conf 里设置以下参数：

并行度相关参数：

max_worker_processes	0 ~ 218 - 1 默认值：30	设置系统支持的最大后台进程数，此参数调整后需要重启数据库生效。
max_parallel_workers	0 ~ 1024 默认值：8	最大并行 worker 数。该数值不能大于 max_worker_processes。
max_parallel_workers_per_gather	默认值：2	最大并行执行 worker 数。不能超过 max_parrellel_workers。
max_parallel_maintenance_workers	默认值：2	最大并行维护操作 worker 数。不能超过 max_parrellel_workers。

这 4 个参数之间的关系为：

```
max_parallel_workers_per_gather + max_parallel_maintenance_workers <= max_parallel_workers <= max_worker_processes
```

并行触发条件参数:

参数名称	取值范围	描述
min_parallel_table_scan_size	0~2G 默认值：8MB	表的存储空间至少大于等于该值，才有可能触发并行
min_parallel_index_scan_size	0~2G 默认值：512KB	索引存储空间至少大于等于该数值，才有可能触发并行索引

可以通过以下语句来获得表、索引的磁盘存储大小：

```
SELECT sys_size_pretty(sys_relation_size('student'));
```

优化器控制开关：

参数名称	取值范围	描述
enable_parallel_append	on/off 默认值：on	优化器控制开关，是否允许并行 append。
enable_parallel_hash	on/off 默认值：on	优化器控制开关，是否允许并行 hash。

其他参数：

parallel_leader_participation	on/off; 默认值：on	控制 Gather、Gather merge 节点是否能执行 subplans。

parallel_setup_cost	默认值：1000	表示启动 woker process 的启动成本，因为启动 worker 进程需要建立共享内存等操作，属于附带的额外成本。其值越小，数据库越有可能使用并行查询。
parallel_tuple_cost	默认值：0.1	woker 进程处理完后的 tuple 要传输给上层 node，即进程间查询结果的交换成本，即后台进程间传输一个元组的代价。其值越小，数据库越有可能使用并行。

优化器主要根据 `max_worker_processes` 决定能够开启 worker 进程数后，根据目标表或者索引的大小（`min_parallel_table_scan_size`、`min_parallel_index_scan_size`），以及根据 `parallel_setup_cost` 和 `parallel_tuple_cost` 算出来的并行总 cost，跟其他不使用并行方案比较后决定是否启用并行，以及本次查询的并行进程数（小于等于 `max_parallel_workers`）。

4.9.3　并行查询支持的范围

目前 KingbaseES 支持并行的操作有四类：表扫描节点、join 节点、物化节点和控制节点。

4.9.3.1　扫描节点

KingbaseES 支持并行顺序扫描、索引扫描、index only 扫描以及 bitmap 扫描。

举例说明如下：

1. 并行顺序扫描：

```
EXPLAIN SELECT * FROM student WHERE age = 6;
                    QUERY PLAN
--------------------------------------------------------------------------------
Gather (cost=1000.00..102331.58 rows=50000 width=8)
    Workers Planned: 2
    -> Parallel Seq Scan on student (cost=0.00..96331.58 rows=20833 width=8)
        Filter: (age = 6)
(4 行记录)
```

2. 创建索引并降低并行代价，使用并行 bitmap 扫描：

```
CREATE INDEX ON student(age);
SET parallel_setup_cost =0;
SET parallel_tuple_cost =0;
EXPLAIN ANALYZE SELECT * FROM student WHERE age = 6;
```

```
                        QUERY PLAN
-----------------------------------------------------------------------------------------------------
------------------------------
Gather  (cost=1755.77..46489.89 rows=93334 width=8) (actual time=15.653..126.009 rows=99768 loops=1)
Workers Planned: 2
Workers Launched: 2
->  Parallel Bitmap Heap Scan on student  (cost=1755.77..46489.89 rows=38889 width=8) (actual time=5.
132..89.690 rows=33256 loops=3)
        Recheck Cond: (age = 6)
        Heap Blocks: exact=16906
        ->  Bitmap Index Scan on student_age_idx  (cost=0.00..1732.44 rows=93334 width=0) (actual
time=13.033..13.034 rows=99768 loops=1)
            Index Cond: (age = 6)
Planning Time: 0.069 ms
Execution Time: 129.468 ms
(10 rows)
```

3. 打开 hint，关闭优化器顺序扫描和 bitmap 扫描，使用并行 hint parallel(student 2) 强制优化器使用并行索引扫描。

```
SET enable_hint = on;
SET enable_seqscan =off;
SET enable_bitmapscan =off;
EXPLAIN SELECT/*+parallel(student 2)*/ * FROM student WHERE age = 6;
                                          QUERY PLAN
-----------------------------------------------------------------------------------------------------
-----
Gather  (cost=0.00..0.00 rows=99333 width=8)
Workers Planned: 2
->  Parallel Index Scan using student_age_idx on student  (cost=0.00..0.00 rows=41389 width=8)
        Index Cond: (age = 6)
(4 rows)
```

4.9.3.2　连接节点

KingbaseES 支持并行 HashJoin、嵌套循环连接、归并连接。但是嵌套循环连接和归并连接中，只有连接的外表是并行的，内表不并行。

本章节中使用物理优化章节中score表定义以及数据进行示例测试。

1. 降低并行代价，优化器选择并行归并连接：

```
SET parallel_setup_cost =0;
SET parallel_tuple_cost =0;
EXPLAIN SELECT * FROM score JOIN student ON score.sid = student.id;
```

```
                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------
-----
Gather  (cost=9.22..6389.26 rows=100000 width=16)
Workers Planned: 2
->  Merge Join  (cost=9.22..6389.26 rows=41667 width=16)
        Merge Cond: (student.id = score.sid)
        ->  Parallel Index Scan using student_pkey on student  (cost=0.43..246206.54 rows=4166687
width=8)
        ->  Index Scan using idx_sc on score  (cost=0.29..3054.29 rows=100000 width=8)
(6 rows)
```

2. 使用 `hint hashjoin(student score)`，强制两表使用 hash 连接。

```
SET enable_hint =on;
EXPLAIN SELECT/*+hashjoin(student, score)*/* FROM score JOIN student ON score.sid = student.id;
                                    QUERY PLAN
-----------------------------------------------------------------------------------
Gather  (cost=3084.00..137985.61 rows=100000 width=16)
Workers Planned: 2
->  Hash Join  (cost=3084.00..137985.61 rows=41667 width=16)
        Hash Cond: (student.id = score.sid)
        ->  Parallel Seq Scan on student  (cost=0.00..85914.87 rows=4166687 width=8)
        ->  Hash  (cost=1443.00..1443.00 rows=100000 width=8)
            ->  Seq Scan on score  (cost=0.00..1443.00 rows=100000 width=8)
(7 rows)
```

3. 使用 `hint nestloop(student score)`，并且关闭索引扫描和 bitmap 扫描，强制两表使用并行嵌套循环连接

```
SET enable_indexscan =off;
SET enable_bitmapscan =off;
EXPLAIN SELECT/*+nestloop(student, score)*/* FROM score JOIN student ON score.sid = student.id;
                                    QUERY PLAN
-----------------------------------------------------------------------------------
Gather  (cost=0.00..11220974005.87 rows=100000 width=16)
Workers Planned: 2
->  Nested Loop  (cost=0.00..11220974005.87 rows=41667 width=16)
        Join Filter: (score.sid = student.id)
        ->  Parallel Seq Scan on student  (cost=0.00..85914.87 rows=4166687 width=8)
        ->  Seq Scan on score  (cost=0.00..1443.00 rows=100000 width=8)
(6 rows)
```

4.9.3.3　物化节点

KingbaseES 支持并行 Sort、聚集等。

1. 并行排序 sort

   此处表 student 使用前面章节中的student表。

```
SET enable_hint=on;
EXPLAIN SELECT/*+parallel(student 2)*/ * FROM student ORDER BY sname;
                                    QUERY PLAN
-------------------------------------------------------------------------------
    Sort  (cost=27.69..29.19 rows=600 width=22)
    Sort Key: sname
    ->  Gather  (cost=0.00..0.00 rows=600 width=22)
            Workers Planned: 2
            ->  Parallel Seq Scan on student  (cost=0.00..0.00 rows=250 width=22)
(5 行记录)
```

2. 并行聚集

```
EXPLAIN SELECT AVG(age) FROM student;

                                        QUERY PLAN
-----------------------------------------------------------------------------------------
Finalize Aggregate (cost=97331.80..97331.81 rows=1 width=32)
    -> Gather (cost=97331.58..97331.79 rows=2 width=32)
        Workers Planned: 2
        -> Partial Aggregate (cost=96331.58..96331.59 rows=1 width=32)
            -> Parallel Seq Scan on student (cost=0.00..85914.87 rows=4166687 width=4)
(5 行记录)
```

4.9.3.4　控制节点

KingbaseES 支持并行 Append 节点。

```
CREATE TABLE student2 AS SELECT * FROM student;
ANALYZE student2;
SET enable_hint=on;
EXPLAIN select /*+parallelappend(2)*/ * from (SELECT * FROM student union all SELECT * FROM student2 WHERE
age = 6);
                                      QUERY PLAN
-------------------------------------------------------------------------------
 Gather  (cost=0.00..333822.18 rows=10105049 width=8)
   Workers Planned: 2
   ->  Parallel Append  (cost=0.00..333822.18 rows=10105049 width=8)
```

```
        ->  Parallel Seq Scan on student  (cost=0.00..85914.87 rows=4166687 width=8)
        ->  Parallel Seq Scan on student2  (cost=0.00..96331.58 rows=43750 width=8)
              Filter: (age = 6)
(6 行记录)
```

4.9.4　Worker 进程数与查询性能关系

虽然理论上增加并行处理能力可以提升查询效率，但实际效果需要综合考虑系统资源分配与任务调度的复杂性。下面通过在同一份数据集上执行相同的查询语句，逐步调整并行参数中的 worker 进程数量进行测试：

```
CREATE TABLE test(n BIGINT);
INSERT INTO test values(generate_series(1, 4000000));
ANALYZE test;
SET max_parallel_workers_per_gather = 1;
EXPLAIN ANALYZE SELECT * FROM test WHERE n = 4000000;

                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------------
---------
Gather (cost=1000.00..59888.81 rows=1 width=4) (actual time=244.328..301.458 rows=1 loops=1)
    Workers Planned: 1
    Workers Launched: 1
    -> Parallel Seq Scan on test (cost=0.00..58888.71 rows=1 width=4) (actual time=268.845..297.372 rows=0
loops=2)
        Filter: (n = 4000000)
        Rows Removed by Filter: 2500000
Planning time: 0.119 ms
Execution time: 302.026 ms
(8 rows)
```

```
SET max_parallel_workers_per_gather = 2;
EXPLAIN ANALYZE SELECT * FROM test WHERE n = 4000000;

                            QUERY PLAN
-----------------------------------------------------------------------------------------------------------
---------
Gather (cost=1000.00..49165.77 rows=1 width=4) (actual time=218.212..218.287 rows=1 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    -> Parallel Seq Scan on test (cost=0.00..48165.67 rows=1 width=4) (actual time=200.619..213.684 rows=0
loops=3)
        Filter: (n = 4000000)
        Rows Removed by Filter: 1666666
```

```
Planning time: 0.117 ms
Execution time: 219.005 ms
```

```
SET max_parallel_workers_per_gather = 4;
EXPLAIN ANALYZE SELECT * FROM test WHERE n = 4000000;

                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------
---------
Gather (cost=1000.00..43285.39 rows=1 width=4) (actual time=190.967..191.017 rows=1 loops=1)
    Workers Planned: 3
    Workers Launched: 3
    -> Parallel Seq Scan on test (cost=0.00..42285.29 rows=1 width=4) (actual time=174.275..182.767 rows=0
loops=4)
        Filter: (n = 4000000)
        Rows Removed by Filter: 1250000
Planning time: 0.119 ms
Execution time: 191.817 ms
(8 rows)
```

通过 `CREATE TABLE test(n INT) WITH (parallel_workers = 4);` 强制启动 4 个 worker 进程测试：

```
DROP TABLE IF EXISTS test;
CREATE TABLE test(n INT) WITH (parallel_workers = 4);
INSERT INTO test values(generate_series(1, 4000000));
ANALYZE test;
EXPLAIN ANALYZE SELECT * FROM test WHERE n = 4000000;

                                QUERY PLAN
-----------------------------------------------------------------------------------------------------
---------
Gather (cost=1000.00..38749.10 rows=1 width=4) (actual time=181.145..181.315 rows=1 loops=1)
    Workers Planned: 4
    Workers Launched: 4
    -> Parallel Seq Scan on test (cost=0.00..37749.00 rows=1 width=4) (actual time=163.565..169.837 rows=0
loops=5)
        Filter: (n = 4000000)
        Rows Removed by Filter: 1000000
Planning time: 0.050 ms
Execution time: 185.391 ms
```

通过对实验数据分析可观察到查询效率并非与 worker 进程数量呈线性增长关系：当配置 3 个和 4 个 worker 进程进行并行查询时，系统展现出趋于平稳的性能表现特征。这一现象表明在实际运行环境中，单纯增加并行处理单元并不能无限制地提升查询效率，而是受到系统资源分配机制的影响存在性能瓶颈。

值得注意的是，在 OLTP 事务型应用环境下，由于 CPU、共享内存等基础计算资源与 worker 进程执行环境均为数据库实例所共有，单个查询对资源的过度占用会显著影响其他并发会话的执行效能。当并行处理单元数量超出系统合理承载范围时，资源争用将导致整体性能下降。因此，并行参数的设置一定需要在一个合理的范围以内。

4.10　改写 SQL

4.10.1　Query Mapping 的功能

Query Mapping 功能是一项用于 SQL 语句转换与执行控制的关键特性。它允许用户预先定义并存储 SQL 语句的映射规则于系统表中。当接收到客户端查询请求时，系统会根据预设规则检查和替换输入的 SQL 语句，并使用优化后的目标 SQL 进行实际执行。

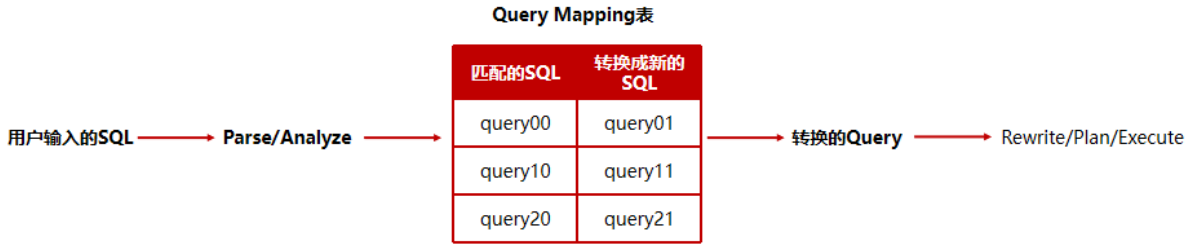
该特性为用户提供了一个灵活且强大的工具，用于自定义复杂的 SQL 转换逻辑，并将其高效应用于实时查询处理流程中，从而实现对查询行为的精确控制和性能优化。

- Query Mapping 的核心机制

  该功能主要提供两种级别的 SQL 映射处理能力：

  – TEXT 级别：不做词法/语法/语义检查，对 SQL 字符串进行替换。保存用户的原始 sql 和期望 mapping 的目标 sql，进行字符串匹配

  – SEMANTICS 级别：做词法/语法/语义语义检查，对 SQL 的查询树进行替换。当希望对不同的 DATABASE 或者不同 SCHEMA 下相同的 SQL 语句应用不同的替换规则时，可以使用该种模式



- Query Mapping 的的可用场景

– SQL 调优：将低效 SQL 语句转换成另一高效的等价 SQL 语句

– 异构数据库迁移：将源数据库的 SQL 语法翻译成 KingbaseES 库的 SQL 语法

提示:

1. 在希望结合使用 Query Mapping 与 Hint 功能时，必须采用 text 的映射规则进行配置。
2. 对于 SEMANTICS 级别的 Query mapping，目前只支持 SELECT 和 CALL 语句，不支持带有 rownum 的语句映射。
3. 当 SQL 语句中包含单引号时，根据 SQL 语法，创建 Query mapping 时应该改为两个单引号。如上面的例子所示，select '2222' 中单引号改为两个单引号，而不是双引号。
4. 匹配的 SQL 中的参数 1,2,…表示可以匹配任何的常量。例如当匹配 SQL 为 SELECT 1 时，代表只能匹配 SELECT 1。而当匹配 SQL 为 SELECT $1 时，可以匹配任何值，如 `SELECT 2`，`SELECT 'a'`。
5. 替换的 SQL 中的参数 1,2,…对应的是匹配 SQL 中常量值。例如匹配 SQL 为 SELECT 1,2, 替换 SQL 为 SELECT 2,1, 则 select 1,2 会转换为 select 2,1。
6. 匹配时忽略空格，回车，字符大小写。Text 模式当不希望忽略字符大小写时可以设置参数 `query_rule_ignorecase =off`。
7. 在系统表 sys_query_mapping 中查询已创建的转换规则。

4.10.2 Query Mapping 的使用

Query Mapping 的使用方法如下：

1. 在 kingbase.conf 配置文件中，配置：`enable_query_rule = on`
2. 启动数据库
3. 使用系统函数 `create_query_rule()` 来新增匹配规则，如：

```
SELECT create_query_rule(
'qm1', -- 规则名
'SELECT $1::TEXT AS col', -- 匹配的 sql
'SELECT ''2222''', -- 需要替换的 sql
true, -- 该规则是否生效
'semantics' -- 级别, text 或者 semantics
);
```

4. 接下来如果用户输入：`SELECT 'aaa' AS col`; 那么就会发生替换，其结果为：

```
?column?
----------
```

```
 2222
 (1 row)
```

5. 如果需要删除该规则，使用 `SELECT drop_query_rule('qm1');--` 入参 `qm1` 为规则名
6. 如果需要使原本不生效的规则生效，可以使用 `SELECT enable_query_rule('qm1');--` 入参 `qm1` 为规则名
7. 如果需要使原本生效的规则失效，可以使用 `SELECT disable_query_rule('qm1');--` 入参 `qm1` 为规则名

Query Mapping 的所有对外函数表如下：

名称	返回类型	描述
create_query_mapping_rule (text text text bool text)	void	创建一条 Query mapping 规则
drop_query_rule(text)	void	根据传入的规则名，删除一条 Query mapping 规则
drop_query_rule()	void	删除所有 Query mapping 规则
enable_query_rule(text)	void	根据传入的规则名，启用一条 Query mapping 规则
enable_query_rule()	void	启用所有 Query mapping 规则
disable_query_rule(text)	void	根据传入的规则名，禁用一条 Query mapping 规则
disable_query_rule()	void	禁用所有 Query mapping 规则

4.10.3　示例

准备数据

```
CREATE TABLE t1(id INT PRIMARY KEY, val INT, name VARCHAR(64));
CREATE TABLE t2(id INT, val INT, name VARCHAR(64));
INSERT INTO t1 SELECT i, i%10, 'Kingbase'||(i%5) FROM generate_series(1,1000) as x(i);
INSERT INTO t2 SELECT i, i%20, 'Kingbase'||(i%10) FROM generate_series(1,1000) as x(i);
```

1. 单个 变量 Query Mapping 规则，使用规则之前数据有 9 行 2 列，使用规则之后，数据变成 4 行 1 列

```
SELECT id,val FROM t1 WHERE id<10;
id  | val
----+-----
```

```
1   | 1
2   | 2
3   | 3
4   | 4
5   | 5
6   | 6
7   | 7
8   | 8
9   | 9
(9 rows)


SELECT create_query_rule('qm1', 'SELECT id,val FROM t1 WHERE id<$1','SELECT id FROM t1 WHERE id<($1-5)',
true, 'text');
create_query_rule;
------------------
(1 row)


SELECT id,val FROM t1 WHERE id<10;
id
----
1
2
3
4
(4 rows)
```

2. 多个 变量 Query Mapping 规则，使用规则之前数据有 2 行 2 列，使用规则之后，数据变成了 val=5 的总数，即 count 的值

```
SELECT id,val FROM t1 WHERE id<20 AND id>3 AND val=5;
id | val
---+-----
5  | 5
15 | 5
(2 rows)

SELECT create_query_rule('qm1', 'SELECT id,val FROM t1 WHERE id<$1 AND id>$2 AND val=$3', 'SELECT COUNT(0)
FROM t1 WHERE val=$3', true, 'text');
create_query_rule
-------------------
(1 row)
```

```
SELECT id,val FROM t1 WHERE id<20 AND id>3 AND val=5;
COUNT
-------
100
(1 row)
```

3. 单个 变量 Query Mapping 规则，使用之前是 11 行 6 列数据，使用规则后对表 t2 进行过滤，只有 1 行 6 列

```
SELECT * FROM t1, t2 WHERE t1.id=t2.id AND t1.id < 12;
id | val | name       | id  | val | name
---+-----+------------+-----+-----+----------
1  |  1  | Kingbase1  |  1  |  1  | Kingbase1
2  |  2  | Kingbase2  |  2  |  2  | Kingbase2
3  |  3  | Kingbase3  |  3  |  3  | Kingbase3
4  |  4  | Kingbase4  |  4  |  4  | Kingbase4
5  |  5  | Kingbase0  |  5  |  5  | Kingbase5
6  |  6  | Kingbase1  |  6  |  6  | Kingbase6
7  |  7  | Kingbase2  |  7  |  7  | Kingbase7
8  |  8  | Kingbase3  |  8  |  8  | Kingbase8
9  |  9  | Kingbase4  |  9  |  9  | Kingbase9
10 |  0  | Kingbase0  |  10 |  10 | Kingbase0
11 |  1  | Kingbase1  |  11 |  11 | Kingbase1
(11 rows)


SELECT create_query_rule('qm1', 'SELECT * FROM t1, t2 WHERE t1.id=t2.id AND t1.id<$1', 'SELECT  * FROM t1,
t2 WHERE t1.id=t2.id AND t1.id>($1-5) AND t1.id<$1 AND t2.val>($1-2) AND t2.val<($1+2)', true, 'text');
create_query_rule
-------------------
(1 row)
SELECT * FROM t1, t2 WHERE t1.id=t2.id AND t1.id < 12;
id  | val |    name    | id  | val | name
----+-----+------------+-----+-----+-----------
11  |  1  | Kingbase1 | 11  | 11  | Kingbase1
(1 row)
```

4. 多个 变量 Query Mapping 规则，使用前是 2 行 6 列数据，使用规则后变成 1 行 6 列

```
SELECT  * FROM t1, t2 WHERE t1.id=t2.id AND t1.id<30 AND t2.val=5;
id  | val |    name    | id | val | name
----+-----+------------+----+-----+-----------
5   |  5  | Kingbase0 | 5  |  5  | Kingbase5
25  |  5  | Kingbase0 | 25 |  5  | Kingbase5
(2 rows)
```

```
SELECT create_query_rule('qm1', 'SELECT * FROM t1, t2 WHERE t1.id=t2.id AND t1.id<$1 AND t2.val=$2', 'SELECT
* FROM t1, t2 WHERE t1.id=t2.id AND t1.id<$1 AND t2.val=$2 AND t2.id=$2', true, 'text');
create_query_rule
------------------
(1 row)


SELECT * FROM t1, t2 WHERE t1.id=t2.id AND t1.id<30 AND t2.val=5;
id  | val |    name    | id | val | name
----+-----+-----------+----+-----+-----------
5   |  5  | Kingbase0 | 5  |  5  | Kingbase5
(1 row)
```

5. $ 变量的交换规则

```
SELECT  * FROM t2 WHERE t2.id<50 AND t2.val=5;
id  | val | name
----+-----+-----------
5   |  5  | Kingbase5
25  |  5  | Kingbase5
45  |  5  | Kingbase5
(3 rows)


SELECT create_query_rule('qm1', 'SELECT * FROM t2 WHERE t2.id<$1 AND t2.val=$2', 'SELECT * FROM t2 WHERE t2.
id<$2 AND t2.val=$1', true, 'text');
create_query_rule
------------------
(1 row)


SELECT * FROM t2 WHERE t2.id<50 AND t2.val=5;
id  | val  | name
----+-----+------
(0 rows)
```

6. 条件下推 Query Mapping 规则，通过 Query Mapping 规则可以将外层连接条件下推到 union 子查询中

```
EXPLAIN SELECT COUNT(0) FROM t1, (SELECT  * FROM t2 WHERE t2.val>5 UNION SELECT  * FROM t1 WHERE t1.val <
20) AS v WHERE t1.id=v.id;

QUERY PLAN
-----------------------------------------------------------------
Aggregate (cost=134.48..134.49 rows=1 width=8)
```

```
    -> Hash Join (cost=91.75..130.23 rows=1700 width=0)
        Hash Cond: (t2.id = t1.id)
        -> HashAggregate (cost=67.25..84.25 rows=1700 width=154)
            Group Key: t2.id, t2.val, t2.name
            -> Append (cost=0.00..54.50 rows=1700 width=154)
                -> Seq Scan on t2 (cost=0.00..14.50 rows=700 width=18)
                    Filter: (val > 5)
                -> Seq Scan on t1 t1_1 (cost=0.00..14.50 rows=1000 width=18)
                    Filter: (val < 20)
        -> Hash (cost=12.00..12.00 rows=1000 width=4)
            -> Seq Scan on t1 (cost=0.00..12.00 rows=1000 width=4)
(12 rows)


SELECT create_query_rule('qm1', 'SELECT COUNT(0) FROM t1, (SELECT * FROM t2 WHERE t2.val>$1 UNION SELECT  *
FROM t1 WHERE t1.val<$2) AS v WHERE t1.id=v.id;',
'SELECT COUNT(0) FROM t1, lateral(SELECT  * FROM t2 WHERE t2.val>$1 AND t1.id=t2.id UNION SELECT  * FROM t1
t WHERE t.val<$2 AND t.id=t1.id) AS v;', true, 'text');
create_query_rule
-------------------
(1 row)


EXPLAIN (usingquerymapping) SELECT COUNT(0) FROM t1, (SELECT * FROM t2 WHERE t2.val>5 UNION SELECT  * FROM
t1 WHERE t1.val < 20) AS v WHERE t1.id=v.id;

QUERY PLAN
-----------------------------------------------------------------
Aggregate (cost=25287.00..25287.01 rows=1 width=8)
    -> Nested Loop (cost=25.21..25282.00 rows=2000 width=0)
        -> Seq Scan on t1 (cost=0.00..12.00 rows=1000 width=4)
        -> Unique (cost=25.21..25.23 rows=2 width=154)
            -> Sort (cost=25.21..25.22 rows=2 width=154)
                Sort Key: t2.id, t2.val, t2.name
                -> Append (cost=0.00..25.20 rows=2 width=154)
                    -> Seq Scan on t2 (cost=0.00..17.00 rows=1 width=18)
                        Filter: ((val > 5) AND (t1.id = id))
                    -> Index Scan using t1_pkey on t1 t (cost=0.15..8.17 rows=1 width=18)
                        Index Cond: (id = t1.id)
                        Filter: (val < 20)
(12 rows)
```

7. CTE Query Mapping 规则，查询语句被映射成 with recursive 处理

```
SELECT * FROM t1 WHERE id = 10;
id  | val  | name
----+-----+-----------
10  | 0  | Kingbase0
(1 row)


SELECT create_query_rule('qm1', 'SELECT * FROM t1 WHERE id = $1;','WITH recursive rtmp AS(SELECT id,val FROM
t1 WHERE id=$1 UNION SELECT t1.id,t1.val FROM t1,rtmp WHERE rtmp.id = t1.val)SELECT * FROM rtmp;',true,'text
');
create_query_rule
-------------------
(1 row)


SELECT * FROM t1 WHERE id = 10;
id  | val
----+-----
10  | 0
(1 row)
```

第5章　自动 SQL 优化组件

5.1　使用 SQL 优化建议器

5.1.1　概述

SQL 优化建议器能够根据数据库内部状态，向数据库管理员（DBA）提供调优建议。在应用这些优化建议后，经过调整的 SQL 语句性能将得到显著提升。

对于基于单条 SQL 语句的调优工作，系统可以直接生成并呈现具体的优化建议内容，无需额外进行任务管理或结果查询视图等操作，即可一次性获得全部调优结果。系统的调优任务采用阻塞方式运行以确保执行的稳定性和可靠性。

目前支持的调优建议有以下三种:

- 统计信息更新建议:

  在执行 SQL 语句的过程中，可能会引用一个或多个关系表。如果这些表的统计信息未能及时更新，或者缺乏多列统计信息，可能导致查询优化器生成非最优的执行计划。

  这一问题在表数据出现显著倾斜（即数据分布极不均衡）时尤为突出。针对输入的 SQL 语句，SQL 调优建议器将分析其引用的所有关系表，并对每个表进行统计信息检查。

  当检测到表中修改的数据量超过 autovacuum 的计算结果时，系统会自动生成收集统计信息的建议。

- 索引建议:

  对单条 SQL 语句进行分析时，系统会检查其涉及的表在过滤或连接条件中是否被引用。如果发现相关表缺少必要的索引，则自动应用虚拟索引技术进行处理。通过对比执行计划前后的性能数据，若性能提升幅度达到 5% 以上，则将给出则给出索引建议、建议索引类型、创建索引的 DDL 语句、预期收益率等优化方案。

- SQL 改写建议:

  通过对预设改写规则进行配置，系统能够识别符合条件的 SQL 语句并生成相应的优化建议。具体支持的规则类型包括：

规则	说明
UNION 转 UNION ALL	在不影响结果集的情况下，建议把 UNION 改写为 UNION ALL
DELETE 转 TRUNCATE	DELETE 删除全表数据时，建议使用 TRUNCATE
隐式类型转换导致的索引失效	索引列的类型被转换之后，没有使用索引。建议显示指定过滤条件的数据类型

5.1.2　调优报告说明

为提升用户体验，SQL 优化功能提供了专门针对数据库管理员（DBA）设计的调优建议报告接口。该报告内容涵盖以下几个方面：

1. GENERAL INFORMATION：一般信息

   包括任务名称、任务所有者、状态、执行开始和完成时间、SQL ID 和 SQL 语句。

```
-----------------------------------------------------------------------------------------------------------+
基本信息                                                                                                    +
-----------------------------------------------------------------------------------------------------------+
调优任务名称          :  quick tune                                                                         +
调优任务所有者        :  coder                                                                              +
负载类型              :  Single SQL Statement                                                               +
调优域                :  COMPREHENSIVE                                                                      +
限制时间(秒)          :  0                                                                                  +
完成状态              :  COMPLETED                                                                          +
开始时间              :  2024-04-24 11:44:38+00                                                             +
完成时间              :  2024-04-24 11:44:38+00                                                             +
数据库名              :  test                                                                               +
SQL ID                :  1238646457857880359                                                                +
SQL 语句              :  select * from t where id = $1 union select * from t where id = $2                  +
```

2. FINGDING：调优发现

   包括优化建议的动作：索引建议和收集统计信息会给出具体 SQL，改写建议不会给出改写后的 SQL。

   预期收益：仅索引建议有，收集统计信息和 SQL 改写没有。

   基本原理：索引建议和收集统计信息有，改写建议没有。

```
-----------------------------------------------------------------------------------------------------------+
发现段 (2 个发现)                                                                                            +
-----------------------------------------------------------------------------------------------------------+
1- 索引发现 (查看执行计划段)                                                                                  +
-----------------------------------------------------------------------------------------------------------+
        创建一个或多个索引，能够改善SQL语句的执行计划。                                                       +
                                                                                                            +
        建议(预估收益：99.52%)                                                                               +
        ----------------------------------------                                                            +
        考虑创建推荐的索引。                                                                                  +
        CREATE INDEX ON public.t USING hash (id)                                                            +
                                                                                                            +
        基本原理                                                                                             +
        ---------                                                                                           +
        创建推荐的索引能够显著改善SQL语句的执行计划。                                                         +
                                                                                                            +
2- 改写SQL发现                                                                                              +
-------------------------                                                                                   +
        在执行计划中发现了代价较高的 "UNION" 操作。                                                           +
                                                                                                            +
        建议                                                                                                 +
        --------------                                                                                      +
        如果数据允许重复或唯一性有保证，考虑使用 "UNION ALL" 代替 "UNION"。                                   +
                                                                                                            +
```

3. EXPLAIN PLANS：执行计划

优化前后执行计划对比（仅索引建议有，收集统计信息和 SQL 改写没有）。

```
-----------------------------------------------------------------------------------------------------------+
执行计划段                                                                                                   +
-----------------------------------------------------------------------------------------------------------+
1- 原始执行计划                                                                                              +
--------------------                                                                                        +
Unique  (cost=3386.04..3386.05 rows=2 width=4)                                                              +
  ->  Sort  (cost=3386.04..3386.05 rows=2 width=4)                                                          +
        Sort Key: t.id                                                                                      +
        ->  Append  (cost=0.00..3386.03 rows=2 width=4)                                                     +
              ->  Seq Scan on t  (cost=0.00..1693.00 rows=1 width=4)                                        +
                    Filter: (id = $1)                                                                       +
              ->  Seq Scan on t t_1  (cost=0.00..1693.00 rows=1 width=4)                                    +
                    Filter: (id = $2)                                                                       +
                                                                                                            +
2- 新索引执行计划                                                                                            +
--------------------                                                                                        +
Unique  (cost=16.08..16.09 rows=2 width=4)                                                                  +
  ->  Sort  (cost=16.08..16.08 rows=2 width=4)                                                              +
        Sort Key: t.id                                                                                      +
        ->  Append  (cost=0.00..16.07 rows=2 width=4)                                                       +
              ->  Index Scan using <12614>hash_t_id on t  (cost=0.00..8.02 rows=1 width=4)                  +
                    Index Cond: (id = $1)                                                                   +
              ->  Index Scan using <12614>hash_t_id on t t_1  (cost=0.00..8.02 rows=1 width=4)              +
                    Index Cond: (id = $2)                                                                   +
                                                                                                            +
```

5.1.3　调优接口说明

报告通过以下接口，通过 SQL 语句、QUERY_ID 指定语句生成。

调优报告接口:

接口名称	说明
PERF.QUICK_TUNE_BY_SQL(SQL_TEXT)	输入一条 SQL 语句，生成调优建议报告

见续表

表 5.1.1 – 续表

接口名称	说明
PERF.QUICK_TUNE_BY_QUERYID(QUERY_ID)	输入一条 SQL 语句的查询 ID，生成调优建议报告。
PERF.QUICK_TUNE_BY_SQL_TO_FILE (SQL_TEXT, FORMAT, FILE_PATH)	输入一条 SQL 语句，生成调优建议报告到指定文件。
PERF.QUICK_TUNE_BY_QUERYID_TO_FILE (QUERY_ID, FORMAT, FILE_PATH)	输入一条 SQL 语句的查询 ID，生成调优建议报告到指定文件。

接口参数说明:

SQL_TEXT	指定输入的 SQL 语句
QUERY_ID	指定输入的 SQL 语句的 QUERY_ID，来源为 sys_stat_statements 视图
FORMAT	报告格式，仅支持 TEXT
FILE_PATH	报告路径和名称

GUC 参数：

参数名称	说明
sys_sqltune.language	会话级参数，指定调优报告语言为中文或英文；默认为中文（chinese 或 chn），可选为英文 (english 或 eng)

5.1.4　调优建议报告生成

使用 SQL 优化建议的方法如下：可直接针对 SQL 语句生成调优建议报告，或通过输入的 SQL 语句的 QUERYID 参数指定具体查询，并自动生成优化建议，结果将存储于目标文件夹中。示例如下：

5.1.4.1　基本配置

1. kingbase.conf 文件设置:

```
shared_preload_libraries = 'plsql, sys_stat_statements, sys_sqltune'
```

2. 启动数据库
3. 创建 sys_sqltune 插件：

```
CREATE extension sys_sqltune ;
```

5.1.4.2　生成统计信息更新建议

1. 配置文件 kingbase.conf 中设置

```
Autovacuum = off; -- 临时关闭统计信息自动更新
```

2. 重启数据库

3. 创建表、插入数据

```
CREATE TABLE t1 (id1 INT);
INSERT INTO t1 SELECT generate_series(1,100000);
```

4. 调用接口生成调优报告:

```
SELECT PERF.QUICK_TUNE_BY_SQL('SELECT * FROM t1');
```

```
test=# select PERF.QUICK_TUNE_BY_SQL('select * from t1');
                                          quick_tune_by_sql
-----------------------------------------------------------------------------------------------------------------------
 -----------------------------------------------------------------------------------------------------------------------+
 基本信息                                                                                                                +
 -----------------------------------------------------------------------------------------------------------------------+
 调优任务名称          :  quick tune                                                                                     +
 调优任务所有者        :  ljy                                                                                            +
 负载类型              :  Single SQL Statement                                                                           +
 调优域                :  COMPREHENSIVE                                                                                  +
 限制时间(秒)          :  0                                                                                              +
 完成状态              :  COMPLETED                                                                                      +
 开始时间              :  2024-11-11 11:27:02+08                                                                         +
 完成时间              :  2024-11-11 11:27:02+08                                                                         +
 数据库名              :  test                                                                                           +
 SQL ID                :                                                                                                 +
 SQL 语句              :  select * from t1                                                                               +
 -----------------------------------------------------------------------------------------------------------------------+
 发现段 (1 个发现)                                                                                                       +
 -----------------------------------------------------------------------------------------------------------------------+
 1- 统计信息发现                                                                                                         +
 --------------------                                                                                                    +
         表 "public.t1" 的优化器统计信息已过时。                                                                         +
                                                                                                                         +
         建议                                                                                                            +
         --------------                                                                                                  +
         考虑收集表的优化器统计信息。                                                                                    +
         执行 analyze public.t1                                                                                          +
                                                                                                                         +
         基本原理                                                                                                        +
         ---------                                                                                                       +
         优化器需要表的最新统计信息，以便选择一个好的执行计划。                                                          +
                                                                                                                         +
 -----------------------------------------------------------------------------------------------------------------------+
 执行计划段                                                                                                              +
 -----------------------------------------------------------------------------------------------------------------------+
 1- 原始执行计划                                                                                                         +
 --------------------                                                                                                    +
 Seq Scan on t1  (cost=0.00..1568.22 rows=112522 width=4)                                                                +
                                                                                                                         +

(1 行记录)
```

5.1.4.3　生成索引建议

1. 创建表、插入数据

```
CREATE TABLE t2 (id1 INT);
INSERT INTO t2 SELECT generate_series(1,100000);
```

2. 调用接口生成调优报告:

```
SELECT PERF.QUICK_TUNE_BY_SQL('SELECT * FROM t2 WHERE id1=12345');
```

```
test=# select PERF.QUICK_TUNE_BY_SQL('select * from t2 where id1=12345');
                                                 quick_tune_by_sql
-----------------------------------------------------------------------------------------------------------------------
 -----------------------------------------------------------------------------------------------------------------------+
 基本信息                                                                                                                +
 -----------------------------------------------------------------------------------------------------------------------+
 调优任务名称          :  quick tune                                                                                     +
 调优任务所有者        :  ljy                                                                                            +
 负载类型              :  Single SQL Statement                                                                           +
 调优域                :  COMPREHENSIVE                                                                                  +
 限制时间(秒)          :  0                                                                                              +
 完成状态              :  COMPLETED                                                                                      +
 开始时间              :  2024-11-11 14:00:22+08                                                                         +
 完成时间              :  2024-11-11 14:00:22+08                                                                         +
 数据库名              :  test                                                                                           +
 SQL ID                :                                                                                                 +
 SQL 语句              :  select * from t2 where id1=12345                                                               +
 -----------------------------------------------------------------------------------------------------------------------+
 发现段 (1 个发现)                                                                                                       +
 -----------------------------------------------------------------------------------------------------------------------+
 1- 索引发现 (查看执行计划段)                                                                                            +
 ------------------------------------------------                                                                      +
         创建一个或多个索引，能够改善SQL语句的执行计划。                                                                 +
                                                                                                                       +
         建议(预估收益：99.53%)                                                                                          +
         ----------------------------------------                                                                      +
         考虑创建推荐的索引。                                                                                            +
         CREATE INDEX ON public.t2 USING hash (id1)                                                                    +
                                                                                                                       +
         基本原理                                                                                                        +
         ---------                                                                                                     +
         创建推荐的索引能够显著改善SQL语句的执行计划。                                                                   +
                                                                                                                       +
 -----------------------------------------------------------------------------------------------------------------------+
 执行计划段                                                                                                              +
 -----------------------------------------------------------------------------------------------------------------------+
 1- 原始执行计划                                                                                                         +
 --------------------                                                                                                  +
 Seq Scan on t2  (cost=0.00..1693.00 rows=1 width=4)                                                                   +
   Filter: (id1 = 12345)                                                                                               +
                                                                                                                       +
 2- 新索引执行计划                                                                                                       +
 --------------------                                                                                                  +
 Index Scan using <12614>hash_t2_id1 on t2  (cost=0.00..8.02 rows=1 width=4)                                           +
   Index Cond: (id1 = 12345)                                                                                           +
                                                                                                                       +

(1 行记录)
```

5.1.4.4　生成 SQL 改写建议

1. 使用 t1,t2 表信息，生成 union 改写建议

若查询结果中不存在重复记录，使用 UNION ALL 替代 UNION 操作可以提高查询效率。因此，在优化 SQL 语

句时，建议对使用了 UNION 操作的部分进行评估，若符合上述条件则推荐采用 UNION ALL 以获得更好的执行效果。

```
SELECT PERF.QUICK_TUNE_BY_SQL('SELECT * FROM t1 UNION SELECT * FROM t2');
```

```
test=#  select PERF.QUICK_TUNE_BY_SQL('select * from t1 union select * from t2');
                                          quick_tune_by_sql
-----------------------------------------------------------------------------------------------------------------
 -----------------------------------------------------------------------------------------------------------------+
 基本信息                                                                                                         +
 -----------------------------------------------------------------------------------------------------------------+
 调优任务名称        :  quick tune                                                                                +
 调优任务所有者      :  coder                                                                                     +
 负载类型            :  Single SQL Statement                                                                      +
 调优域              :  COMPREHENSIVE                                                                             +
 限制时间(秒)        :  0                                                                                         +
 完成状态            :  COMPLETED                                                                                 +
 开始时间            :  2025-02-14 03:28:44+00                                                                    +
 完成时间            :  2025-02-14 03:28:44+00                                                                    +
 数据库名            :  test                                                                                      +
 QueryID             :                                                                                            +
 SQL 语句            :  select * from t1 union select * from t2                                                   +
 -----------------------------------------------------------------------------------------------------------------+
 发现段 (3 个发现)                                                                                                +
 -----------------------------------------------------------------------------------------------------------------+
 1- 统计信息发现                                                                                                  +
 --------------------                                                                                             +
         表 "public.t1" 的优化器统计信息已过时.                                                                   +
                                                                                                                  +
         建议                                                                                                     +
         --------------                                                                                           +
         考虑收集表的优化器统计信息.                                                                              +
         执行 analyze public.t1                                                                                   +
                                                                                                                  +
         基本原理                                                                                                 +
         ---------                                                                                                +
         优化器需要表的最新统计信息, 以便选择一个好的执行计划.                                                    +
                                                                                                                  +
 2- 统计信息发现                                                                                                  +
 --------------------                                                                                             +
         表 "public.t2" 的优化器统计信息已过时.                                                                   +
                                                                                                                  +
         建议                                                                                                     +
         --------------                                                                                           +
         考虑收集表的优化器统计信息.                                                                              +
         执行 analyze public.t2                                                                                   +
                                                                                                                  +
         基本原理                                                                                                 +
         ---------                                                                                                +
         优化器需要表的最新统计信息, 以便选择一个好的执行计划.                                                    +
                                                                                                                  +
 3- 改写SQL发现                                                                                                   +
 -------------------------                                                                                        +
         在执行计划中发现了代价较高的 "UNION" 操作.                                                               +
                                                                                                                  +
         建议                                                                                                     +
         --------------                                                                                           +
         如果数据允许重复或唯一性有保证, 考虑使用 "UNION ALL" 代替 "UNION".                                       +
                                                                                                                  +
 -----------------------------------------------------------------------------------------------------------------+
 执行计划段                                                                                                       +
 -----------------------------------------------------------------------------------------------------------------+
 1- 原始执行计划                                                                                                  +
 --------------------                                                                                             +
 Unique  (cost=29598.34..30723.56 rows=225044 width=4)                                                            +
   ->  Sort  (cost=29598.34..30160.95 rows=225044 width=4)                                                        +
         Sort Key: t1.id1                                                                                         +
         ->  Append  (cost=0.00..6512.10 rows=225044 width=4)                                                     +
               ->  Seq Scan on t1  (cost=0.00..1568.22 rows=112522 width=4)                                       +
               ->  Seq Scan on t2  (cost=0.00..1568.22 rows=112522 width=4)                                       +
                                                                                                                  +

(1 row)
```

2. 使用 t2 表信息，生成 delete 改写建议

TRUNCATE 命令可以快速删除表中所有记录，并且无需后续执行 VACUUM 操作来释放已占用的空间。但该操作存在一定的局限性：它会破坏严格的多版本并发控制（MVCC）机制，可能影响事务的隔离性和一致性特性。因此，在识别出需要执行全表删除操作时，建议采用更为安全的数据清除方法。

```
SELECT PERF.QUICK_TUNE_BY_SQL('DELETE FROM t2');
```

```
test=# select PERF.QUICK_TUNE_BY_SQL('delete from t2');
                                                  quick_tune_by_sql
-----------------------------------------------------------------------------------------------------------------------
 -----------------------------------------------------------------------------------------------------------------------+
 基本信息                                                                                                                +
 -----------------------------------------------------------------------------------------------------------------------+
 调优任务名称          :  quick tune                                                                                     +
 调优任务所有者        :  ljy                                                                                            +
 负载类型              :  Single SQL Statement                                                                           +
 调优域                :  COMPREHENSIVE                                                                                  +
 限制时间(秒)          :  0                                                                                              +
 完成状态              :  COMPLETED                                                                                      +
 开始时间              :  2024-11-11 14:15:24+08                                                                         +
 完成时间              :  2024-11-11 14:15:24+08                                                                         +
 数据库名              :  test                                                                                           +
 SQL ID                :                                                                                                 +
 SQL 语句              :  delete from t2                                                                                 +
 -----------------------------------------------------------------------------------------------------------------------+
 发现段 (1 个发现)                                                                                                       +
 -----------------------------------------------------------------------------------------------------------------------+
 1- 改写SQL发现                                                                                                          +
 --------------------------                                                                                              +
         在执行计划中发现了全表数据的 "DELETE" 操作。                                                                    +
                                                                                                                         +
         建议                                                                                                            +
         --------------                                                                                                  +
         考虑使用 "TRUNCATE" 代替 "DELETE"，注意 truncate 操作违反了 MVCC 语义。                                        +
                                                                                                                         +
 -----------------------------------------------------------------------------------------------------------------------+
 执行计划段                                                                                                              +
 -----------------------------------------------------------------------------------------------------------------------+
 1- 原始执行计划                                                                                                         +
 --------------------                                                                                                    +
 Delete on t2  (cost=0.00..1443.00 rows=100000 width=6)                                                                  +
   ->  Seq Scan on t2  (cost=0.00..1443.00 rows=100000 width=6)                                                          +
                                                                                                                         +

(1 行记录)
```

3. 类型转换的改写建议

由于隐式数据类型转换可能导致索引失效的问题，在检测到查询条件中的常量或变量与索引列的数据类型不匹配时，给出改写建议。

```
CREATE TABLE t3(id VARCHAR(32));
CREATE INDEX idx ON t3(id);
INSERT INTO t3 SELECT generate_series(1,1000000);

SELECT PERF.QUICK_TUNE_BY_SQL('SELECT * FROM t3 WHERE id = 1000');
```

```
test=# select PERF.QUICK_TUNE_BY_SQL('select * from t3 where id = 1000');
                                                  quick_tune_by_sql
-----------------------------------------------------------------------------------------------------------------------
 -----------------------------------------------------------------------------------------------------------------------+
 基本信息                                                                                                                +
 -----------------------------------------------------------------------------------------------------------------------+
 调优任务名称          :  quick tune                                                                                     +
 调优任务所有者        :  ljy                                                                                            +
 负载类型              :  Single SQL Statement                                                                           +
 调优域                :  COMPREHENSIVE                                                                                  +
 限制时间(秒)          :  0                                                                                              +
 完成状态              :  COMPLETED                                                                                      +
 开始时间              :  2024-11-11 14:27:09+08                                                                         +
 完成时间              :  2024-11-11 14:27:09+08                                                                         +
 数据库名              :  test                                                                                           +
 SQL ID                :                                                                                                 +
 SQL 语句              :  select * from t3 where id = 1000                                                               +
 -----------------------------------------------------------------------------------------------------------------------+
 发现段 (2 个发现)                                                                                                       +
 -----------------------------------------------------------------------------------------------------------------------+
 1- 索引发现 (查看执行计划段)                                                                                            +
 ------------------------------------------------                                                                      +
         创建一个或多个索引，能够改善SQL语句的执行计划。                                                                 +
                                                                                                                       +
         建议(预估收益：60.4%)                                                                                           +
         ----------------------------------------                                                                      +
         考虑创建推荐的索引。                                                                                            +
         CREATE INDEX ON public.t3 USING hash (int4(id))                                                               +
                                                                                                                       +
         基本原理                                                                                                        +
         ---------                                                                                                     +
         创建推荐的索引能够显著改善SQL语句的执行计划。                                                                   +
                                                                                                                       +
 2- 改写SQL发现                                                                                                          +
 -------------------------                                                                                             +
         在执行计划中发现了表的字段类型转换操作。                                                                        +
         表 public.t3 字段 (id) 原类型是 varchar，转换类型是 int4                                                        +
                                                                                                                       +
         建议                                                                                                            +
         ---------------                                                                                               +
         考虑改写操作数的数据类型，以避免字段类型发生转换。                                                              +
                                                                                                                       +
 -----------------------------------------------------------------------------------------------------------------------+
 执行计划段                                                                                                              +
 -----------------------------------------------------------------------------------------------------------------------+
 1- 原始执行计划                                                                                                         +
 --------------------                                                                                                  +
 Gather  (cost=1000.00..12175.00 rows=5000 width=6)                                                                    +
   Workers Planned: 2                                                                                                  +
   ->  Parallel Seq Scan on t3  (cost=0.00..10675.00 rows=2083 width=6)                                                +
         Filter: ((id)::integer = 1000)                                                                                +
                                                                                                                       +
 2- 新索引执行计划                                                                                                       +
 --------------------                                                                                                  +
 Bitmap Heap Scan on t3  (cost=110.75..4821.13 rows=5000 width=6)                                                      +
   Recheck Cond: ((id)::integer = 1000)                                                                                +
   ->  Bitmap Index Scan on <12622>hash_t3_expr  (cost=0.00..109.50 rows=5000 width=0)                                 +
         Index Cond: ((id)::integer = 1000)                                                                            +
                                                                                                                       +

(1 行记录)
```

5.1.4.5　使用 queryId 生成建议

使用 queryid 来指定语句生成建议，并保存到文件。

1. 开启 sys_stat_Statements 统计

```
set sys_stat_statements.track = 'top';
```

```
CREATE TABLE t1(id1 INT);
```

2. 执行语句插入 sys_stat_statements

```
SELECT * FROM t1 WHERE id1 = 1 UNION SELECT * FROM t1 WHERE id1 = 1;
```

3. 查询语句 queryid

```
SELECT queryid FROM sys_stat_statements WHERE query LIKE 'SELECT * FROM t1 WHERE%';
```

```
test=# select queryid from sys_stat_statements where query like 'select * from t1 where%';
       queryid
----------------------
 -8959188754910793169
(1 行记录)
```

4. 生成调优建议

```
SELECT PERF.QUICK_TUNE_BY_QUERYID_TO_FILE(-8959188754910793169, 'text', 'sqltuning.txt');
```

```
test=# select PERF.QUICK_TUNE_BY_QUERYID_TO_FILE(-8959188754910793169, 'text', 'sqltuning.txt');
 quick_tune_by_queryid_to_file
-------------------------------
 t
(1 行记录)
```

5. 报告文件默认在 data 目录下

```
-----------------------------------------------------------------------------------------------------------
基本信息
-----------------------------------------------------------------------------------------------------------
调优任务名称          :  quick tune
调优任务所有者        :  ljy
负载类型              :  Single SQL Statement
调优域                :  COMPREHENSIVE
限制时间(秒)          :  0
完成状态              :  COMPLETED
开始时间              :  2024-11-11 14:55:06+08
完成时间              :  2024-11-11 14:55:06+08
数据库名              :  test
SQL ID                :  -8959188754910793169
SQL 语句              :  select * from t1 where id1 = $1 union select * from t1 where id1 = $2
-----------------------------------------------------------------------------------------------------------
发现段 (2 个发现)
-----------------------------------------------------------------------------------------------------------
1- 索引发现 (查看执行计划段)
-----------------------------------------------
        创建一个或多个索引，能够改善SQL语句的执行计划。

        建议(预估收益：99.52%)
        ----------------------------------------
        考虑创建推荐的索引。
        CREATE INDEX ON public.t1 USING hash (id1)

        基本原理
        ---------
        创建推荐的索引能够显著改善SQL语句的执行计划。

2- 改写SQL发现
-------------------------
        在执行计划中发现了代价较高的 "UNION" 操作。

        建议
        --------------
        如果数据允许重复或唯一性有保证，考虑使用 "UNION ALL" 代替 "UNION"。

-----------------------------------------------------------------------------------------------------------
执行计划段
-----------------------------------------------------------------------------------------------------------
1- 原始执行计划
--------------------
Unique  (cost=3386.04..3386.05 rows=2 width=4)
  ->  Sort  (cost=3386.04..3386.05 rows=2 width=4)
        Sort Key: t1.id1
        ->  Append  (cost=0.00..3386.03 rows=2 width=4)
              ->  Seq Scan on t1  (cost=0.00..1693.00 rows=1 width=4)
                    Filter: (id1 = $1)
              ->  Seq Scan on t1 t1_1  (cost=0.00..1693.00 rows=1 width=4)
                    Filter: (id1 = $2)

2- 新索引执行计划
--------------------
Unique  (cost=16.08..16.09 rows=2 width=4)
  ->  Sort  (cost=16.08..16.08 rows=2 width=4)
        Sort Key: t1.id1
        ->  Append  (cost=0.00..16.07 rows=2 width=4)
              ->  Index Scan using <12624>hash_t1_id1 on t1  (cost=0.00..8.02 rows=1 width=4)
                    Index Cond: (id1 = $1)
              ->  Index Scan using <12624>hash_t1_id1 on t1 t1_1  (cost=0.00..8.02 rows=1 width=4)
                    Index Cond: (id1 = $2)

~
~
~
```

5.2　SQL 监控

本章重点介绍 KingbaseES 所支持的 SQL 监控功能。当 SQL 监控功能被启用后，系统将自动跟踪并实时更新所有 SQL 语句的执行状态及其资源占用情况。该功能可有效解决传统环境中对于那些执行时间过长、难以实时追踪其

运行状态的问题。数据库管理员（DBA）能够通过这一功能实时获取每个 SQL 语句在运行过程中的详细指标信息，包括但不限于资源消耗状况、执行时长以及具体的执行计划等关键数据。

5.2.1　SQL 监控的功能

当 SQL 监视器功能处于启用状态时，若满足以下任一条件，则相应的 SQL 语句将被视为受控对象：

- 该 SQL 语句或 PL/SQL 子程序在单次执行过程中至少占用 5 秒的 CPU 时间或 I/O 操作时长。
- SQL 语句并行执行。
- SQL 语句使用 hint`/* +MONITOR */`指定。

5.2.2　SQL 监控的目的

实时 SQL 监视功能用于定位具体语句的执行耗时问题，能够提供关于各查询执行时间及资源使用情况的详细信息。通过这种方式，管理员可以实现对具体 SQL 语句执行成本的有效评估。

实时 SQL 监控的场景包括：

- 频繁执行的 SQL 语句的执行速度比正常情况慢。
- 数据库会话性能降低。
- 并行 SQL 语句需要很长时间。
- 生成 KWR 快照花费的时间比预期的要长得多。

5.2.2.1　SQL 监控的示例

- 预置条件

  使用 SQL 监控的全部功能，需要在 kingbase.conf 里添加以下参数

```
shared_preload_libraries = 'plsql, sys_stat_statements, sys_sqltune'
sql_monitor.track = 'all'
sys_stat_statements.track = 'all'
enable_hint = on
```

  并在数据库中创建插件

```
CREATE EXTENSION sys_sqltune;
```

  创建表和注入数据

```
CREATE TABLE smon_test(id BIGINT);
INSERT INTO smon_test SELECT i FROM generate_series(1, 20000) i;
```

- 自动将满 5s 的语句纳入监控

```
SELECT * FROM sys_sleep(5);
```

- 可使用 hint 避免满 5s 的语句纳入监控

```
SELECT/*+no_monitor*/ * FROM sys_sleep(5);
```

- 使用 hint 将不满 5s 的语句纳入监控

```
SELECT/*+monitor*/ * FROM sys_sleep(1);
```

- 自动将并行语句纳入监控

```
SELECT/*+Parallel(smon_test 1)*/ COUNT(*) FROM smon_test;
```

- 通过查看视图获取语句整体统计信息

```
\x on
SELECT * FROM V$SQL_MONITOR ORDER BY last_ts DESC LIMIT 1;
```

```
-[ RECORD 1 ]--------+-------------------------------------------------------
query                | SELECT/*+Parallel(smon_test 1)*/ count(*) FROM smon_test;
sql_id               | -7195113866045537130
exec_id              | 39748
session_id           | 1684116144.57683
px_server            | -1
px_is_cross_instance | t
px_maxdop            | 1
px_servers_allocated | 1
px_servers_requested | 1
db_id                | 12312
user_id              | 10
query_id             | -5392400042537214696
start_ts             | 2023-05-15 15:31:13.902172+08
last_ts              | 2023-05-15 15:31:13.924597+08
duration             | 22.425
status               | DONE_ALL
statup_time          | 22.39
total_time           | 22.39
tuples_num           | 1
loops_num            | 1
lock_wait_time       | 0
lightlock_wait_time  | 0
buffer_wait_time     | 0
client_wait_time     | 0
extension_wait_time  | 0
timeout_wait_time    | 0
ipc_wait_time        | 0.70632
user_io_wait_time    | 0
other_wait_time      | 0
shared_blks_hit      | 108
shared_blks_read     | 0
shared_blks_dirtied  | 0
shared_blks_written  | 0
local_blks_hit       | 0
local_blks_read      | 0
local_blks_dirtied   | 0
local_blks_written   | 0
temp_blks_read       | 0
temp_blks_written    | 0
blk_read_time        | 0
blk_write_time       | 0
wal_records          | 0
wal_fpi              | 0
wal_bytes            | 0
```

- 通过视图查看执行节点的统计信息

```
\x off
SELECT plan_index, plan_type, plan_object_name, plan_total_cost, total_time, tuples_num
FROM V$SQL_PLAN_MONITOR WHERE sql_id = -7195113866045537130;
```

```
 plan_index | plan_type | plan_object_name | plan_total_cost | total_time | tuples_num
------------+-----------+------------------+-----------------+------------+------------
          0 | Aggregate |                  |              50 |      21.83 |          1
          1 | Gather    |                  |               0 |      16.16 |      20000
          2 | Seq Scan  | public.smon_test |               0 |       3.16 |      11526
          2 | Seq Scan  | public.smon_test |               0 |       0.00 |       8474
(4 rows)
```

- 生成 TEXT 版监控报告

```
SELECT DBMS_SQL_MONITOR.REPORT_SQL_MONITOR(type=>'TEXT');
```

```
                                                    report_sql_monitor
-------------------------------------------------------------------------------------------------------------------------------------
SQL Text
-----------------------------
SELECT/*+Parallel(smon_test 1)*/ count(*) FROM smon_test;


Global Information
-----------------------------
 Status              :  DONE_ALL
 Session ID          :  1684116144.57683
 SQL ID              :  -7195113866045537130
 SQL Execution ID    :  39748
 Execution Started   :  2023-05-15 15:31:13.902172+08
 Last Refresh Time   :  2023-05-15 15:31:13.924597+08
 Duration(s)         :  0.02
 Fetch Calls         :  1

Global Stats
=================================================================
|   Elapsed   |    Cpu     |   Wait   |   Fetch    |   Buffer   |
|   Time(s)   |  Time(s)   |  Time(s) |   Calls    |    Gets    |
=================================================================
|         0.02|        0.02|      0.00|           1|     0 bytes|
=================================================================

Parallel Execution Details (DOP=1 , Servers Allocated=1)
=====================================================================
| Server |   Elapsed   |     Cpu     |    Wait   |    Fetch    |   Buffer   |
|        |   Time(s)   |   Time(s)   |  Time(s)  |    Calls    |    Gets    |
=====================================================================
|      -1|         0.02|         0.02|       0.00|            1|     0 bytes|
|       0|         0.01|         0.01|       0.00|          8 K|     0 bytes|
=====================================================================

SQL Plan Monitoring Details
=====================================================================================================================================
| Id |   Operation               |   Name           |   Rows   |   Cost   |   Cost   |      Time      |     Start      | Execs |   Rows   |
|    |                           |                  | (Estim)  | (statup) | (total)  |   Active(ms)   |   Active(ms)   |       | (Actual) |
=====================================================================================================================================
|  0 | Aggregate                 |                  |        1 |    50.00 |    50.00 |          21.83 |          21.83 |     1 |        1 |
|  1 |  Gather                   |                  |    20 K  |     0.00 |     0.00 |          16.16 |           0.81 |     1 |     20 K |
|  2 |   Seq Scan(Relation Name)| public.smon_test|    12 K  |     0.00 |     0.00 |           3.16 |           0.04 |     1 |     12 K |
=====================================================================================================================================
```

```
SELECT DBMS_SQL_MONITOR.REPORT_SQL_MONITOR_LIST(type=>'TEXT');
```

```
SQL Monitoring List

=================================================================================================================================================================================================================
|   Status   | Duration(s) |        SQL Id        |  Exec Id  |      Session Id      |        Start        |  DB Name  | DOP | DB Time(s)|  Wait(s) |            SQL Text            |
=================================================================================================================================================================================================================
| DONE_ALL   |        0.02 | -7195113866045537130|     39869| 1684116144.57683     | 2023-05-15 16:19:46 | test      |   1 |      0.02 |     0.00 |SELECT/*+Parallel(smon_test 1)*/ c|
| DONE_ALL   |        1.00 |  6642828457661277803|     39868| 1684116144.57683     | 2023-05-15 16:19:35 | test      |   0 |      1.00 |     1.00 |select/*+monitor*/ * from pg_sleep|
| DONE_ALL   |        5.01 |  4175071384139695159|     39866| 1684116144.57683     | 2023-05-15 16:19:25 | test      |   0 |      5.01 |     5.01 |SELECT * FROM sys_sleep(5);       |
=================================================================================================================================================================================================================
```

- 生成网页版监控报告 (默认路径在 data 目录下)

```
SELECT * FROM  PERF.REPORT_SQL_MONITOR_TO_FILE('1.html',type=>'HTML');
```

SQL 监控详细报告

SQL语句

SELECT/*+Parallel(smon_test 1)*/ count(*) FROM smon_test;

全局信息

名称	实例信息
语句执行状态	DONE_ALL
会话 ID	1684116144.57683
SQL ID	-7195113866045537130
语句执行 ID	39748
语句执行开始时间	2023-05-15 15:31:14
语句执行最后更新时间	2023-05-15 15:31:14
语句执行总时间	0.02 s
语句返回行数	1

全局统计

执行时间(s)	CPU时间(s)	等待时间(s)	返回元组数	内存使用
0.02	0.02	0.00	1	0 bytes

SQL并行进程

(DOP=1 , Servers Allocated=1)

进程编号	进程执行时间(s)	CPU时间(s)	等待时间(s)	返回元组数	内存使用
-1	0.02	0.02	0.00	1	0 bytes
0	0.01	0.01	0.00	8 K	0 bytes

SQL执行计划节点

执行节点ID	执行节点类型	对象名称	计划			实际			
			元组数	总成本	启动成本	总时间(ms)	启动时间(ms)	执行次数	元组数
0	Aggregate		1	50.00	50.00	21.83	21.83	1	1
1	Gather		20 K	0.00	0.00	16.16	0.81	1	20 K
2	Seq Scan	public.smon_test	12 K	0.00	0.00	3.16	0.04	1	12 K

```
SELECT DBMS_SQL_MONITOR.REPORT_SQL_MONITOR_LIST(type=>'HTML');
```

SQL 监控列表

语句执行状态	语句执行时间(s)	SQL ID	执行 ID	会话 ID	用户	数据库	实际并行数	等待时间(s)	内存大小	执行开始时间	执行结束时间	SQL语句
DONE_ALL	0.02	-7195113866045537130	39869	1684116144.57683	xiaotuo	test	1	0.00	0 bytes	2023-05-15 16:19:46	2023-05-15 16:19:46	SELECT/*+Parallel(smon_test 1)*/ count(*) FROM smon_test;
DONE_ALL	1.00	6642828457661277803	39868	1684116144.57683	xiaotuo	test	0	1.00	0 bytes	2023-05-15 16:19:35	2023-05-15 16:19:36	select/*+monitor/ * from pg_sleep(1);
DONE_ALL	5.01	4175071384139695159	39866	1684116144.57683	xiaotuo	test	0	5.01	0 bytes	2023-05-15 16:19:25	2023-05-15 16:19:30	SELECT * FROM sys_sleep(5);

- 生成英文网页版监控报告

```
set sql_monitor.language = 'english';
SELECT DBMS_SQL_MONITOR.REPORT_SQL_MONITOR(type=>'HTML');
```

SQL Monitoring Report

SQL TEXT

SELECT/*+Parallel(smon_test 1)*/ count(*) FROM smon_test;

Global Information

Name	Instance Info
Status	DONE_ALL
Session ID	1684116144.57683
SQL ID	-7195113866045537130
SQL Execution ID	39748
Execution Started	2023-05-15 15:31:14
Last Refresh Time	2023-05-15 15:31:14
Duration	0.02 s
Fetch Calls	1

Global Stats

Elapsed Time(s)	Cpu Time(s)	Wait Time(s)	Fetch Calls	Buffer Gets
0.02	0.02	0.00	1	0 bytes

Parallel Execution Details

(DOP=1 , Servers Allocated=1)

Server	Elapsed(s)	Cpu Time(s)	Wait time(s)	Fetch Calls	Buffer Gets
-1	0.02	0.02	0.00	1	0 bytes
0	0.01	0.01	0.00	8 K	0 bytes

SQL Plan Monitoring Details

Id	Operation	Name	Plan			Active			
			Rows	total Cost	statup Cost	Time(ms)	Start(ms)	Execs	Rows
0	Aggregate		1	50.00	50.00	21.83	21.83	1	1
1	Gather		20 K	0.00	0.00	16.16	0.81	1	20 K
2	Seq Scan	public.smon_test	12 K	0.00	0.00	3.16	0.04	1	12 K

注意:

报告中 SQL 语句或 SQL TEXT 最多显示 200 个字符。

5.2.3　SQL 监控的外部接口

5.2.3.1　SQL 监控参数

全部相关参数

参数名称	类型	说明
sql_monitor.track	会话级	SQL 语句监控层级，ENUM 类型，默认值为 none top：监控顶层 SQL 语句 all：监控嵌套层数小于 64 层的 SQL 语句 none：不监控
sql_monitor.max	系统级	视图最大数据量，整数，默认 1000，最小 100 当系统内的统计结果数据量达到预设上限时，旧的统计信息将被自动移除。这一过程旨在释放部分存储空间，以确保新监控的 SQL 语句能够正常记录其相关信息。
sql_monitor.track_options	会话级	采集统计信息控制参数，ENUM 类型，默认为‘basic’basic：采集语句全部统计信息和执行计划节点基本统计信息 full ：在‘basic’监控级别基础上，系统可进一步采集执行计划节点中的 buffer 及 WAL 相关统计信息。然而，此扩展功能会带来一定的性能消耗。
sql_monitor.language	会话级	网页版监控报告使用语言，TEXT 类型，默认为中文（chinese 或 chn），可选为英文 (english 或 eng) chinese/chn：网页版报告输出中文 english/eng：网页版报告输出英文

5.2.3.2　DBMS_SQL_MONITOR 子程序

下表列出了 DBMS_SQL_MONITOR 所提供的子程序。DBMS_SQL_MONITOR 包信息参考《PLSQL 系统包和类型参考手册》中的 DBMS_SQL_MONITOR 章节。

DBMS_SQL_MONITOR 包仅支持在数据库 Oracle 模式下使用，perf 前缀的 SQL 监控相关函数支持在所有数据库模式下使用（包括 oracle）。

子程序名称	说明
SQL_MONITOR_RESET 函数	清空视图数据
REPORT_SQL_MONITOR 函数	返回监控详细报告
REPORT_SQL_MONITOR_LIST 函数	返回监控列表报告
REPORT_SQL_MONITOR_TO_FILE 函数	将监控详细报告写入磁盘指定路径
REPORT_SQL_MONITOR_LIST_TO_FILE 函数	将监控列表报告写入磁盘指定路径

5.2.3.3　SQL 监控视图

下面列出了 SQL 监控相关的动态视图。更多视图信息参考《KingbaseES 插件参考手册》中的 sys_sqltune 章节。

子程序名称	说明
V$SQL_MONITOR	此视图包含 SQL 语句的语句级统计信息。与 sys_stat_statements 不同，SQL 监控功能的统计信息不进行累积。具体而言，本视图中的每个条目仅反映相应 SQL 语句单次执行的情况。当 SQL 语句采用并行查询模式时，主进程及其每个并行 worker 在每次执行过程中均会产生独立的条目。尽管如此，所有条目仍共享相同的 session_id 和 exec_id。
V$SQL_PLAN_MONITOR	此视图包含 SQL 语句的执行计划中每个步骤的监控统计信息。当 SQL 语句被执行时，本视图中为每个被监控的语句均生成多个相关条目。每个条目代表该语句执行计划中的一个具体步骤。

第6章　使用 KDB_HINT_PLAN（已废弃）

6.1　概述

尽管查询优化器会生成最优的执行计划，但由于受到多种因素的限制，该过程仍存在固有缺陷和局限性。因此，在很多情况下，优化器基于其依赖的统计信息所提供的建议可能并非最佳选择，需要进行人工干预以实现更优的结果。

为了解决这一问题，KDB_HINT_PLAN 应运而生。该功能允许用户对执行计划进行人工干预和优化。

注意:

本功能自 KingbaseES V8R6C3 版本起已进入维护阶段不再继续开发，故不再继续推荐使用。我们从 V8R6C4 版本开始对该功能进行了全面增强和改进，详细说明请参照本手册章节使用 HINT

6.2　KDB_HINT_PLAN 的功能

根据 HINT 注释所作用的对象类型及其对执行计划的影响机制，可将其划分为以下六类：

- Scan methods
- join methods
- join order
- row number correction
- parallel query configuration
- GUC setting

6.2.1 Scan method

Scan method 相关的 HINT 注释仅适用于以下类型的表：普通表、继承表、UNLOGGED 表、临时表和系统表。需要注意的是，该 HINT 信息的参数值只能指定单个表名。此外，支持在一个查询中对多个目标表分别施加 Scan method HINT 注释。

- SeqScan(table)：强制在表上使用顺序扫描。
- TidScan(table)：强制在表上使用 Tid 扫描。
- IndexScan(table[index…])：强制在表上使用索引扫描，只限制指定的索引。
- IndexOnlyScan(table[index…])：强制在表上使用 Indexonly scan，限制特定的索引。当 Indexonlyscan 不可用时，可以使用索引扫描。
- BitmapScan(table[index…])：强制在表上使用位图扫描。
- NoSeqScan(table)：强制不在表上使用顺序扫描。
- NoTidScan(table)：强制不在表上使用 Tid 扫描。
- NoIndexScan(table)：强制不在表上使用索引扫描和 indexonlyscan。
- NoIndexOnlyScan(table)：强制不在表上使用 indexonlyscan。
- NoBitmapScan(table)：强制不在表上使用位图索引。

6.2.2 Join method

Join method 相关的 HINT 注释支持作用于以下对象类型：普通表、继承表、UNLOGGED 表、临时表、外部表、系统表、表函数、常量值命令结果集和通用表达式（CTE）。HINT 括号内参数的顺序不会对执行计划的执行顺序产生影响。

- NestLoop(table table[table…])：在对指定的表进行连接时，强制使用循环嵌套连接。
- HashJoin(table table[table…])：在对指定的表进行连接时，强制使用散列连接。
- MergeJoin(table table[table…])：在对指定的表进行连接时，强制使用排序合并连接。
- NoNestLoop(table table[table…])：在对指定的表进行连接时，强制不使用循环嵌套连接。
- NoHashJoin(table table[table…])：在对指定的表进行连接时，强制不使用散列连接。
- NoMergeJoin(table table[table…])：在对指定的表进行连接时，强制不使用排序合并连接。

6.2.3 Join order

Leading 短语括号内的参数必须指定至少两个且不重复的表。需要注意的是，在实际应用中，支持通过圆括号嵌套的方式来定义具体的执行顺序。但需要特别指出的是：当使用 UNION 操作符对多个 SELECT 子句进行合并时，无法利用 Leading 短语来干预其内部连接的执行顺序。

- Leading(table table[table…])：强制使用指定的连接顺序。
- Leading(<join pair>)：强制使用指定的连接顺序和方向。其中一个 joinpair 可以是一对表和/或者用括号括起来的可以构成嵌套结构的其他连接对。

6.2.4 Row number correction

Rows 短语的圆括号内必须包含至少两个表。需要注意的是，在处理多个不相关连接时，只有最左侧的连接会被生效。即使查询中存在 where 子句，也不会对结果修正产生任何影响。

- Rows(table table[table…]correction)：修正特定的表连接后的结果的行数。

  可用的修正方法有 absolute(#<n>)，addition (+<n>)，subtract (-<n>) 和 multiplication (* <n>)，<n> 必须是函数 strtod() 可以读取的字符串。

6.2.5 Parallel query configuration

HINT 注释仅适用于以下对象类型：普通表、继承表、UNLOGGED 表和系统表。对于外部表、表函数、常量值子句、通用表达式（CTE）、视图和子查询，此类 HINT 信息将不会产生任何作用。

- Parallel(table <# of workers>[soft|hard])：在指定的表上强制执行或者禁止并行执行。
  - <# of workers> 是所需的并行 workers 数，0 代表禁止并行执行。
  - 如果第三个参数是 soft (默认值)，仅会改变 max_parallel_workers_per_gather 的值。
  - Hard 表示把 workers 数强制为指定的值。

6.2.6 GUC setting

Set 短语支持一次性修改多个相关参数。需要注意的是，当有多个相同类型的 HINT 被应用到同一 GUC 参数时，将遵循最后指定的原则。所有参数调整仅在当前查询范围内有效。

- Set(GUC-param value)：在优化器运行时，将这个 GUC 参数设置为指定的值。

kdb_hint_plan 也可以用于在 PLSQL 脚本中的查询，但有一定的限制。仅适用于返回一行的查询 (SELECT, INSERT, UPDATE, DELETE)、返回多行的查询 (RETURN QUERY)、动态 SQL 语句 (EXECTE)、Cursor 开启 (OPEN)、循环查询的结果 (FOR)。在 PLSQL 中 HINT 注释需要放在 SELECT 与目标列之间。

6.3 KDB_HINT_PLAN 的使用

HINT 是以插件形式提供给用户的。使用前，需要通过 SQL 语句 `CREATE EXTENSION kdb_hint_plan` 来创建 HINT 拓展，然后使用 `LOAD 'kdb_hint_plan'`命令来加载并激活，需要注意的是，通过此种方式使用插件仅在当前会话生效。

另外，也可以使用以下两种 SQL 语句，实现用户和数据库级自动加载:

```
alter user kingbase set session_preload_libraries = 'kdb_hint_plan';
alter database test set session_preload_libraries='kdb_hint_plan';
```

通过将参数 shared_preload_libraries 的值修改为’kdb_hint_plan’，然后重启数据库服务来使 kdb_hint_plan 生效，以此实现 cluster 级的加载。

kdb_hint_plan 通过在目标 SQL 语句给出的特殊形式的注释来读取 HINT 注释。注释的形式以字符序列`'/*+'`开头以`'*/'`结尾，例如`/*+SeqScan (tablename)*/`。此外，hints 短语会区别其中对象名称的大小写，即大小写敏感。

提示:

除了在 SQL 语句前显式指定 HINT 注释外，还可以通过 HINT_TABLE 实现隐式的规则映射。这种方法可以在匹配到特定的语句时，自动加载预先设定的优化规则，无需对应用程序进行任何修改。

- 配置参数
  - 参数 kdb_hint_plan.enable_hint 默认开启，当此参数关闭后，SQL 查询语句中的 HINT 注释将不会对执行计划产生影响。
  - 当参数 kdb_hint_plan.debug_print 设置为 on，并且 kdb_hint_plan.message_level 设置为 log 后，在一个带有正确格式的 HINT 注释的 SQL 语句运行时，会一同输出 HINT 的调试信息，内容包含 usedhint、not used hint、duplication hint、error hint。

Parameter name	discription	Default
kdb_hint_plan.enable_hint	启用 kdb_hint_plan。	on
kdb_hint_plan.enable_hint_table	True enbles hinting by table。	off
kdb_hint_plan.parse_messages	指定 hints 解析错误的日志级别。可用的值 error, warning, notice, info, log, debug。	INFO
kdb_hint_plan.debug_print	控制 debug 信息打印和详细程度。可用的值为 off, on, detailed and verbose。	off
kdb_hint_plan.message_level	指定 debug 打印的信息级别。可用的值为 error, warning, notice, info, log, debug。	log

当运行 `create extension kdb_hint_plan` 时，将会自动创建一个名为 hint_plan 的模式，并在此模式下建立用于存储提示信息的 hints 表。该表的所有权归属于扩展创建者，其他用户仅具有该表的查询权限。

column	description
id	用于标识每行的唯一编号。此列按顺序自动填充。

-续表-

column	description
norm_query_string	HINT 注释影响的目标 sql 查询语句。查询语句中的常量必须替换为'?'
application_name	要应用 hint 的会话的 application_name 值
hints	使用的 HINT 注释。其中不包括注释标记/*+ 和 */

hints 表的启用状态由参数 kdb_hint_plan.enable_hint_table 控制。无论该参数当前设为 off 还是 on，表的所有者均可以向其中添加记录。需要注意的是，只有当 GUC 参数 kdb_hint_plan.enable_hint_plan 的值设为 on 时，表中的数据才会真正生效。一旦数据生效，norm_query_string 列中的查询语句将受到同一行 hints 列中 hints 短语的影响。

- 示例
  - 强制在扫描时使用 Seqscan

```
CREATE TABLE t1 (id INT PRIMARY KEY, val INT);
CREATE TABLE t2 (id INT PRIMARY KEY, val INT);
INSERT INTO t1 SELECT i, i % 100 FROM (SELECT generate_series(1, 10000) i) t;
INSERT INTO t2 SELECT i, i % 10 FROM (SELECT generate_series(1, 1000) i) t;
ANALYSE;
load 'kdb_hint_plan';
CREATE extension kdb_hint_plan;
set kdb_hint_plan.enable_hint to on;
```

使用`/*+SeqScan(T1) SeqScan(T2)*/`让表 t1、t2 扫描时使用顺序扫描

```
/*+SeqScan(T1) SeqScan(T2)*/
EXPLAIN ANALYSE SELECT * FROM t1, t2 WHERE t1.id = t2.id;
                     QUERY PLAN
-----------------------------------------------------------------------------------------
Hash Join (cost=27.50..220.00 rows=1000 width=16) (actual time=1.354..6.377 rows=1000 loops=1)
    Hash Cond: (t1.id = t2.id)
    -> Seq Scan on t1 (cost=0.00..145.00 rows=10000 width=8) (actual time=0.030..2.046 rows=10000
loops=1)
    -> Hash (cost=15.00..15.00 rows=1000 width=8) (actual time=1.299..1.299 rows=1000 loops=1)
        Buckets: 1024 Batches: 1 Memory Usage: 48kB
        Seq Scan on t2 (cost=0.00..15.00 rows=1000 width=8) (actual time=0.016..0.579 rows=1000
loops=1)
Planning time: 0.635 ms
Execution time: 6.606 ms
```

```
(8 行记录)
```

– 强制在连接时使用 MergeJoin

```
CREATE TABLE t3 (id INT PRIMARY KEY, val INT);
CREATE TABLE t4 (id INT PRIMARY KEY, val INT);
INSERT INTO t3 SELECT i, i FROM (SELECT generate_series(1, 100) i) t;
INSERT INTO t4 SELECT i, i FROM (SELECT generate_series(1, 10) i) t;
ANALYSE;
load 'kdb_hint_plan';
set kdb_hint_plan.enable_hint to on;
```

使用`/*+MergeJoin(T4 T1 T2 T3)*/`让表 t1,t2,t3,t4 连接时使用 mergejoin。

```
/*+MergeJoin(T4 T1 T2 T3)*/
EXPLAIN ANALYSE SELECT * FROM t1, t2, t3, t4 WHERE t1.id = t2.id AND t1.id = t3.id AND t1.id = t4.id;
                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------
---------------------------
Merge Join (cost=1.97..4.34 rows=1 width=32) (actual time=0.178..0.313 rows=10 loops=1)
    Merge Cond: (t1.id = t3.id)
    ->Merge Join (cost=0.56..90.36 rows=1000 width=16) (actual time=0.098..0.130 rows=11 loops=1)
        Merge Cond: (t1.id = t2.id)
        -> Index Scan using t1_pkey on t1 (cost=0.29..318.29 rows=10000 width=8) (actual time=0.058..0.
069 rows=11 loops=1)
        -> Index Scan using t2_pkey on t2 (cost=0.28..43.27 rows=1000 width=8) (actual time=0.027..0.
036 rows=11 loops=1)
    -> Materialize (cost=1.41..2.82 rows=10 width=16) (actual time=0.074..0.110 rows=10 loops=1)
        -> Merge Join (cost=1.41..2.80 rows=10 width=16) (actual time=0.066..0.095 rows=10 loops=1)
            Merge Cond: (t3.id = t4.id)
            -> Index Scan using t3_pkey on t3 (cost=0.14..13.64 rows=100 width=8) (actual time=0.010..
0.022 rows=11 loops=1)
            -> Sort (cost=1.27..1.29 rows=10 width=8) (actual time=0.050..0.056 rows=10 loops=1)
                Sort Key: t4.id
                Sort Method: quicksort Memory: 25kB
                -> Seq Scan on t4 (cost=0.00..1.10 rows=10 width=8) (actual time=0.018..0.024 rows=10
loops=1)
Planning time: 2.077 ms
Execution time: 0.546 ms
(16 行记录)
```

– 强制指定多表连接时的连接顺序

使用`/*+Leading(((T1 T2) T3))*/`指定表 t1、t2、t3 的连接顺序

```
/*+Leading(((T1 T2) T3))*/
EXPLAIN (COSTS false) SELECT * FROM t1, t2, t3 WHERE t1.id = t2.id AND t2.val = t3.val AND t1.id < 10;
                    QUERY PLAN
-------------------------------------------------------------------
Nested Loop
    Join Filter: (t2.val = t3.val)
    -> Hash Join
        Hash Cond: (t1.id = t2.id)
        -> Index Scan using t1_pkey on t1
            Index Cond: (id < 10)
        -> Hash
            -> Seq Scan on t2
    -> Seq Scan on t3
(9 行记录)
```

– 修正连接操作的行数

```
/*+Rows(A B #10)*/ SELECT... ; 把连接后的行数设定为 10
/*+Rows(A B +10)*/ SELECT... ; 从行数中增加 10
/*+Rows(A B -10)*/ SELECT... ; 从行数中减去 10
/*+Rows(A B *10)*/ SELECT... ; 将行数放大 10 倍

/*+Rows(T1 T3 T2 #22)*/
EXPLAIN SELECT * FROM t1 JOIN t2 ON (t1.id = t2.id) JOIN t3 ON (t3.id = t2.id);
                            QUERY PLAN
-------------------------------------------------------------------------------
Merge Join (cost=5.88..7.40 rows=22 width=24)
    Merge Cond: (t1.id = t3.id)
    -> Merge Join (cost=0.56..90.36 rows=1000 width=16)
        Merge Cond: (t1.id = t2.id)
        -> Index Scan using t1_pkey on t1 (cost=0.29..318.29 rows=10000 width=8)
        -> Index Scan using t2_pkey on t2 (cost=0.28..43.27 rows=1000 width=8)
    -> Sort (cost=5.32..5.57 rows=100 width=8)
        Sort Key: t3.id
        -> Seq Scan on t3 (cost=0.00..2.00 rows=100 width=8)
(9 行记录)
```

– 扫描时强制执行并行执行配置

```
EXPLAIN SELECT/*+Parallel(t1 3 hard) Parallel(t2 5 hard)*/ t2.id FROM t1 JOIN t2 ON (t1.val=t2.val);

                            QUERY PLAN
-----------------------------------------------------------------------
Hash Join (cost=125.00..1252.50 rows=100000 width=4)
    Hash Cond: (t2.val = t1.val)
```

```
    -> Gather (cost=0.00..0.00 rows=1000 width=8)
        Workers Planned: 5
        -> Parallel Seq Scan on t2 (cost=0.00..0.00 rows=417 width=8)
    -> Hash (cost=0.00..0.00 rows=10000 width=4)
        -> Gather (cost=0.00..0.00 rows=10000 width=4)
            Workers Planned: 3
            -> Parallel Seq Scan on t1 (cost=0.00..0.00 rows=3226 width=4)
(9 rows)
```

– 优化器运行时修改相关的 GUC 参数

```
/*+Set(random_page_cost 2.0)*/
SELECT * FROM t1 WHERE id = 20;
```

– 在 PLSQL 中的使用

```
CREATE FUNCTION testfunc() RETURNS RECORD AS $$
DECLARE
ret record;
BEGIN
SELECT /*+SeqScan(T1)*/ * INTO ret FROM t1 LIMIT 1;
RETURN ret;
END;
$$ LANGUAGE plsql;

SELECT testfunc();
LOG: kdb_hint_plan:
used hint:
SeqScan(t1)
not used hint:
duplication hint:
error hint:
CONTEXT: SQL statement "SELECT /*+SeqScan(t1)*/ * FROM t1 LIMIT 1"
PLSQL function testfunc() line 5 at SQL statement
testfunc
----------
(1,1)
(1 row)
```

– 参数 kdb_hint_plan.enable_hint_table 的使用

```
set kdb_hint_plan.enable_hint_table to on;
show kdb_hint_plan.enable_hint_table;

kdb_hint_plan.enable_hint_table
--------------------------------
```

```
on
(1 行记录)

EXPLAIN ANALYSE SELECT * FROM t1 WHERE t1.id = 10;
                            QUERY PLAN
-----------------------------------------------------------------------------------------------------
-----
Index Scan using t1_pkey on t1 (cost=0.29..8.30 rows=1 width=8)(actual time=0.018..0.020 rows=1
loops=1)
    Index Cond: (id = 10)
Planning time: 0.127 ms
Execution time: 0.051 ms
(4 行记录)

INSERT INTO hint_plan.hints(norm_query_string, application_name, hints) VALUES ('EXPLAIN ANALYSE SELECT
* FROM t1 WHERE t1.id = ?;',' ','SeqScan(t1)');
EXPLAIN ANALYSE SELECT * FROM t1 WHERE t1.id = 10;
                        QUERY PLAN
-----------------------------------------------------------------------------------------------
Seq Scan on t1 (cost=0.00..170.00 rows=1 width=8) (actual time=0.027..18.739 rows=1 loops=1)
    Filter: (id = 10)
    Rows Removed by Filter: 9999
Planning time: 0.217 ms
Execution time: 18.773 ms
(5 行记录)）
```

版权声明

中电科金仓（北京）科技股份有限公司（简称：电科金仓）版权所有，并保留对本手册及本声明的一切权利。

未得到电科金仓的书面许可，任何人不得以任何方式或形式对本手册内的任何部分进行复制、摘录、备份、修改、传播、翻译成其他语言、将其全部或部分用于商业用途。

免责声明

本手册内容依据现有信息制作，由于产品版本升级或其他原因，其内容有可能变更。电科金仓保留在没有任何通知或者提示的情况下对手册内容进行修改的权利。

本手册仅作为使用指导，电科金仓在编写本手册时已尽力保证其内容准确可靠，但并不确保手册内容完全没有错误或遗漏，本手册中的所有信息也不构成任何明示或暗示的担保。

技术支持

- 电科金仓官方网站：http://www.kingbase.com.cn/
- 电科金仓文档中心：http://help.kingbase.com.cn/
- 全国服务热线：400-601-1188
- 电科金仓技术支持与反馈信箱：support@kingbase.com.cn

服务周期承诺

由于市场需求在不断变化，技术创新和发展的进程不断加剧，产品的版本更迭不可避免。电科金仓对于产品版本生命周期的有效管理，有助于您提前规划项目，更好地从产品服务终止上过渡。

表 3: KingbaseES 产品生命周期里程碑

关键里程碑点	定义
产品发布日期	产品正式发布版本，即 GA（general availability）版本的发布日期。
停止销售日期	正式停止销售的日期，版本停止接受订单日。该日之后，产品将不再销售。
停止功能升级日期	在该日期之后，不再提供新特性和新硬件支持。但依旧提供错误修复、安全修复、功能维护等服务。
停止功能维护日期	在该日期之后，不再维护功能，修复问题。但依旧提供安全修复等服务
停止安全维护日期	在该日期之后，不再发布补丁版本修复中高风险漏洞，仅提供有限的支持。
产品服务终止日期	停止提供产品服务和支持的日期。包括软件维护版本，缺陷修复，以及针对该产品的所有服务支持（包括服务热线和远程/现场支持）。

服务周期策略

金仓数据库管理系统 KingbaseES 产品确保以下的服务周期：

1）产品自发布之日起至产品停止功能升级（包含新特性、新硬件支持）之日止不少于 5 年。

2）产品停止功能升级之日起至产品停止功能维护（主要包括问题修复）之日止不少于 4 年。

3）产品功能维护停止之日起至产品停止安全维护（包括中高风险漏洞修复）之日止不少于 2 年。

针对金仓数据库管理系统 KingbaseES V9 产品（发布日期：2022 年 7 月 15 日），其具体服务周期安排如下：

- 产品销售日期：以产品销售合同所约定日期为准。
- 停止升级日期：自产品销售日期起计算，满 5 年之日止。
- 服务终止日期：自产品销售日期起计算，满 11 年之日止。

服务终止策略

金仓数据库管理系统 KingbaseES 产品确保在销售后，至少提供 6 年的服务支持。

备注:

电科金仓将会综合各方因素来确定产品服务终止日期。并将在实际产品服务终止日期之前至少 90 天，通过公开方式宣布产品服务终止日期。